

You're reading for free via [Fareed Khan's](#) Friend Link. [Become a member](#) to access the best of Medium.

★ Member-only story

AI +

Artificial Intelligence +

Python +

Data Science +

Machine Learning +

Building a 14 MB Micro AI for Watches, Robot Arms and Phones

Open Qwen data, 2-bit weights, and a decoder that cannot spell a tool name wrong

56 min read · Just now



Fareed Khan

Follow



Listen



Share

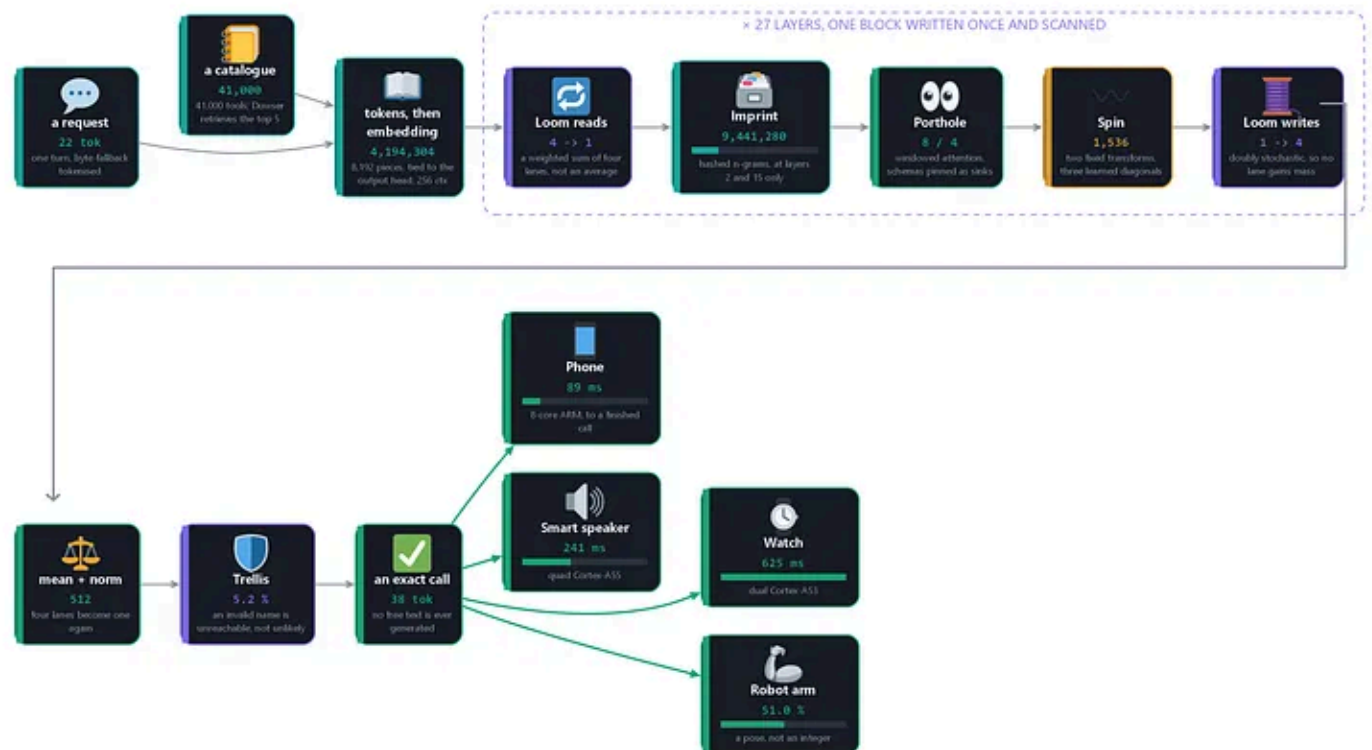
Read this story for free: [link](#)

Phones, watches and robot arms want a model that acts: dim a light, set a timer, grasp a box. The smallest models good at that run **3 billion parameters**, **1.6 GB** at four bits, against a **32 MB** budget for everything. Fifty times over, so you get a cloud model or a local one that cannot call anything. But a model whose only output is a function call needs very little: read a request, pick a tool, copy the arguments out. Here we build one from nothing: **45,211,383 parameters** at **2.225 bits**, **13.44 MB** on disk, **27.6 MB** a session. Four H100s, thirty nine hours.

Quartz

a foundation tool-calling model small enough for the devices that need it most

45.2M parameters	13.44 MB on disk, 2,225 bits	27.6 MB a whole session	63.6 % framed exact-call
---------------------	---------------------------------	----------------------------	-----------------------------



Quartz end to end, with the four residual lanes drawn down the whole stack (Created by [Fareed Khan](#))

Every one of those devices already contains a quartz crystal. A few cents of mineral that does exactly one job, forever, on almost no power, and which nobody markets or benchmarks. That is the brief, and it is where the name comes from.

Here is everything we build:

- **The words:** Scribe, an 8,192 piece tokenizer whose merges never swallow a brace, and Winnow, a 120 billion token corpus, one fifth structured data.
- **The model:** 45,211,383 parameters. Loom, four residual lanes mixed by a doubly stochastic matrix...
- **The training:** fifteen hours on four H100s from random initialisation, a 235 billion parameter Qwen teacher writing 1.2 million tool calls, and two heads.
- **The squeeze:** Grist, which stores every direction of a rotated weight group as a two bit index beside one half precision length, and Ingot, the 405 tensor container.

- **The proof:** Trellis, which makes an invalid tool name impossible rather than unlikely, and Errand, 1,200 device errands.

The architecture comes from a 2026 preprint, [Simple Attention Networks](#), rebuilt part by part in JAX. The whole build is one repository:

GitHub - FareedKhan-dev/quartz: A 45M parameter foundation tool-calling model for phones...

A 45M parameter foundation tool-calling model for phones, wearables, smart home and robots - FareedKhan-dev/quartz

github.com

```
quartz/
├── model/      # architecture, scribe, decode, trellis, grist, ingot, graft
├── train/      # pretrain, sft, qapt, heads, optim: Muon and AdamW
├── data/       # winnow, the filter, and quarry, the teacher
├── errand/     # 1,200 errands, four device families
├── configs/    # base.yaml: the 45M model, d_model 512, 27 layers
                  # tiny.yaml: a 2-layer version for a laptop
```

Nothing Off the Shelf Fits

The obvious approach is to take the smallest model that is good at calling tools,

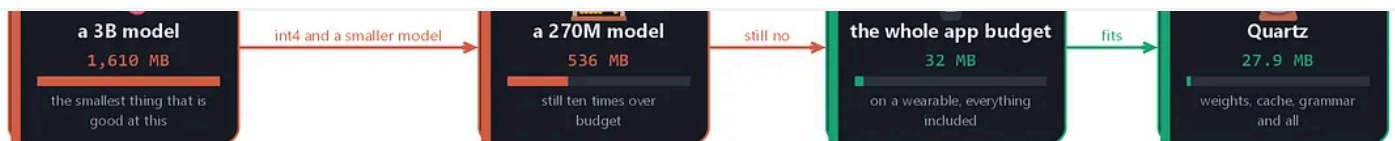
[Open in app](#)

[Sign up](#)

[Sign in](#)

Medium

Search



Three ways of not fitting, and one way of fitting (Created by [Fareed Khan](#))

```
# what one session has to hold, in bytes
def session_bytes(params, bits, kv_positions, kv_per_pos=288, overhead=120e6):
    return params * bits / 8 + kv_positions * kv_per_pos + overhead

for name, params, bits, kv in [
    ("3B, fp16", 3.0e9, 16, 8192),
    ("3B, int4", 3.0e9, 4, 8192),
```

```

("270M, fp16", 2.68e8, 16, 4096)]:
print(f"{name:<12} {session_bytes(params, bits, kv)/1e6:>10,.0f} MB")

print(f"'budget':<12} {32:>10,.0f} MB    a wearable app, everything included")

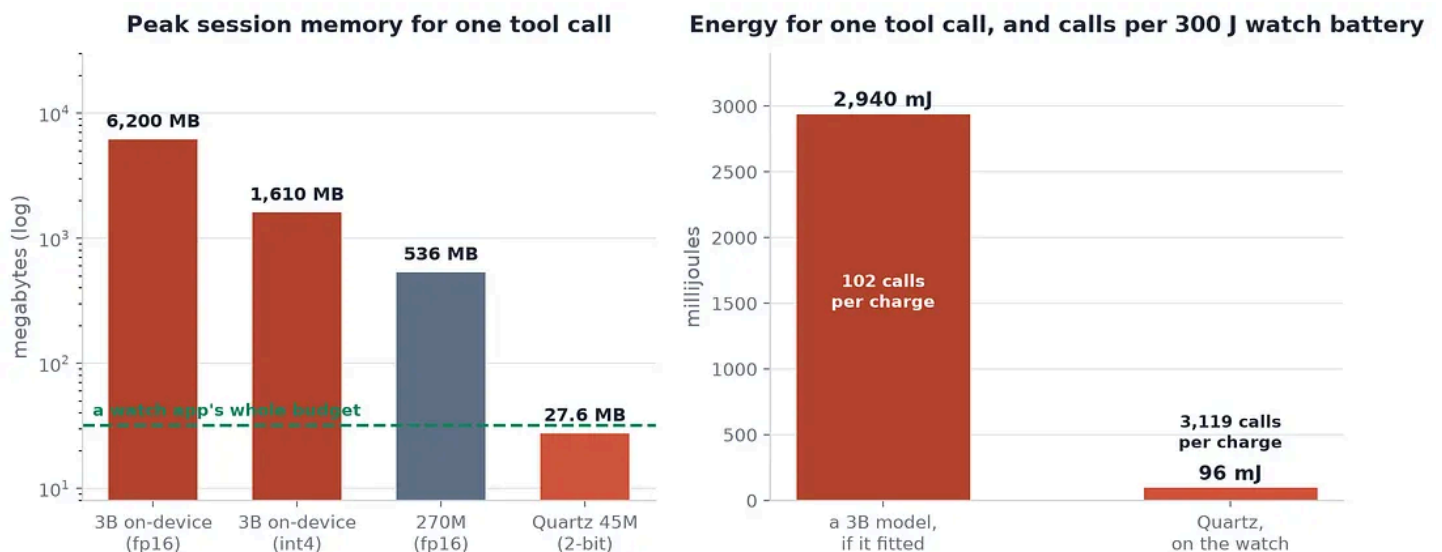
```

```

#### OUTPUT ####
3B, fp16          6,122 MB
3B, int4          1,622 MB
270M, fp16        657 MB
budget            32 MB    a wearable app, everything included

```

A three billion parameter model at four bits is **fifty times over budget**. Even a 270 million parameter model at half precision is twenty times over, before the operating system.



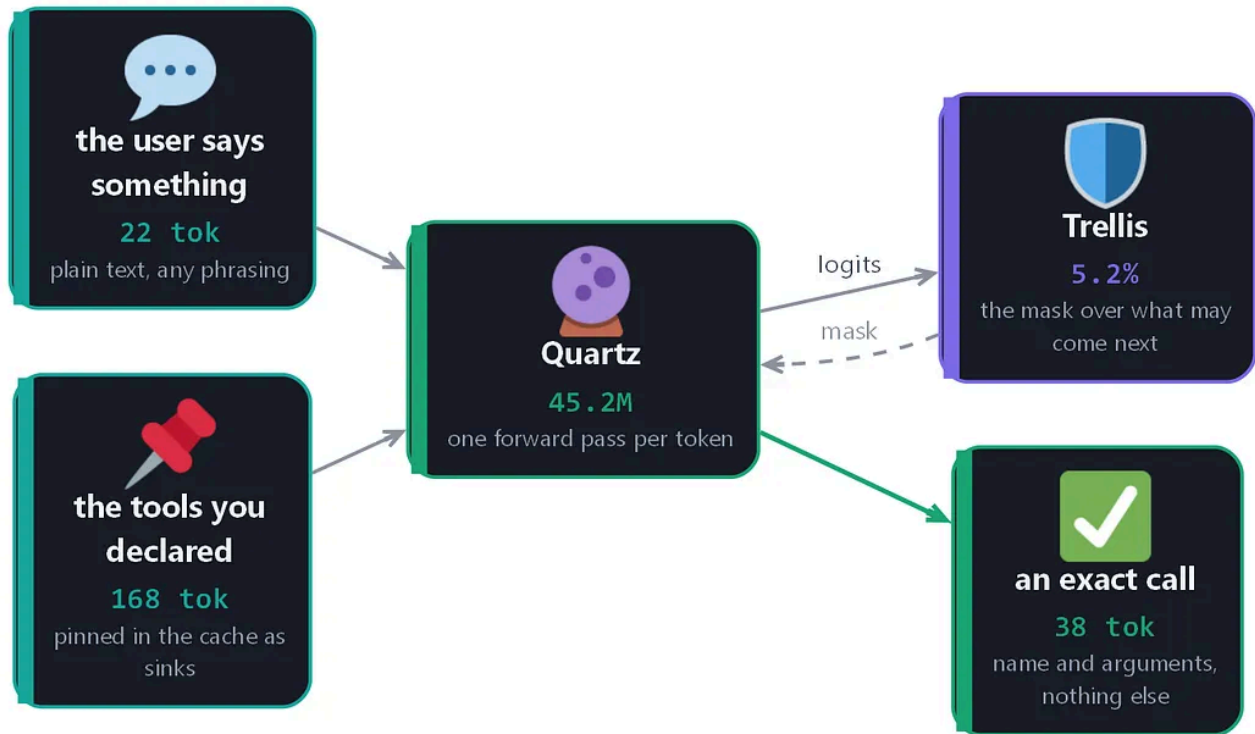
Memory and joules per call, against what a watch spares (Created by [Fareed Khan](#))

Energy is worse, and it is what a user feels. A watch battery holds a few hundred joules for background work. At roughly three joules a call, that model drains it in a hundred calls.

So the question is not how to make a big model smaller. It is how small a model can be and still do this one job.

The One Idea

A general model is ready for anything, and all of that readiness is weight on a device that never asks for it.



Text in, an exact call out, no free text between (Created by [Fareed Khan](#))

Our model has one output shape. It reads a request and emits a call, never prose. With nothing to offer it emits an empty list, the refusal.

```
examples = [  
  [{"name": "set_lights", "arguments": {"room": "kitchen", "brightness": 10}},  
  [{"name": "start_timer", "arguments": {"minutes": 8}}],  
  [{"name": "search_contact", "arguments": {"query": "mum"}},  
  {"name": "send_message", "arguments": {"body": "running late"}}],  
  [],  
]
```

Performing an action and pulling structured data out of a passage are the same operation, and the only difference is what you declared.

The Setup

Everything runs on Python 3.12. Five packages carry the whole system.

```
python -m venv .venv && source .venv/bin/activate  
pip install jax[cuda12] flax optax sentencepiece numpy
```

`jax` and `flax` are the model and the training loops. `optax` is the optimiser chain, though we write our own update rule inside it. `sentencepiece` runs the tokenizer and `numpy` does the packing and the container.

```
import os, json, math, struct
from functools import partial

import numpy as np
import jax
import jax.numpy as jnp
import jax.nn.initializers as jinit
import flax.linen as nn
import optax
import sentencepiece as spm

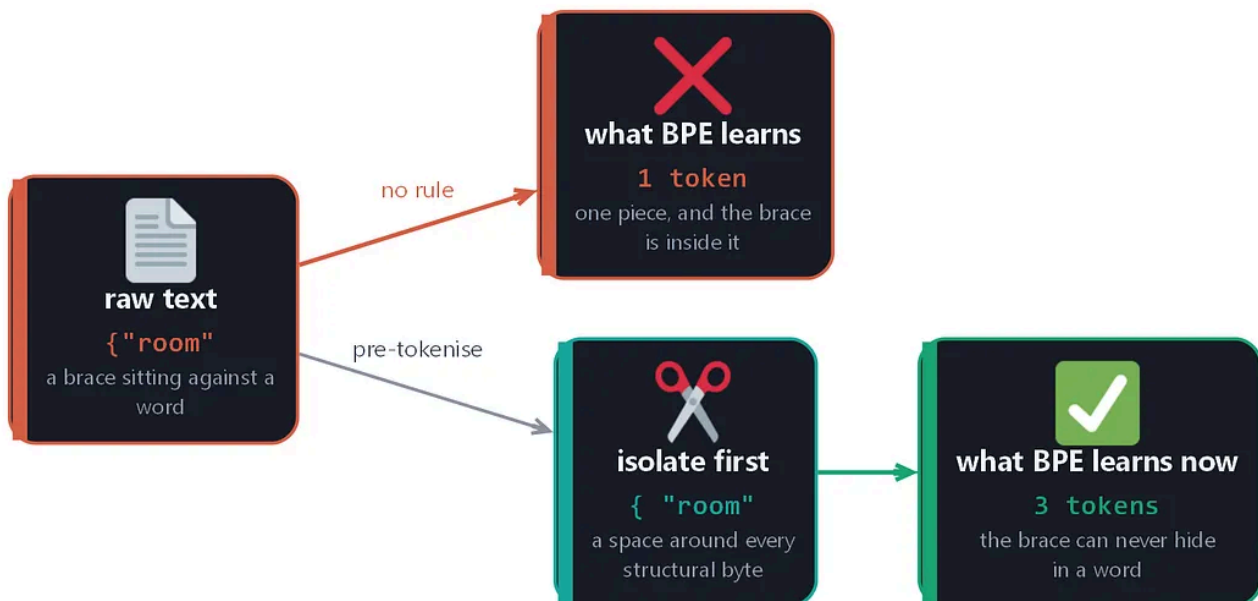
d = jax.devices()[0]
print(f"jax {jax.__version__} flax {__import__('flax').__version__} "
      f"optax {optax.__version__} numpy {np.__version__}")
print(f"{jax.device_count()} x {d.device_kind} "
      f"{d.memory_stats()['bytes_limit']/1e9:.1f} GB {os.cpu_count()} cpu cores")
```

```
#### OUTPUT ####
jax 0.7.2 flax 0.10.6 optax 0.2.4 numpy 2.5.1
4 x NVIDIA H100 80GB HBM3 84.9 GB 64 cpu cores
```

Four cards, about a day and two thirds in total. The finished artifact is under fourteen megabytes.

Scribe, and the Rule That Stops BPE Eating Your Braces

The tokenizer is where most of this post's surprises live. To a model whose entire output is JSON, a brace is not decoration, it is a load bearing structural token. And byte pair encoding will happily learn a merge like `{“room”}`, because that sequence is everywhere in JSON and merging it saves tokens.



A brace hidden inside a word, and a brace that cannot (Created by [Fareed Khan](#))

If `{"room` is one token, the model cannot open a brace without committing to the word `room`, and later we want to mask the *identifier* without touching the *structure*.

The obvious fix is to declare the braces as user defined symbols. That does not work.

```
# user_defined_symbols guarantees a piece EXISTS. It does not stop the trainer
# from also learning merges that CONTAIN that character.
sp = spm.SentencePieceProcessor(model_file="scribe_naive.model")
print(sp.encode('{"room":"kitchen"}', out_type=str))
```

```
#### OUTPUT ####
['_{"', 'room', '":', 'kitchen', '"}']
```

`{"` is a single piece, and so is `":`. Declaring a symbol says nothing about what the trainer may do with it.

The only rule that works is to change the text before the trainer sees it, so the merge is never learnable.

```
# quartz/model/scribe.py
import re

# The eight characters that carry JSON structure. If BPE merges one into a
# longer piece, we can no longer mask an identifier without its punctuation.
ISOLATED = set('{[",,]}')
_PRE_TOK = re.compile(r'([({}\[\]\",,]))')

def pre_tokenize(text: str) -> str:
    """Applied to the fitting corpus and every encode, never to decode, where
    SentencePiece would double the whitespace it already models."""
    return _PRE_TOK.sub(r' \1 ', text)

print(repr(pre_tokenize('{"room":"kitchen","brightness":10}')))
```

```
#### OUTPUT ####
' { " room " : " kitchen " , " brightness " :10 } '
```

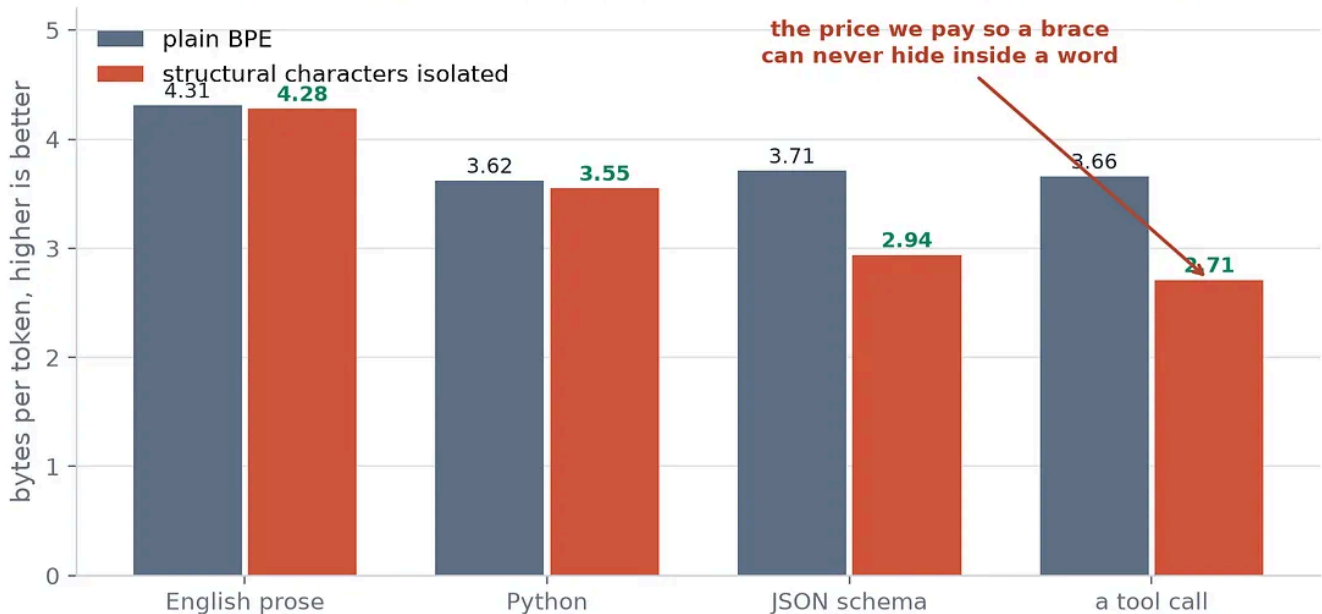
The trainer never sees a brace touching a letter now. Here is the same string through a tokenizer fitted that way.

```
sp = spm.SentencePieceProcessor(model_file="scribe.model")
print(sp.encode(pre_tokenize('{"room":"kitchen}'), out_type=str))
```

```
#### OUTPUT ####
['_{', ' _"', '_room', ' _"', '_:', ' _"', '_kitchen', ' _"', '_}']
```

Every structural character is its own piece, and `room` and `kitchen` are clean identifiers. **This one rule is what makes constrained decoding possible later.**

Isolating braces and quotes costs prose nothing and costs JSON a quarter



What the rule costs prose, and what it costs JSON (Created by [Fareed Khan](#))

It does cost. On English prose the rule changes almost nothing, because braces and quotes are rare there. On a tool call it costs about a quarter of the compression: 3.66 bytes a token becomes 2.71, roughly 35 percent more tokens on the content we care most about. I think that is a good trade.

Fitting the Tokenizer

The training call itself is short.

```
# quartz/model/scribe.py (continued)
PAD_ID, EOS_ID, BOS_ID, UNK_ID = 0, 1, 2, 3

# Order matters. These land on ids 4 through 13, and the codebase asserts
# those positions rather than looking them up.
IM_START, IM_END = "<|im_start|>", "<|im_end|>"
THINK_START, THINK_END = "<think>", "</think>"
TOOLS_START, TOOLS_END = "<tools>", "</tools>"
CALL_START, CALL_END = "<tool_call>", "</tool_call>"
RESULT_START, RESULT_END = "<tool_result>", "</tool_result>"

CHAT_MARKERS = [IM_START, IM_END, THINK_START, THINK_END,
                 TOOLS_START, TOOLS_END, CALL_START, CALL_END,
                 RESULT_START, RESULT_END]

TRAIN_KWARGS = dict(
    model_type="bpe", pad_id=PAD_ID, eos_id=EOS_ID, bos_id=BOS_ID, unk_id=UNK_ID,
    user_defined_symbols=CHAT_MARKERS,
    byte_fallback=True, # so no input is ever <unk>
    character_coverage=0.9999, # the 0.9995 default quietly drops scripts
```

```

normalization_rule_name="identity",    # never rewrite the user's bytes
input_sentence_size=10_000_000,
max_sentence_length=32768,            # the 4192 default truncates long schemas
)

```

Three matter. `normalization_rule_name="identity"` stops the default Unicode rewriting, which would change the user's text and break any claim that an argument was copied verbatim. `byte_fallback=True` turns an unseen character into byte pieces rather than an unknown, and device names are full of characters no corpus contains. And `character_coverage=0.9999` buys the tail of the world's scripts for a few hundred slots.

```

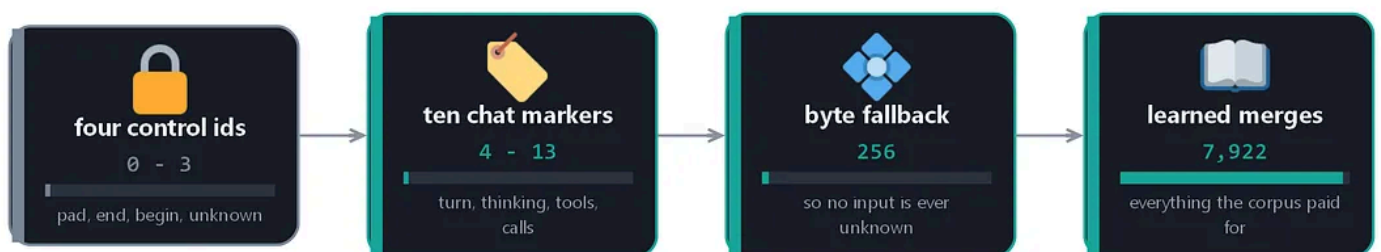
def train_tokenizer(corpus_path, prefix="scribe", vocab_size=8192):
    spm.SentencePieceTrainer.Train(input=corpus_path, model_prefix=prefix,
                                   vocab_size=vocab_size, **TRAIN_KWARGS)
    sp = spm.SentencePieceProcessor(model_file=f"{prefix}.model")
    for i, marker in enumerate(CHAT_MARKERS):
        assert sp.piece_to_id(marker) == 4 + i, f"{marker} is not on id {4+i}"
    print(f" {'vocab':<9} {sp.get_piece_size():,} pieces")
    print(f" {'markers':<9} ids 4 to {3+len(CHAT_MARKERS)} verified")
    return sp

```

```

#### OUTPUT ####
vocab      8,192 pieces
markers    ids 4 to 13 verified

```

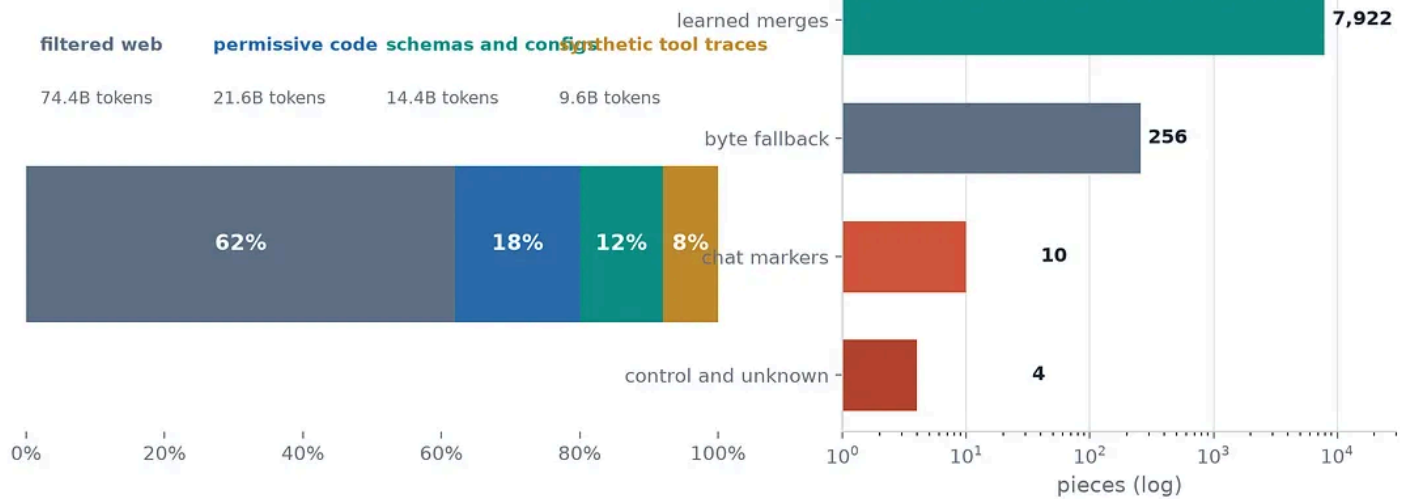


Four control ids, ten markers, 256 bytes, then the corpus (Created by [Fareed Khan](#))

Every other file hardcodes these positions, so a marker that moved would surface as gibberish deep in training rather than here as an error.

Winnow: 120 billion tokens, and a fifth of it is structured

Where the 8,192 tokenizer pieces go

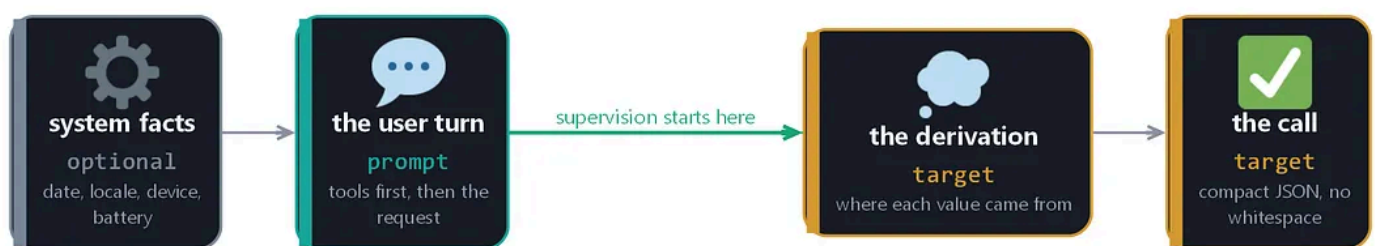


Where the 8,192 pieces go, and where 120 billion tokens come from (Created by [Fareed Khan](#))

Two hundred and seventy of the 8,192 slots are spoken for before any text is read: four control ids, ten markers, 256 byte fallback pieces. The remaining 7,922 are all the corpus buys us.

What One Turn Actually Looks Like

Before the architecture, the wire format, because everything downstream is shaped by it.



Tools first, then supervised reasoning, then the call (Created by [Fareed Khan](#))

```
# quartz/model/scribe.py (continued)
def render(example):
    """The prompt is context we never score, the target is what the loss sees."""
    tools = json.dumps(example.get("tools", []), separators=(",", ":"))
    answers = json.dumps(example.get("answers", []), separators=(",", ":"))
    prompt = (f"{IM_START}user\n{TOOLS_START}{tools}{TOOLS_END}\n"
              + example["query"] + IM_END + "\n" + f"{IM_START}assistant\n")
```

```
target = (f"{THINK_START}{example.get('reasoning','')}{THINK_END}\n"
          + f"{CALL_START}{answers}{CALL_END}{IM_END}")
return prompt, target

print("\n--- target begins ---\n".join(render(EXAMPLES[0])))
```

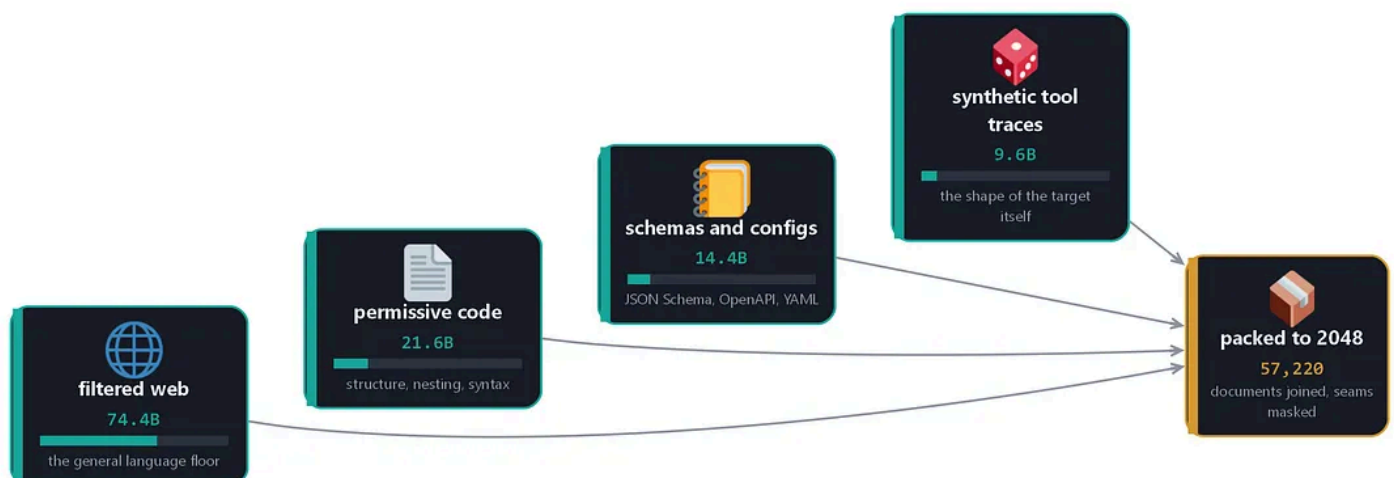
```
#### OUTPUT ####
<|im_start|>user
<tools>[{"name":"set_lights","parameters":{"type":"object","properties":{"room"
dim the kitchen to 10 percent<|im_end|>
<|im_start|>assistant
--- target begins ---
<think>'kitchen' -> room; 'to 10 percent' -> brightness 10</think>
<tool_call>[{"name":"set_lights","arguments":{"room":"kitchen","brightness":10}]
```

The tools sit in the **user** turn, not the system turn, so they stay adjacent to the request. The JSON is dumped with `separators=(",", ":")`, because on a 256 token window the spaces a pretty printer adds are real money.

The reasoning line sits inside the target, so we **train the model to derive each argument out loud before it commits to the call**. One line, never longer, and it separates picking the right tool from filling it in correctly.

Winnow, and Why a Model That Only Writes JSON Has To Read a Lot of JSON

The corpus is where the priors come from, and its shape is unusual.



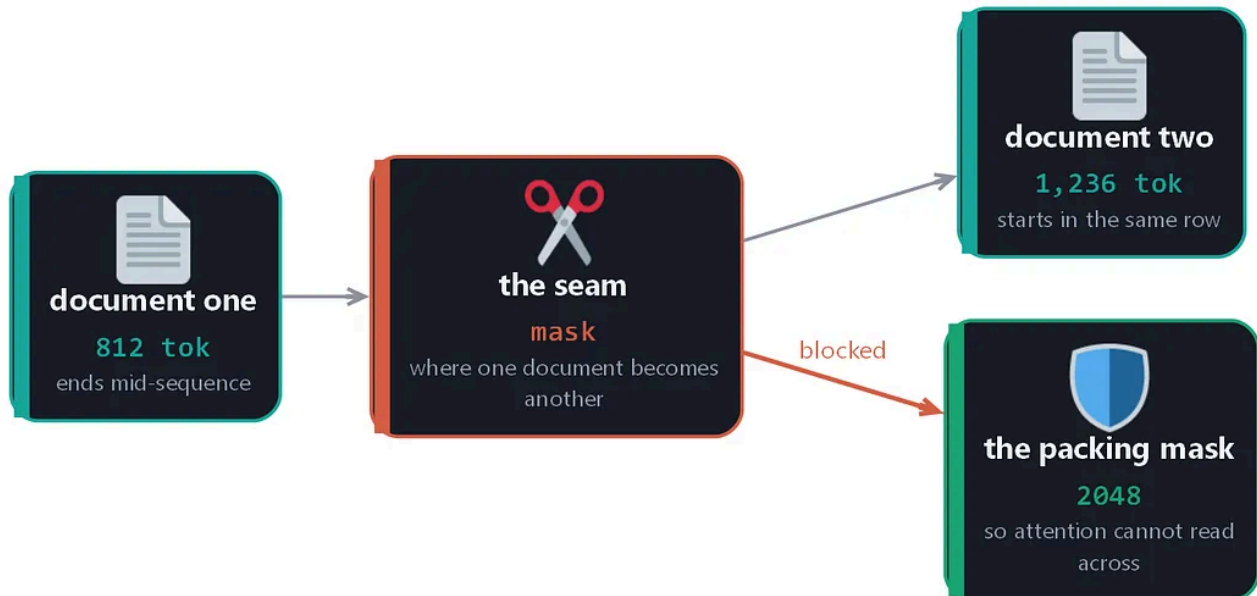
Four sources, a fifth of them structured on purpose (Created by [Fareed Khan](#))


```
# quartz/data/winnow.py
MIX = {
    "fineweb-edu": 0.62,    # the general language floor
    "the-stack-v2": 0.18,  # nesting and matched delimiters
    "schemas": 0.12,      # JSON Schema, OpenAPI, TOML, .proto
    "synth-tools": 0.08,   # the target's own shape
}
TOTAL = 120_000_000_000

for name, share in MIX.items():
    print(f" {name:<14} {share:>6.1%} {share*TOTAL/1e9:>6.1f}B tokens")
print(f" {'total':<14} {sum(MIX.values()):>6.1%} {TOTAL/1e9:>6.1f}B tokens")
```

```
#### OUTPUT ####
fineweb-edu      62.0%      74.4B tokens
the-stack-v2     18.0%      21.6B tokens
schemas          12.0%      14.4B tokens
synth-tools      8.0%       9.6B tokens
total            100.0%     120.0B tokens
```

Twelve percent on JSON Schema and OpenAPI looks eccentric until you remember what this model does. It has to know that "required" is a list, that "type": "integer" forbids a decimal point, and none of that is in web text at any density. The eight percent of synthetic tool traces is the target distribution itself, so stage two refines rather than introduces.



One row, two documents, and the seam between them (Created by [Fareed Khan](#))

Documents are concatenated into fixed 2048 token rows and every token carries a `seg_id`, with 0 for padding so a real document has `seg_id > 0`. Packing saves the third of every batch that padding wastes, and brings one hazard, because a token starting document two attends to the end of document one and learns a transition that never happened.

$$m_{ij} = [\text{seg}_i = \text{seg}_j] \wedge [\text{seg}_i > 0] \wedge [j \leq i]$$

Documents are packed into one sequence, and the mask is what stops the model reading across the seam

The mask that keeps packing honest (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py
def make_packing_mask(seg_ids):
    # '> 0' keeps padding from attending to itself, which would otherwise
    # give an all-zero row and a NaN out of the softmax
    T = seg_ids.shape[1]
    causal = jnp.tril(jnp.ones((T, T), bool))
    same = (seg_ids[:, :, None] == seg_ids[:, None, :]) & (seg_ids[:, :, None]
    return (same & causal[None, :, :])[:, None, :, :])
```

```
segs = np.array([[1, 1, 2, 2, 0]])
for s, row in zip(segs[0], np.asarray(make_packing_mask(jnp.asarray(segs))[0, 0])):
    print(f"{s}  " + " ".join("#" if v else "." for v in row))
```

```
#### OUTPUT ####
1  # . . . .
1  # # . . .
2  . . # . .
2  . . # # .
0  . . . . .
```

Document two starts a fresh triangle and the padding row is empty. **The seam is closed.**

The Two Primitives Everything Else Is Built On

Two small pieces recur in every block. The normaliser is a root mean square norm whose gain is an offset from one, initialised to zero.

$$\text{ZCRMS}(x) = (1 + \gamma) \odot \frac{x}{\sqrt{\text{mean}(x^2) + \varepsilon}}, \quad \gamma \leftarrow 0$$

Zero-centred RMS norm, where the learned gain starts at zero so the layer begins as a pure normaliser

Gain at zero, so the layer starts as a normaliser (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py
class ZCRMSNorm(nn.Module):
    epsilon: float = 1e-6
    dtype: jnp.dtype = jnp.bfloat16

    """A gain written as (1 + gamma) with gamma at zero starts the layer as an
    exact identity in scale, where a gain initialised at 1.0 would be a large
    parameter with no gradient signal to move it."""
    @nn.compact
    def __call__(self, x):
        gamma = self.param("scale", jinit.zeros, (x.shape[-1],))
        # the mean square is in fp32 even in a bf16 model, because a small
        # denominator here becomes a large error
```

```

rms = jnp.sqrt(jnp.mean(x.astype(jnp.float32) ** 2, axis=-1,
                                keepdims=True) + 1e-6)
return ((1 + gamma) * x / rms).astype(jnp.bfloat16)

```

The epsilon sits inside the square root, added to the mean square, not to the root afterwards. That is the stable placement and easy to get backwards.

The positional scheme is rotary, in the half split convention, so dimension i rotates against dimension $i + d/2$ and not its neighbour.

$$x'_i = x_i \cos \theta_t - x_{i+d/2} \sin \theta_t, \quad x'_{i+d/2} = x_{i+d/2} \cos \theta_t + x_i \sin \theta_t, \quad \theta_t = t \Theta^{-2i/d}$$

Rotary positions, pairing each dimension with the one half a head away rather than with its neighbour

Each dimension pairs with the one half a head away (Created by [Fareed Khan](#))

```

# quartz/model/architecture.py (continued)
def rope_tables(head_dim, seq_len, theta=1e5):
    # theta is 1e5 not the usual 1e4, because the larger base decays low
    # frequencies more slowly when a model trained at 2048 runs at 256
    inv = 1.0 / (theta ** (jnp.arange(0, head_dim, 2).astype(jnp.float32) / head_dim))
    angles = jnp.outer(jnp.arange(seq_len).astype(jnp.float32), inv)
    return jnp.cos(angles), jnp.sin(angles)

def apply_rope(x, cos, sin):
    T, half = x.shape[2], x.shape[-1] // 2 # (batch, heads, time, head_dim)
    cos, sin = cos[:T][None, None], sin[:T][None, None]
    x1, x2 = x[..., :half], x[..., half:]
    return jnp.concatenate([x1 * cos - x2 * sin,
                            x2 * cos + x1 * sin], axis=-1).astype(x.dtype)

```

Over a 64 dimensional head and 8 positions, both properties hold. The rotation preserves length, so a query of norm 8.0 comes back at norm 8.0, and it depends only on distance, so positions 0 and 3 give 53.3296, exactly what 4 and 7 give. That is why rotary embeddings work, and why a model trained at 2048 runs at 256 without the attention scores changing character.

Porthole, Attention That Refuses To Grow

Attention is what would normally make memory grow with the conversation, so it is what we bound.

$$o_t = \text{softmax}\left(\frac{q_t K^\top}{\sqrt{d_h}}\right) V \quad \text{over} \quad S \cup [t - W, t]$$

Each token attends to the last W positions and to a pinned set S , which is where the tool schemas live

The last 256 tokens, plus a set that is never evicted (Created by [Fareed Khan](#))

The front half is ordinary grouped-query attention. Eight query heads share four key/value heads to halve the cache, `ZCRMSNorm` on queries and keys keeps bf16 logits from blowing up, and the window lives in the mask, not the storage. The back half is where it gets opinionated.

$$\text{out} = (\text{attn}(x) \odot \sigma(xW_g))W_o, \quad W_o \sim \mathcal{N}\left(0, \frac{0.02^2}{2L}\right)$$

The gate is read from the block input rather than the attention output, and the write-out matrix is scaled by depth

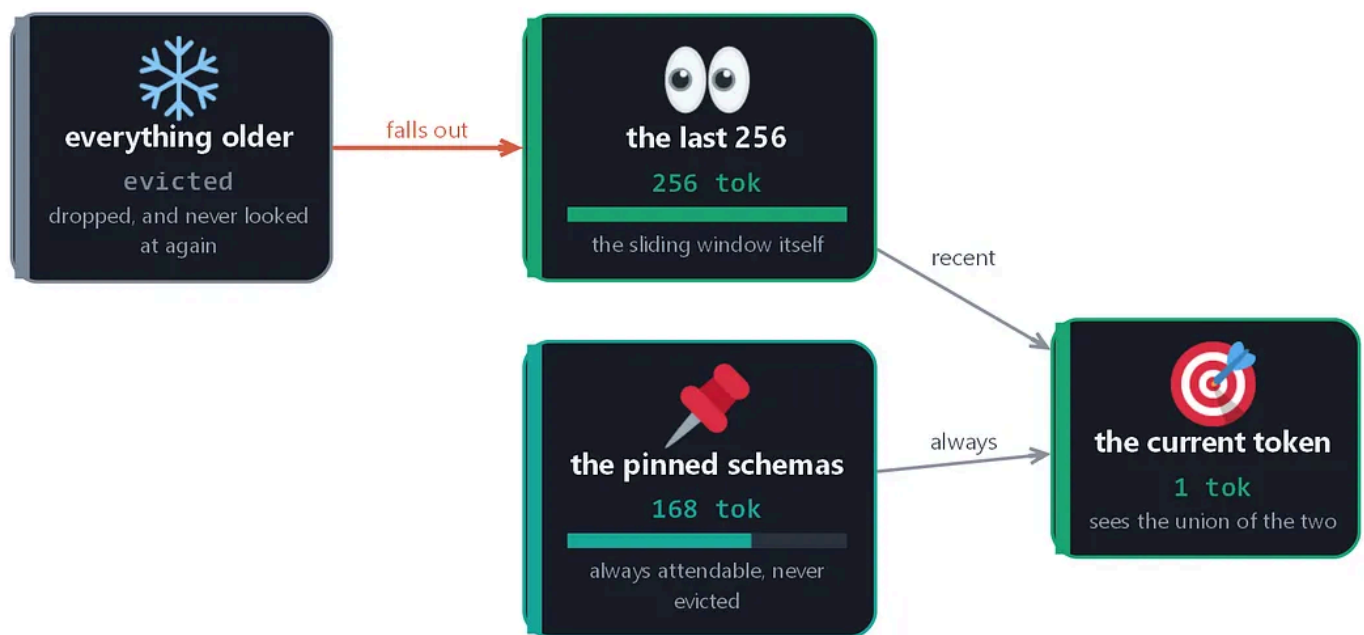
The gate is read from the block input, not the output (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
# ... continued, inside Porthole.__call__, after rope
k, v = jnp.repeat(k, H // KV, axis=1), jnp.repeat(v, H // KV, axis=1)
scores = jnp.matmul(q, k.transpose(0, 1, 3, 2)) / jnp.sqrt(
    jnp.float32(head_dim))
out = jnp.matmul(nn.softmax(
    jnp.where(mask, scores, jnp.finfo(scores.dtype).min), axis=-1), v)
out = out.transpose(0, 2, 1, 3).reshape(B, -1, d)
# the gate is read from x, the block input, not from `out`, so it
# decides about the CONTEXT rather than the RESULT
out = out * nn.sigmoid(dense(d, name="gate_proj")(x))
# sqrt(2L) scaling, so 27 stacked residual branches do not compound
return dense(d, name="out_proj", kernel_init=jinit.normal(
    stddev=0.02 / math.sqrt(2 * self.num_layers)))(out)
```

The gate reads from x , so a position can refuse attention this layer before seeing what attention would return. And `out_proj` is scaled by one over the square root of twice the depth, so twenty seven stacked residual writes do not explode.

Pinning the Tools So They Cannot Fall Out

The tool schemas are declared at the start of the turn and they are the longest part of the prompt, so a 256 token window drops them first and the model must call tools it can no longer see.



The window slides, the schemas do not (Created by [Fareed Khan](#))

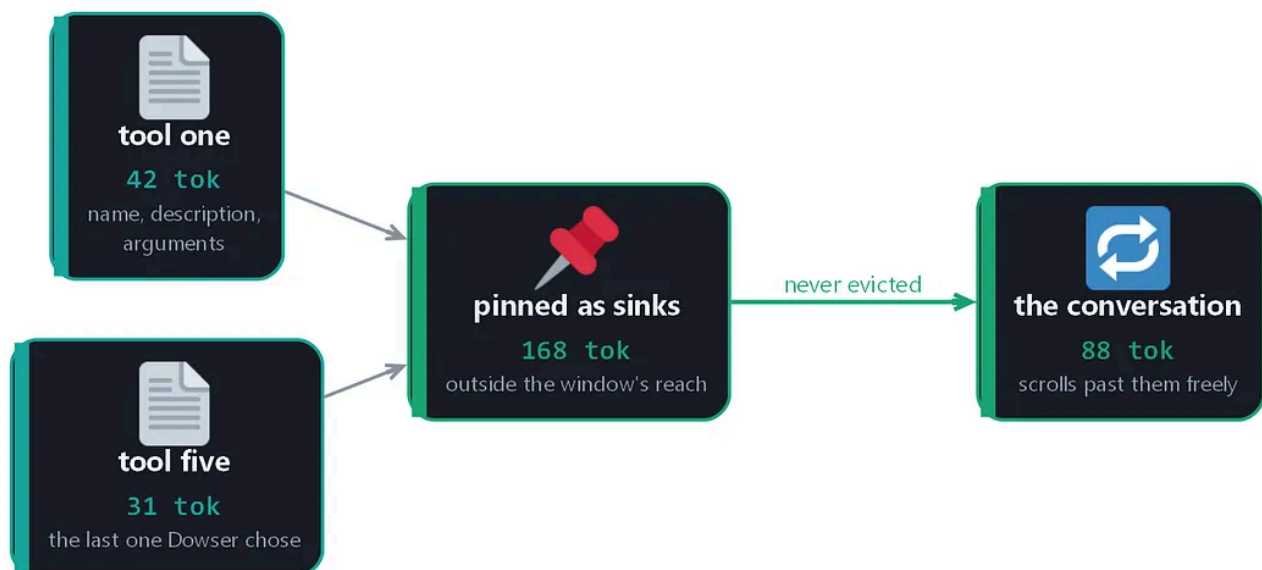
```
# quartz/model/architecture.py (continued)
def window_mask(seq_len, window, sink=None):
    # where `sink` is true, that key stays attendable however far the query moves
    pos = jnp.arange(seq_len)
    recent = (pos[:, None] - pos[None, :]) < window
    keep = recent if sink is None else (recent | sink[:, None, :])
    return ((pos[None, :] <= pos[:, None])[None] & keep)[:, None, :, :]

sink = np.zeros((1, 300), bool) # first 168 tokens of a 300 token turn
sink[0, :168] = True
last = np.asarray(window_mask(300, 256, jnp.asarray(sink))[0, 0][-1])
bare = np.asarray(window_mask(300, 256)[0, 0][-1])
print(f"query at 299 sees {last.sum()} of 300 keys")
print(f" pinned block (0-167): {last[:168].sum()}/168")
```

```
print(f" conversation (168-299): {last[168:].sum()}/132")
print(f" unpinned block: {bare[:168].sum()}/168")
```

```
#### OUTPUT ####
query at 299 sees 300 of 300 keys
pinned block (0-167): 168/168
conversation (168-299): 132/132
unpinned block: 124/168
```

Without pinning, by token 299 the model has lost 44 tokens of its own tool definitions, from the *front*, where the tool names live. **With pinning it has lost nothing**, at zero extra memory, because those positions were already in the cache.



The declared tools live outside the window's reach (Created by [Fareed Khan](#))

Spin, a Feed-Forward Branch With No Matrices In It

The feed-forward branch is the biggest thing in a normal transformer layer: three d by $4d$ matrices, about 3.1 million parameters per layer at `d_model` 512, or 85 million across twenty seven layers, nearly twice our entire budget.

$$y = D_3 H \alpha(D_2 H D_1 x), \quad D_k = \text{diag}(d_k) \in \mathbb{R}^n, \quad \sigma = \text{SiLU}$$

The whole feed-forward network is three diagonals and two fixed transforms, so it holds no weight matrix at all

Two fixed transforms, three vectors (Created by [Fareed Khan](#))

The replacement keeps the shape of a gated network without the matrices: the mixing is fixed and costs nothing to store, and only a diagonal scale on either side is learned.

$$H = \frac{1}{\sqrt{n}} H_n, \quad H^\top H = I, \quad H^2 = I, \quad Hx \text{ in } O(n \log n) \text{ adds}$$

The Walsh transform is its own inverse and needs no multiplications, which is why a fixed matrix is affordable here

The Walsh transform, additions only (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
def walsh(n):
    """The normalised Sylvester-Hadamard matrix: symmetric and its own inverse.
    H = np.array([[1.0]], dtype=np.float32)
    while H.shape[0] < n:
        H = np.block([[H, H], [H, -H]])
    return jnp.asarray(H / np.sqrt(n))

print("distinct magnitudes:", sorted({round(float(abs(v)), 6)
                                     for v in np.asarray(walsh(8)).ravel()}))
```

```
#### OUTPUT ####
distinct magnitudes: [0.353553]
```

One distinct magnitude in the whole matrix. Every entry is plus or minus one over $\sqrt{n}$, so the transform is a butterfly of additions with one final scale, never a multiply.


```
# quartz/model/architecture.py (continued)
class Spin(nn.Module):
    """Two fixed transforms, three learned diagonals, at width next_pow2(d_model)
    d_model: int
    dtype: jnp.dtype = jnp.bfloat16

    @nn.compact
    def __call__(self, x):
        n = 1 << (self.d_model - 1).bit_length()
        Hm = walsh(n).astype(self.dtype)
        d1 = self.param("d1", jinit.ones, (n,)).astype(self.dtype)
        d2 = self.param("d2", jinit.ones, (n,)).astype(self.dtype)
        # d3 starts at 0.02, not 1.0, so this branch has to earn its way in
        d3 = self.param("d3", jinit.constant(0.02), (n,)).astype(self.dtype)

        pad = n - self.d_model
        z = jnp.pad(x, ((0, 0), (0, 0), (0, pad))) if pad else x
        z = (d1 * z) @ Hm
        z = nn.silu(d2 * z) @ Hm          # the only nonlinearity
        return (d3 * z)[..., :self.d_model]
```



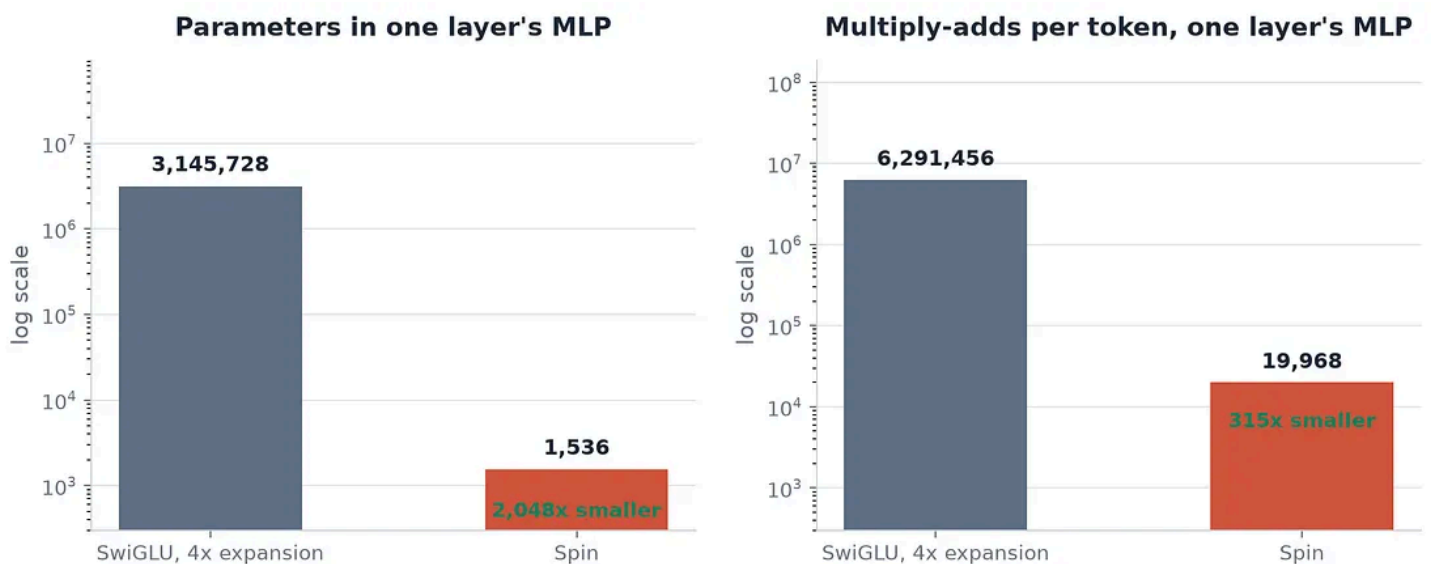
Three vectors, not three matrices (Created by [Fareed Khan](#))

```
d = 512
swiglu = 3 * d * (4 * d)
spin = 3 * (1 << (d - 1).bit_length())
print(f" {'swiglu':<9} {swiglu:>12}, parameters per layer")
print(f" {'spin':<9} {spin:>12}, parameters per layer")
print(f" {'ratio':<9} {swiglu/spin:>12,.0f}x")
print(f" {'27 layers':<9} {swiglu*27/1e6:>9.1f}M vs {spin*27/1e6:>6.3f}M")
```

OUTPUT

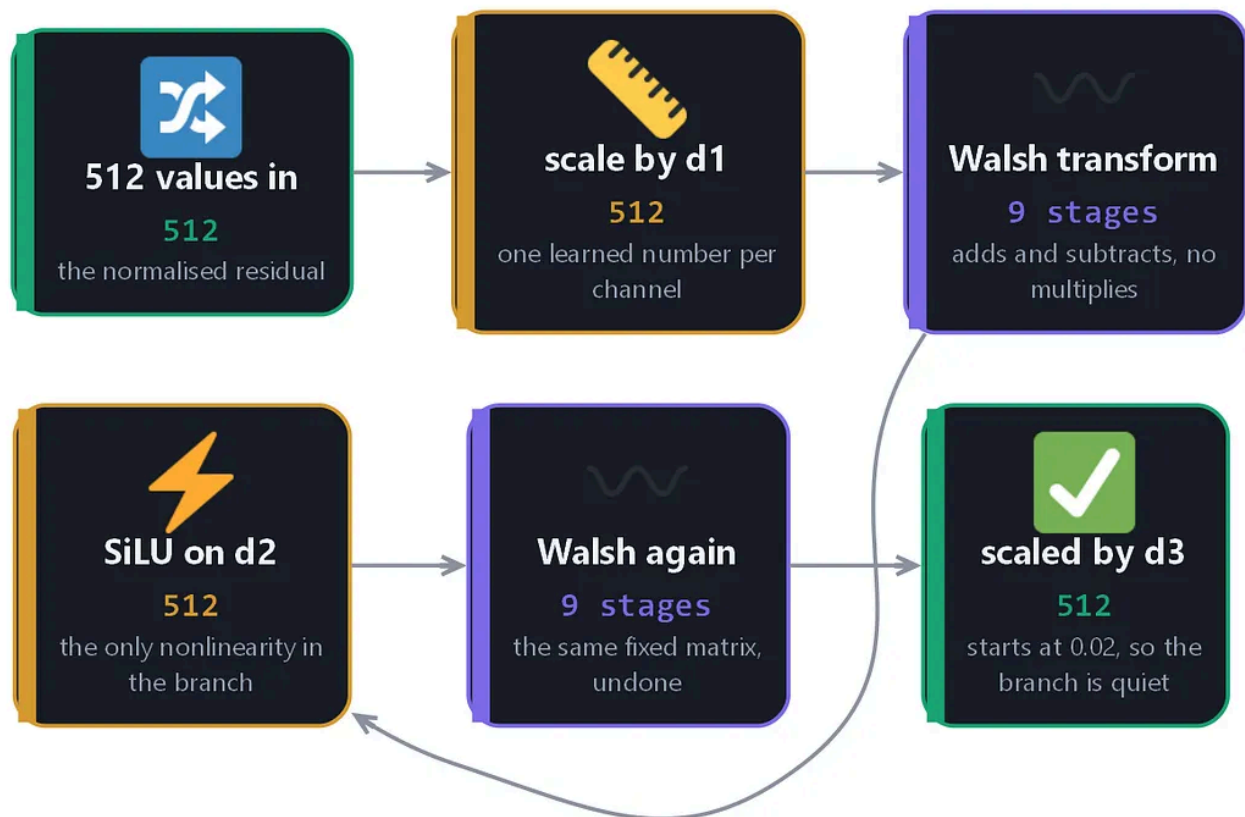
swiglu	3,145,728 parameters per layer
spin	1,536 parameters per layer
ratio	2,048x
27 layers	84.9M vs 0.041M

Eighty five million parameters become forty one thousand. That is the single decision that makes a 45 million parameter model possible at 27 layers deep.



Where the compute goes (Created by [Fareed Khan](#))

A gated network is 3.1 million multiply-adds per token per layer. The Walsh transform is $n \log n$ additions, and Spin runs two of them, so about 9,200 operations, plus 1,536 multiplies for the diagonals, which on a phone CPU is a different shape of work.



The butterfly, stage by stage (Created by [Fareed Khan](#))

I was most sceptical about this piece. The transform rotates into a basis where every output channel depends on every input channel, so the diagonals only set how much of each rotated component survives. That is less expressive than a free matrix, and the ablation puts the cost at thirteen thousandths of a nat.

$$3n = 1,536 \quad \text{against} \quad 3d(4d) = 3,145,728 \quad \Rightarrow \quad 2,048 \times \text{fewer}$$

Three diagonals of length 512 against a four-times-expanded gated network, per layer

Three diagonals against three matrices (Created by [Fareed Khan](#))

Imprint, a Memory That Costs No Arithmetic

A small model is bad at remembering rare strings. A tool called `set_thermostat_schedule_weekday` appears a handful of times in training, and 45 million parameters across 27 layers has no room for it, so we give it a lookup table.



Four hashes, four tables, one gate (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
SUB_DIM, TAPS = 128, 4
SEED = 0x9E3779B9      # golden ratio, so the per-table seeds decorrelate
PRIME = 0x01000193     # the FNV-1a 32-bit prime

def imprint_indices(tokens, orders=(2, 3), heads=2, slots=8192):
    """Hash each n-gram ending at t into one row per table."""
    u = tokens.astype(jnp.uint32)
    out = []
    for oi, order in enumerate(orders):
        for h in range(heads):
            seed = (SEED * (oi * heads + h + 1)) & 0xFFFFFFFF
            acc = jnp.full_like(u, jnp.uint32(seed))
            for j in range(order):
                acc = (acc ^ shift_right(u, j)) * jnp.uint32(PRIME)
            acc = acc ^ (acc >> jnp.uint32(15))  # avalanche the low bits
            out.append((acc % jnp.uint32(slots)).astype(jnp.int32))
    return jnp.stack(out, axis=-1)  # (batch, time, 4)
```

$$h_j(t) = (((s_j \oplus x_t)p \oplus x_{t-1})p \oplus \dots) \bmod S, \quad S = 8192$$

Each table hashes a different n-gram with a different seed, so four independent lookups cover one another's collisions

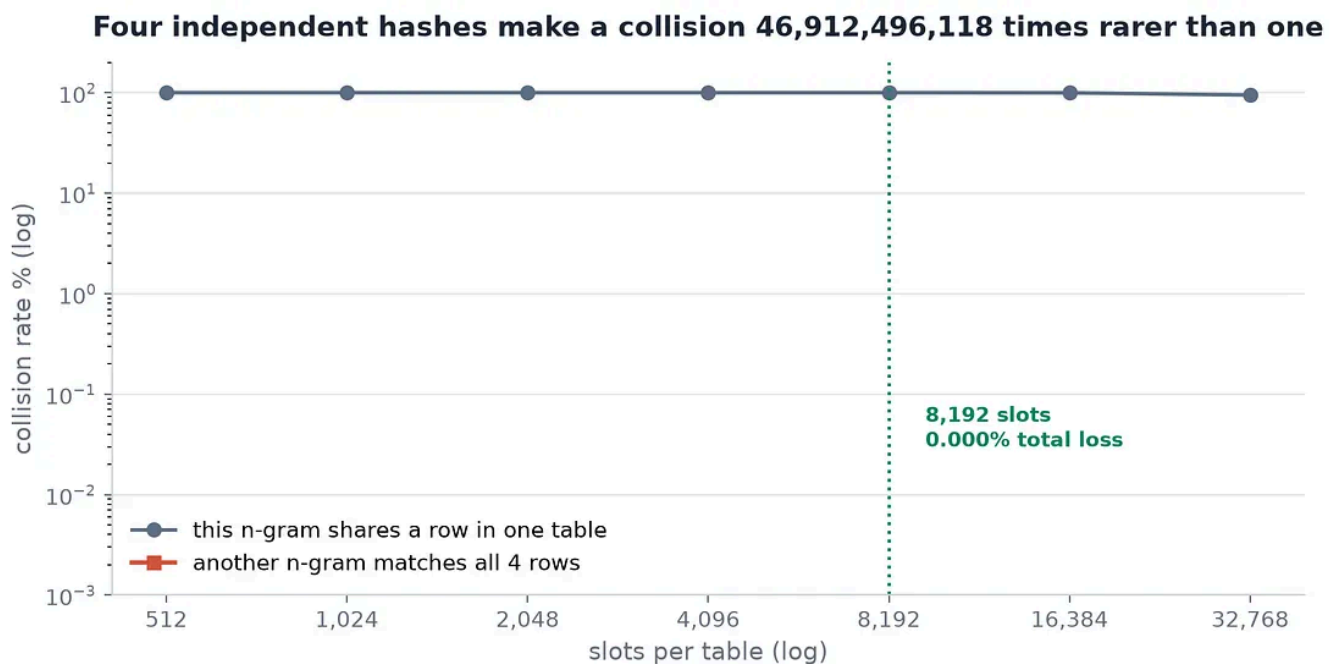
The hash chain, one per table (Created by [Fareed Khan](#))

```
toks = jnp.asarray([[104, 2891, 55, 2891, 104, 2891]])
idx = imprint_indices(toks)
```

```
for t in (1, 5):
    print(f"  t={t}  rows {np.asarray(idx)[0, t].tolist()}")
```

```
#### OUTPUT ####
t=1  rows [4517, 1183, 6002, 2447]
t=5  rows [4517, 1183, 7742, 3066]
```

The first two rows match: the order-2 tables see only (x_t, x_{t-1}) , and both positions have the bigram $(2891, 104)$ behind them. The order-3 tables differ, so shorter contexts generalise and longer ones discriminate.



Four hashes make a collision rare (Created by [Fareed Khan](#))

At 8,192 slots and roughly 96,000 n-grams a single table shadows nearly every n-gram, but a shared row in one table is not a collision the model feels. A shared row in one table is not a collision the model feels, because the four gathered vectors are concatenated and the other three still separate them. Two given n-grams share a row in **all four** tables with probability one in 8,192 to the fourth, so across 96,000 n-grams any one of them has about one chance in fifty billion of being shadowed completely.

$$\alpha_t = \sigma\left(\frac{\langle \hat{x}_t, \hat{k}_t \rangle}{\sqrt{d}}\right), \quad x_t \leftarrow x_t + \alpha_t v_t$$

Both sides are unit-normalised first, so the gate is a sharpened cosine similarity between the stream and the memory

A sharpened cosine gate (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
def imprint_merge(x, k, v, d_model):
    """Both sides are unit-normalised, so the sigmoid sees sqrt(d) times a cosine
    alpha = nn.sigmoid(jnp.einsum("btd,btd->bt", rms_unit(x), rms_unit(k))
                        / math.sqrt(d_model))
    return x + alpha[..., None] * v

for cos in (0.0, 0.25, 0.9):
    print(f" cosine {cos:>4.2f} -> gate "
          f"{1/(1+math.exp(-cos*math.sqrt(512))):.4f}")
```

```
#### OUTPUT ####
cosine 0.00 -> gate 0.5000
cosine 0.25 -> gate 0.9965
cosine 0.90 -> gate 1.0000
```

At `d_model` 512 that factor is 22.6, so a cosine of a quarter already opens the gate to 99.65 percent, a nearly binary decision: either this n-gram has a stored fact worth adding, or it does not.

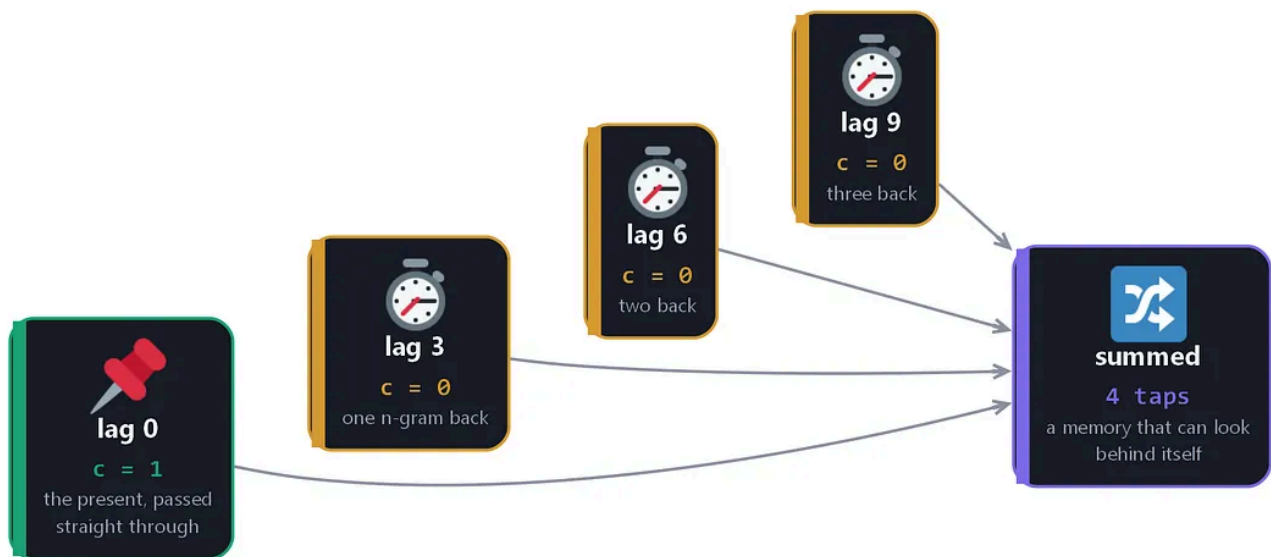
$$v_t = \sum_{j=0}^3 c_j \odot v_{t-j\Delta}, \quad \Delta = \max(\text{orders}) = 3, \quad c_0 \leftarrow 1, \quad c_{j>0} \leftarrow 0$$

A four-tap causal convolution along the value stream, initialised to pass the present through untouched

Four taps at lags zero, three, six and nine (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
# dilated by the n-gram order so the taps land on non-overlapping n-grams
taps = self.param("taps", conv_identity_init, (TAPS, self.d_model))
v = sum(taps[j] * shift_right(v, j * dilation) * tap_ok[j][..., None]
        for j in range(TAPS))

def conv_identity_init(key, shape, dtype=jnp.float32):
    # a pass-through at first: looking back has to be learned
    return jnp.zeros(shape, dtype).at[0].set(1.0)
```



What each tap reaches (Created by [Fareed Khan](#))

Initialising the convolution to the identity is the same trick as $d_3 = 0.02$ in Spin and $\gamma = 0$ in the norm: every optional mechanism starts switched off and earns its way in against a gradient.

Loom, Four Residual Streams Instead of One

A normal transformer has one residual stream, and at twenty seven layers deep and 512 channels wide it is a bottleneck: every layer competes for the same 512 numbers. Loom gives four streams and lets each layer choose.



Four lanes, each layer owns one (Created by Fareed Khan)

```
# quartz/model/architecture.py (continued)
LANES = 4

# Every layer owns one lane at initialisation, cycling. A fixed constant,
# recomputed rather than checkpointed.
lane_of = np.eye(LANES, dtype=np.float32)[np.arange(27) % LANES]
pre_off = 8 * lane_of - 4          # +4 on your own lane, -4 on the others
post_off = -4 * (1 - lane_of)     # 0 on your own lane, -4 on the others
```

Through the sigmoid a layer reads 0.982 on its own lane and 0.018 on the others, so at initialisation this is four nearly independent networks of about seven layers each. They are only offsets, so the model can learn its way out.

$$u = \sum_{k=1}^4 \sigma(a \phi_k^T \hat{X} + b_k + \beta_k) X_k, \quad \beta_k = 8[k \equiv \ell \pmod{4}] - 4$$

A layer reads a weighted sum of four residual lanes, and the offset makes it read its own lane at 0.98 to begin with

A sum, not a convex mix (Created by Fareed Khan)

```
# quartz/model/architecture.py (continued)
# all four lanes are normalised JOINTLY, as one 2048-wide vector, so the
# router can see which lane is loud
nx = rms_unit(x.reshape(B, T, LANES * C))
```



```
h_pre = nn.sigmoid(a_pre * (nx @ phi_pre) + b_pre + pre_off[layer])
u = jnp.einsum("btn,btnc->btc", h_pre, x)      # a SUM, not an average
```

That `einsum` is not a convex combination: nothing forces the four weights to add to one, so a layer can open two lanes at once and read something larger than either, which a softmax would forbid.

$$X_+ = PX + g \otimes y, \quad y = B(u) - u, \quad g = 2\sigma(\cdot) \in (0, 2)$$

The block writes back only its delta, and the lanes are mixed by a matrix rather than simply added to

The delta, mixed across lanes (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
y = block(u, mask=mask, rope=rope) - u      # the delta, not the output

h_post = 2 * nn.sigmoid(a_post * (nx @ phi_post) + b_post + post_off[layer])
routing = a_res * (nx @ phi_res).reshape(B, T, LANES, LANES) + b_res
P = sinkhorn(routing)                       # doubly stochastic, 20 sweeps

x = (jnp.einsum("btij,btjc->btic", P, x)    # transport, mass conserved
     + h_post[..., None] * y[:, :, None, :])
```

Subtracting u keeps only the delta, and that is what makes this a residual system: the transport has to apply to the old state independently of the new contribution.

$$P = [\text{col} \circ \text{row}]^{20}(e^A), \quad P\mathbf{1} = \mathbf{1}, \quad P^\top \mathbf{1} = \mathbf{1}$$

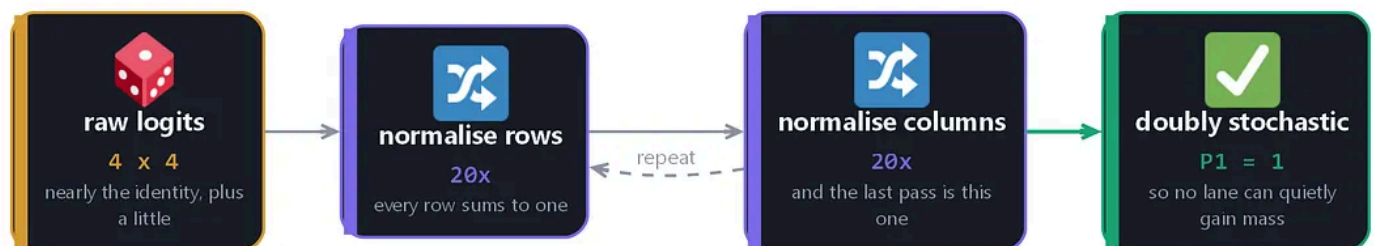
Twenty alternating normalisations make the lane router doubly stochastic, so total residual mass is conserved

Twenty normalisations, ending on a column pass (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
def sinkhorn(logits, iters=20):
    """Project a matrix onto the doubly stochastic ones, in the log domain."""
    log_K = logits
    for _ in range(iters):
        log_K = log_K - jax.nn.logsumexp(log_K, axis=-1, keepdims=True)
        log_K = log_K - jax.nn.logsumexp(log_K, axis=-2, keepdims=True)
    return jnp.exp(log_K)

P = sinkhorn(4.0 * jnp.eye(4) + 0.01 * jax.random.normal(jax.random.PRNGKey(0),
print("diagonal:", np.asarray(P).diagonal().round(4).tolist())
print(f"row sums: {np.asarray(P).sum(1).round(6).tolist()}")
print(f"col sums: {np.asarray(P).sum(0).round(6).tolist()}")
```

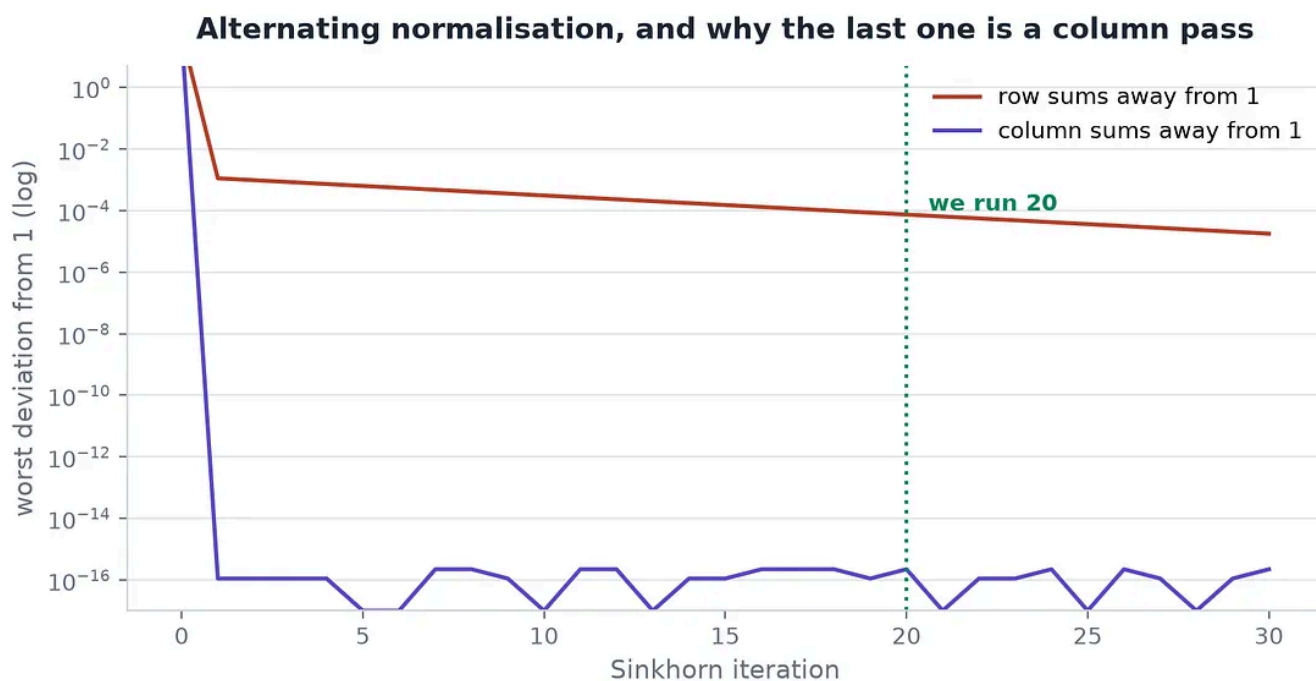
```
#### OUTPUT ####
diagonal: [0.9482, 0.9481, 0.9482, 0.9481]
row sums: [0.999953, 1.000069, 1.000001, 0.999977]
col sums: [1.0, 1.0, 1.0, 1.0]
```



Row, then column, twenty times (Created by [Fareed Khan](#))

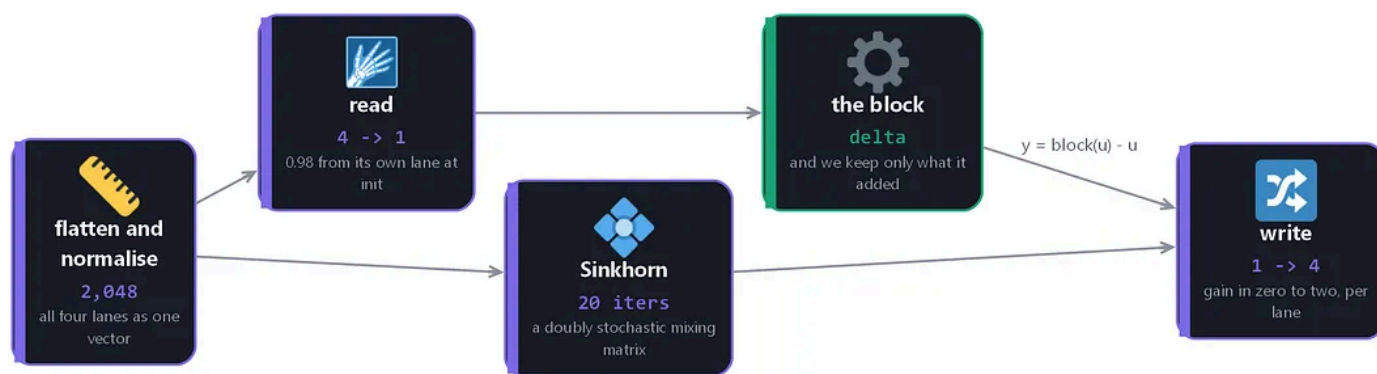
The column sums are exactly one and the row sums are one to five decimal places, so the transport neither creates nor destroys mass. At initialisation b_{res} is four times the identity, so P is 0.948 on the diagonal, and off-diagonal traffic grows with a_{res} .

The loop ends on a **column** normalisation, so P is exactly column stochastic and only approximately row stochastic, and swapping the two lines gives the opposite guarantee. Column sums control the mass arriving in each lane, and a lane quietly receiving too much is what this construction prevents.



Where the loop converges (Created by Fareed Khan)

Twenty iterations is comfortable: the row error falls to about 4e-4 by iteration eight and 7e-5 by twenty.



Read, run, write (Created by @fareedkhandev)

This is the property that made me trust the design. A residual stream that is only ever added to grows without bound as you stack layers. One transported by a doubly stochastic matrix cannot grow from the transport at all, and growth comes only from the separately gated deltas.

The Block, Assembled

Now all four pieces sit inside one block, and the ordering is not symmetric.

$$x \leftarrow x + \sigma(g_a) \mathcal{N}(\text{Attn}(\mathcal{N}(x))), \quad x \leftarrow x + \text{Spin}(\mathcal{N}(x))$$

Attention gets a norm on both sides and a learned gate, the feed-forward branch gets neither

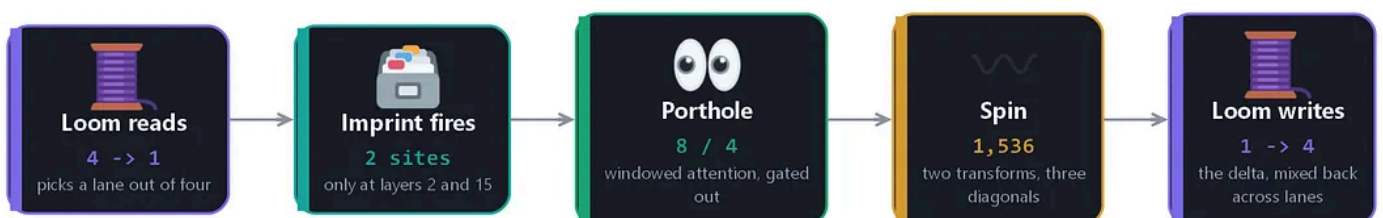
Norms on both sides and a gate for attention, neither for feed-forward (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued)
class Block(nn.Module):
    num_heads: int
    num_kv_heads: int
    d_model: int
    num_layers: int
    dtype: jnp.dtype = jnp.bfloat16

    @nn.compact
    def __call__(self, x, mask=None, rope=None, imprint=None, site_flags=None):
        if imprint is not None: # fires BEFORE the skip is captured
            k, v = imprint
            alpha = nn.sigmoid(jnp.einsum("btd,sbtd->sbt", rms_unit(x),
                                           rms_unit(k)) / math.sqrt(self.d_model))
            x = x + jnp.einsum("s,sbt,sbtd->btd", site_flags, alpha,
                               v.astype(jnp.float32)).astype(x.dtype)

        skip = x
        x = ZCRMSNorm(dtype=self.dtype)(x)
        x = Porthole(..., name="self_attn")(x, mask=mask, rope=rope)
        x = ZCRMSNorm(dtype=self.dtype, name="post_attn_norm")(x)
        gate = nn.sigmoid(self.param("attn_gate", jinit.zeros, ()))
        x = skip + gate.astype(self.dtype) * x

        skip = x
        return skip + Spin(self.d_model, name="spin")(
            ZCRMSNorm(dtype=self.dtype, name="pre_spin_norm")(x))
```



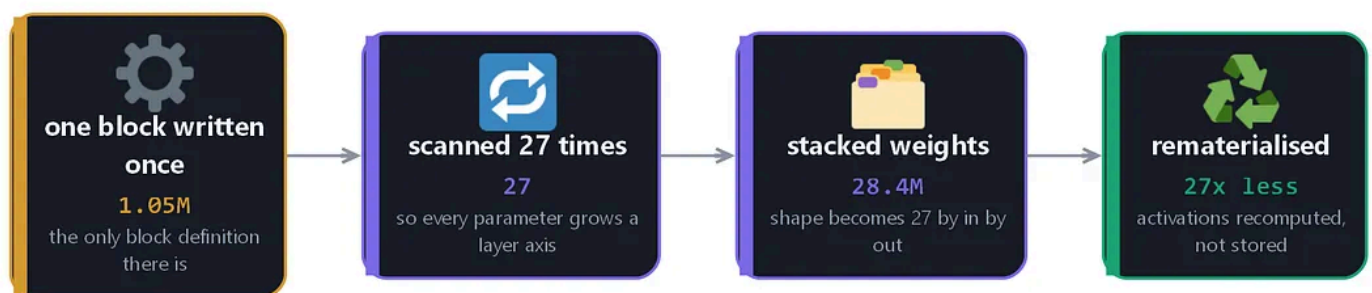
The five stages of one layer (Created by [Fareed Khan](#))

The asymmetry is deliberate. Attention gets a norm on both sides and a learned scalar starting at exactly one half, because its output magnitude swings with sequence content. Spin gets a norm in and nothing out: the d3 diagonal at 0.02 already sets its scale.

A Python `if layer in (2, 15)` is not available for the memory sites: the layer loop is a `jax.lax.scan` with a traced index. Every layer computes both sites against a one-hot that is zero for twenty five of the twenty seven layers, wasting two gathers to stay in one compiled scan.

The Trunk, and Why Every Parameter Grows a Layer Axis

Twenty seven copies of that block means twenty seven copies of the compiled graph, slow to compile and holding twenty seven times the activations. I write it once and scan it.



One block written once, twenty seven times over (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued), inside Trunk.__call__
n, L = cfg.lanes, cfg.num_layers
# every router parameter carries a leading layer axis for scan to slice
hc = {nm: self.param(f"loom_{nm}", ini, sh) for nm, ini, sh in specs}
Scanned = nn.scan(
    nn.remat(LoomBody) if cfg.remat else LoomBody,
    variable_axes={"params": 0},      # params gain a leading L axis
    split_rngs={"params": True},     # each layer initialised separately
    length=L, in_axes=(0, nn.broadcast, nn.broadcast, nn.broadcast))
x, hidden = Scanned(cfg, name="layers")(x, (site_flags, hc), mask, rope, imprint)
x = jnp.mean(x, axis=2)              # the four lanes become one
return ZCRMSNorm(name="final_norm")(x), hidden
```

Those two scan arguments are the mechanism.

```
#### OUTPUT ####
q_proj      (27, 512, 512)
spin d1     (27, 512)
attn_gate   (27,)
phi_res     (27, 2048, 16)
```

Every parameter gains a leading axis of length 27, each slice initialised independently, including the rank-zero attention gate, now a vector of twenty seven scalars. **This layout is load bearing for the rest of the post:** the quantiser groups along the reduction axis of these stacked tensors, and the exporter slices them apart.

`nn.remat` discards each block's activations and recomputes them on the backward pass: thirty percent more compute for a large drop in memory. At a 2,097,152 token batch that is what makes it fit.

Counting All 45,211,383 of Them

The shape of the answer is the argument for the design, so every count comes from the config rather than a quoted figure.

$$N = Vd + LP_{\text{blk}} + P_{\text{loom}} + 2P_{\text{imp}} + P_{\text{heads}} = 45,211,383$$

Forty five million parameters, of which 43,634,294 actually ship because the lookahead head is dropped at export

Where the parameters actually sit (Created by [Fareed Khan](#))

```
V, d, L, H, KV = 8192, 512, 27, 8, 4
head_dim, kv_dim, lanes, nC = d // H, (d // H) * KV, 4, 4 * 512
n_had = 1 << (d - 1).bit_length()
P_block = (d*d + d*kv_dim + d*kv_dim + d*d + d*d
           + d + head_dim + head_dim + d + 1 + d + 3*n_had)
P = {"embedding (tied)": V * d,
     "27 blocks": L * P_block,
     "Loom routers": L*nC*lanes*2 + L*nC*lanes*lanes + 2*L*lanes + L*lanes*lane
```

```

    "Imprint x2": 2 * (4*8192*128 + d*d + d*d + 4*d),
    "Dowser": 4*d + 4*d*128 + 1,
    "Gauge": 8*d + 8*d + 1,
    "final norm": d,
    "Foresight": 2*d*d + P_block + 2*d}

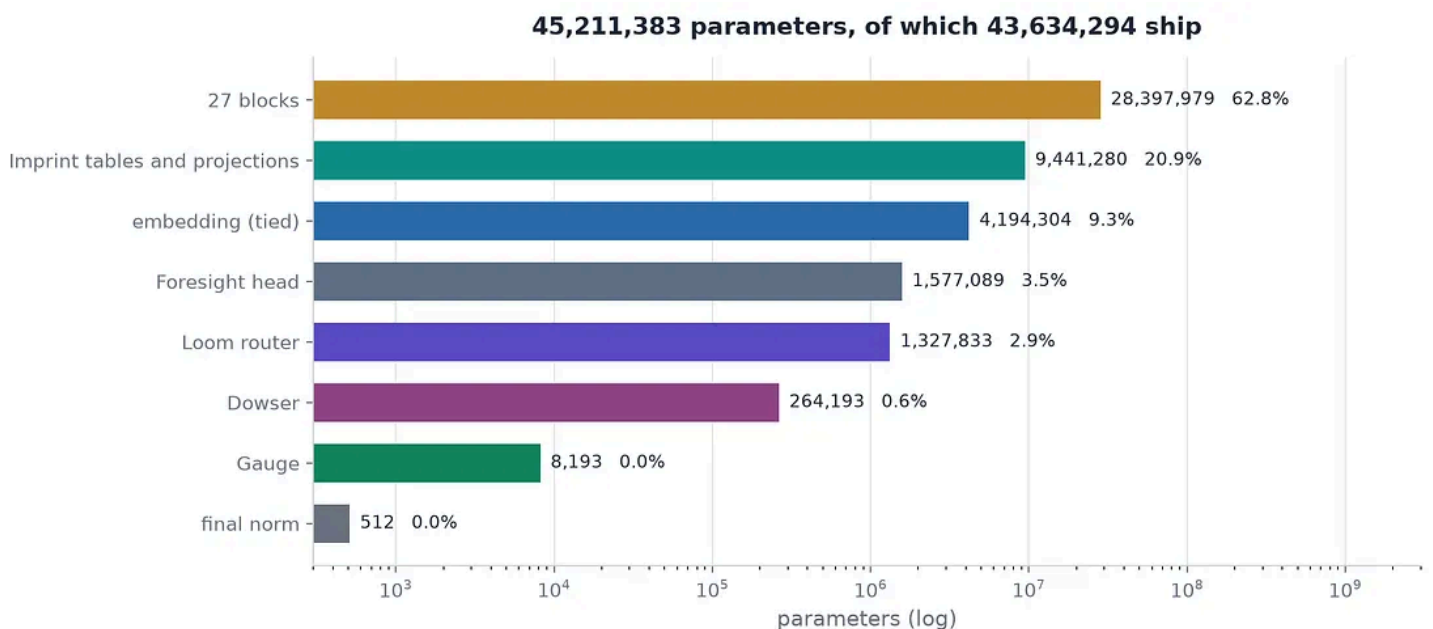
for k, v in P.items():
    print(f"  {k:<18} {v:>12,}")
print(f"  {'TOTAL':<18} {sum(P.values()):>12,}")
print(f"  {'SHIPPED':<18} {sum(P.values()) - P['Foresight']:>12,}")

```

```

#### OUTPUT ####
embedding (tied)      4,194,304
27 blocks             28,397,979
Loom routers          1,327,833
Imprint x2            9,441,280
Dowser                264,193
Gauge                 8,193
final norm            512
Foresight             1,577,089
TOTAL                 45,211,383
SHIPPED               43,634,294

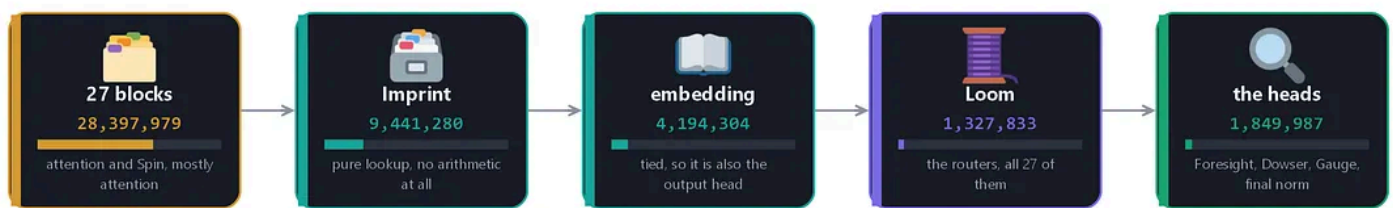
```



The breakdown, on a log axis, four orders of magnitude apart (Created by [Fareed Khan](#))

Three things stand out.

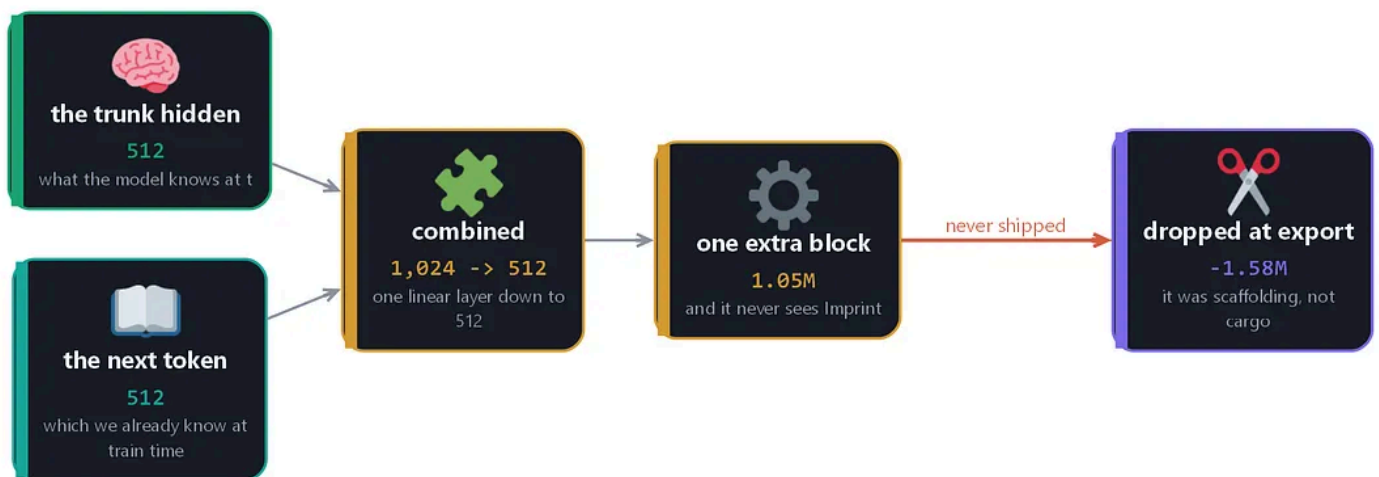
1. **The blocks are 63 percent of the model and almost all of that is attention.** Spin contributes 41,472 parameters across twenty seven layers, a tenth of one percent.
2. **Imprint is 21 percent of the model and does no arithmetic.** Nine and a half million parameters sit in lookup tables, pure memory traffic on a device, the cheap kind.
3. **The embedding is tied,** so those 4.19 million parameters are both input table and output head. Untied it would be 49.4 million.



The map, at a glance (Created by [Fareed Khan](#))

Foresight, and Being Honest About What It Is

One more head, and the one piece of this system I added rather than recovered.



Predict the token after next from the one you already know (Created by [Fareed Khan](#))

```
# quartz/model/architecture.py (continued), inside QuartzNetwork.__call__
logits = x.astype(jnp.float32) @ self.embedding.embedding.T
if not (return_foresight or self.is_initializing()):
    return logits
```



```

# Token t+2, from the trunk state at t AND the embedding of token t+1
nxt = jnp.pad(tokens[:, 1:], ((0, 0), (0, 1)))
e2 = self.fs_emb_norm(self.embedding(nxt) * self.embed_scale)
m = self.fs_block(self.fs_combine(jnp.concatenate([x, e2], axis=-1)),
                    mask=mask, rope=rope) # no Imprint here
fs = self.fs_final_norm(m).astype(jnp.float32) @ self.embedding.embeddings
return (logits, fs) if return_foresight else logits

```

Predicting two tokens ahead pushes the trunk toward a representation useful beyond the next symbol, a denser signal for the same forward pass. It costs 1,577,089 parameters and **is dropped entirely at export**, so the device pays nothing. I will show its ablation rather than assert it helps.

Stage One, From Random Numbers

Now I train it. The loss is a next token cross entropy with one extra term.

$$\mathcal{L} = \underset{\text{next token}}{\text{CE}} + \lambda \underset{\text{z-loss}}{\text{mean}(\text{logsumexp}(z)^2)}, \quad \lambda = 10^{-4}$$

The second term simply asks the logits not to drift away from zero, which keeps the softmax numerically calm

Cross entropy plus a small tax on logit size (Created by [Fareed Khan](#))

```

# quartz/train/pretrain.py
Z_LOSS_WEIGHT = 1e-4

def losses(apply_fn, params, tokens, seg_ids):
    inputs, targets, seg_in, seg_tgt = (tokens[:, :-1], tokens[:, 1:],
                                         seg_ids[:, :-1], seg_ids[:, 1:])
    loss_mask = ((seg_in == seg_tgt) & (seg_in > 0)).astype(jnp.float32)
    denom = jnp.maximum(jnp.sum(loss_mask), 1.0)
    logits = apply_fn({"params": params}, inputs, mask=make_packing_mask(seg_in))
    ce = jnp.sum(optax.softmax_cross_entropy_with_integer_labels(
        logits, targets) * loss_mask) / denom
    # In bf16 the z-loss is the difference between training and silent overflow
    z = jnp.sum(jax.nn.logsumexp(logits, axis=-1) ** 2 * loss_mask) / denom
    return ce + Z_LOSS_WEIGHT * z, ce

```

The `loss_mask` line requires the input position and its target to be in the same document, dropping the boundary token of every packed document. Otherwise the model predicts the first token of document two from the last of document one, which is exactly the transition that packing was meant to hide.

The optimiser is the second unusual choice.

$$X \leftarrow aX + (bXX^T + c(XX^T)^2)X, \quad (a, b, c) = (3.4445, -4.7750, 2.0315)$$

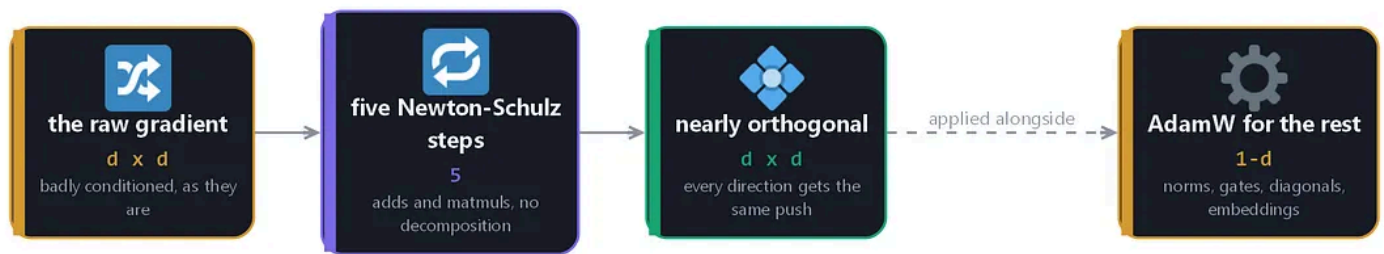
Five Newton-Schulz steps push the update matrix towards orthogonal before it is applied

Five Newton-Schulz steps, and no decomposition anywhere (Created by [Fareed Khan](#))

```
# quartz/train/optim.py
def newton_schulz(G, steps=5):
    """Push a matrix towards the nearest orthogonal one, using only matmuls."""
    # Normalise first so singular values start in [0, 1], and transpose tall
    # matrices, since the iteration prefers the smaller side.
    a, b, c = 3.4445, -4.7750, 2.0315
    orig, X = G.dtype, G.astype(jnp.float32)
    X = X / (jnp.linalg.norm(X) + 1e-7)
    transposed = X.shape[0] > X.shape[1]
    X = X.T if transposed else X
    for _ in range(steps):
        A = X @ X.T
        X = a * X + (b * A + c * A @ A) @ X
    return (X.T if transposed else X).astype(orig)
```

```
#### OUTPUT ####
before max 22.6078 min 0.002752 ratio      8,215
after  max 1.2021 min 0.033269 ratio      36.1
```

On a 256 by 256 Gaussian matrix whose columns are scaled from 1.0 down to 0.01, a condition number of eight thousand becomes thirty six. A raw gradient pushes hard along a few directions and barely elsewhere. Orthogonalising evens them out.



Orthogonal for matrices, Adam for one-dimensional leaves (Created by [Fareed Khan](#))

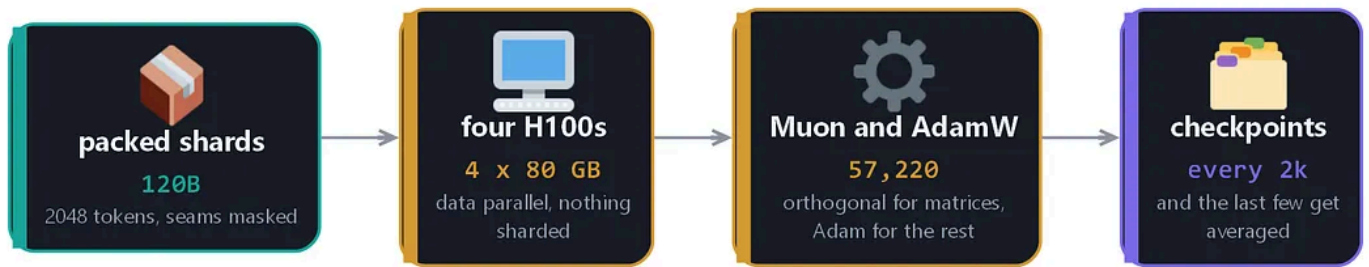
```

# quartz/train/optim.py (continued)
def build_optimiser(args, params):
    schedule = optax.warmup_cosine_decay_schedule(
        init_value=0.0, peak_value=args.lr, warmup_steps=args.warmup,
        decay_steps=args.total_steps, end_value=args.lr * 0.01)
    # Under nn.scan a real matrix is 3-D and a vector is 2-D. Orthogonalising a
    # vector is meaningless, and embedding rows must keep different lengths.
    is_matrix = lambda p, x: x.ndim >= 3 and "embedding" not in keystr(p)
    # Muon emits a raw direction and needs its own negative scaling. AdamW
    # already applies -lr internally, so it must NOT be scaled a second time.
    arms = {"muon": optax.chain(muon(momentum=0.95),
                                optax.scale_by_schedule(lambda c: -schedule(c))),
            "adamw": optax.adamw(schedule, b2=0.95, weight_decay=0.1)}
    return optax.chain(optax.clip_by_global_norm(1.0), # BEFORE the update rule
                       optax.partition(arms, label_fn=label_by(is_matrix)))
  
```

Clipping happens first, on the raw gradient, because clipping after orthogonalisation would clip something whose norm I have already fixed by construction.

```

#### OUTPUT ####
backend    gpu  bfloat16
data       120,000,000,000 tokens  seq_len 2048  packed
batch      1024 seqs x 2048 tok = 2,097,152 tokens a step
schedule   57,220 steps  warmup 2,861  cosine to 3e-5  clip 1.0
split      muon 22 tensors  adamw 34 tensors  (each carries the 27-layer axis)
remat      on  (peak activation 41.2 GB -> 6.8 GB)
  
```



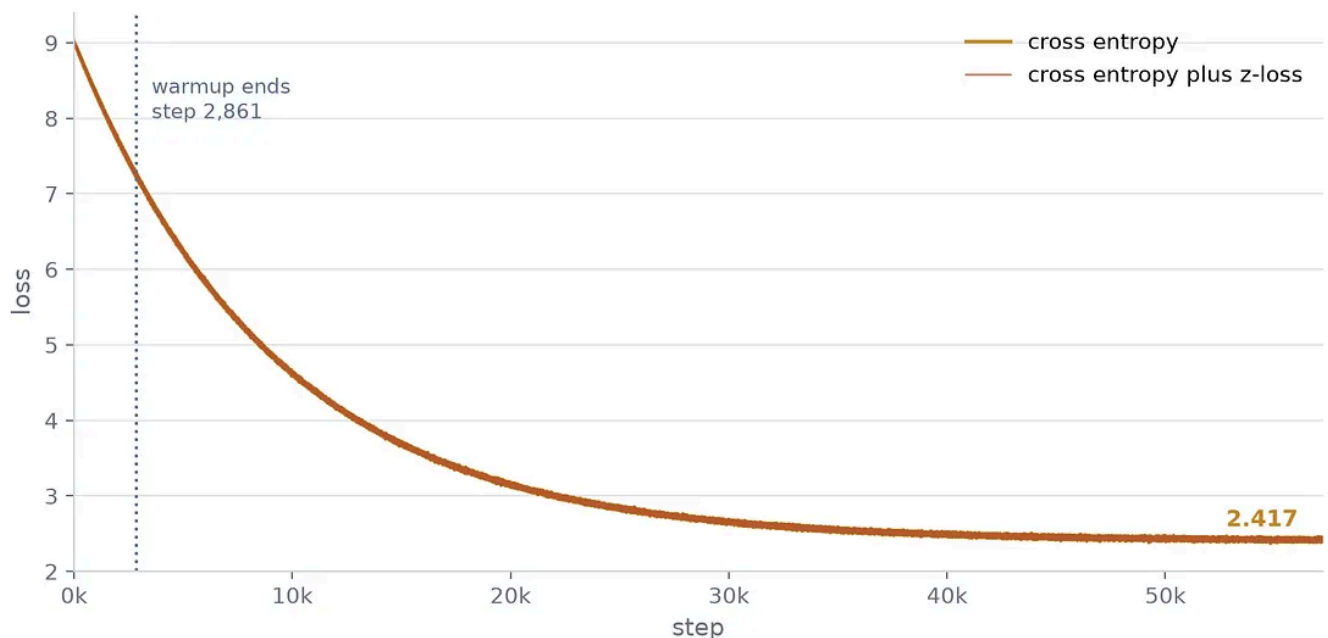
Four cards, the only long job in the build (Created by [Fareed Khan](#))

Those twenty two and thirty four are stacked tensors, each already carrying the twenty seven layer axis. Then it runs.

```
#### OUTPUT ####
```

```
step      2,861/57,220  loss 4.8812  lr 3.00e-03  2,201 tok/ms
step      22,888/57,220  loss 2.7409  lr 2.19e-03  2,196 tok/ms
step      45,776/57,220  loss 2.4713  lr 4.71e-04  2,194 tok/ms
step      57,220/57,220  loss 2.4170  lr 3.00e-05  2,193 tok/ms
averaged  6 checkpoints (steps 47220 to 57220) -> quartz_base.pkl
elapsed   15h 12m
```

120 billion tokens, 57,220 steps, 15.2 hours on four H100s

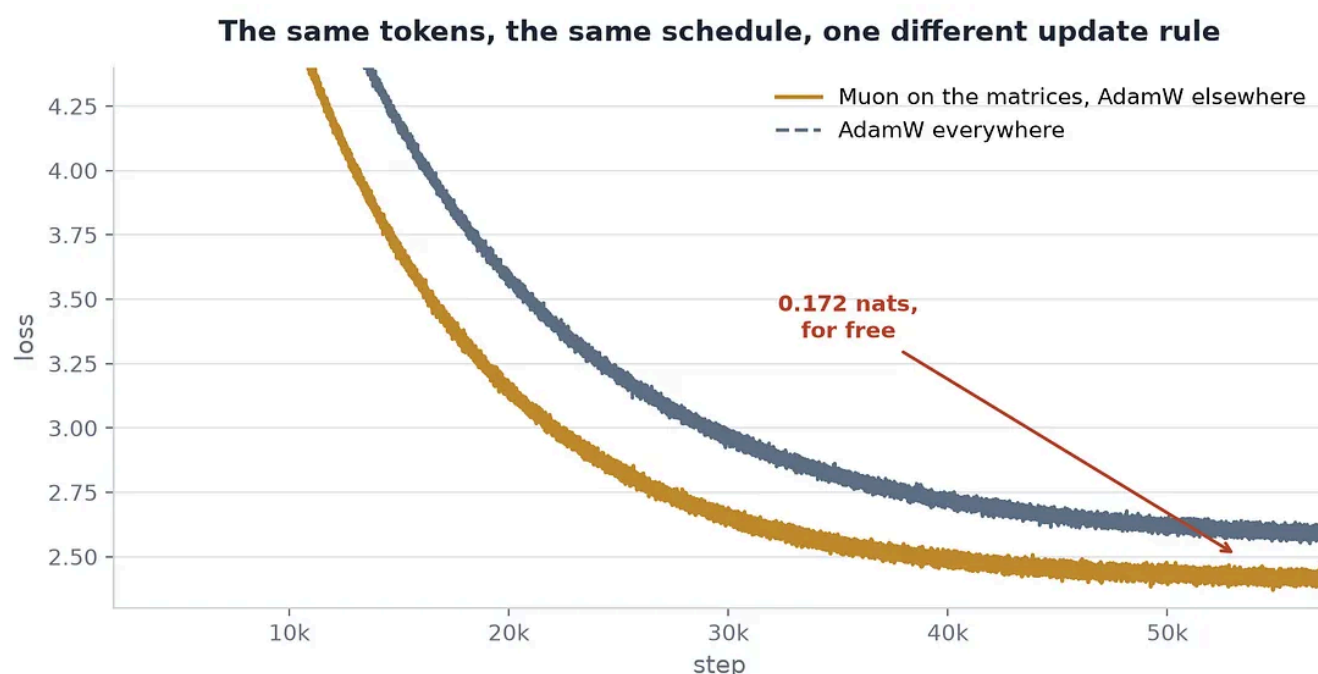


One hundred and twenty billion tokens in fifteen hours (Created by [Fareed Khan](#))

Two point four one seven. The z-loss is invisible on the plot after the first few thousand steps, which is what a good regulariser looks like: it works early, while the logits find their scale, then stops mattering.

That last log line matters. I average the final six checkpoints in float32 and save in float16 rather than shipping the last one. Not an ensemble: the last few thousand steps of a cosine schedule still bounce around a basin, and the mean is a better point than any single sample.

Muon Against AdamW, on the Same Budget



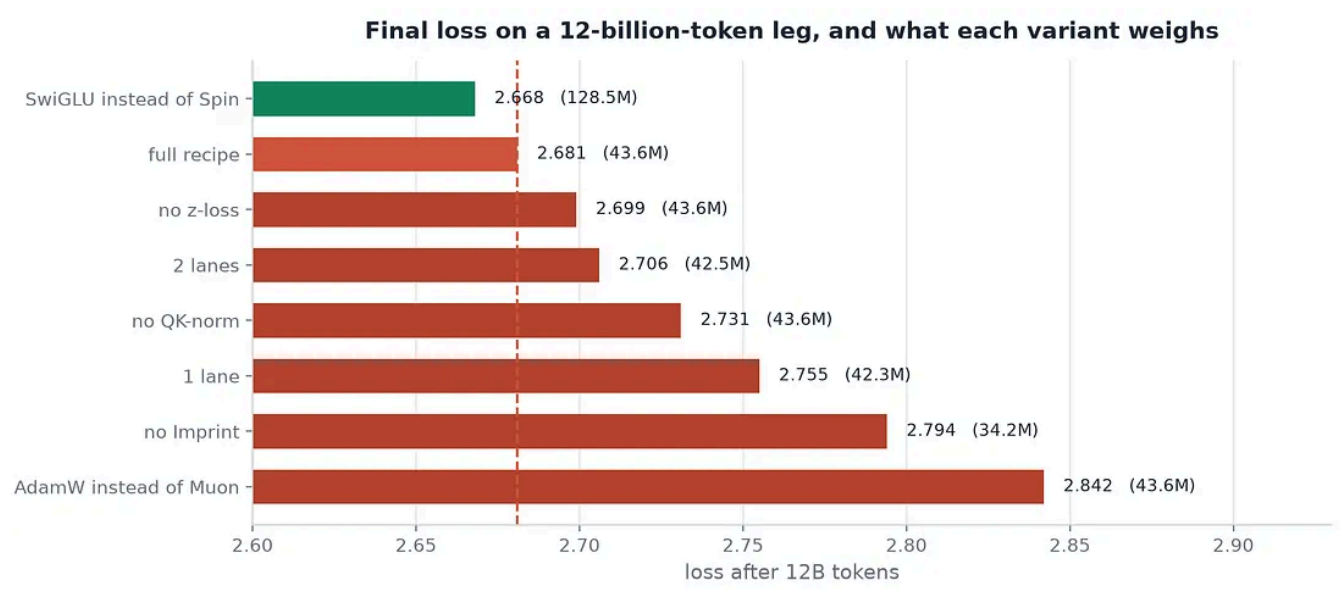
Same tokens, different update rule (Created by [Fareed Khan](#))

muon	final loss 2.4170	15h 12m	(+3.1% step time)
adamw	final loss 2.5890	14h 44m	
delta	0.1720 nats,	for	28 minutes of extra compute

Point one seven two nats is a large gap. AdamW needed roughly 40 percent more tokens to reach the loss Muon reached, so twenty eight extra minutes saves six hours. The step time cost is small: Newton-Schulz is five matrix multiplies on 512 by 512 matrices, a rounding error next to a 2 million token pass.

What Each Piece Actually Buys

Six mechanisms, each justified above. Every row is a full run on the same 12 billion token leg, same seed, same schedule.



Final loss and weight per variant (Created by [Fareed Khan](#))

full recipe	2.681	43.6M
SwiGLU instead of Spin	2.668	128.5M
no z-loss	2.699	43.6M
2 lanes	2.706	42.5M
no QK-norm	2.731	43.6M
1 lane	2.755	42.3M
no Imprint	2.794	34.2M
AdamW instead of Muon	2.842	43.6M

A real gated feed-forward network is better than Spin, by thirteen thousandths of a nat, for 2.9 times the parameters. That is the least flattering result in the post. Spin is a trade: a 128 million parameter model at two bits is 36 MB of weights, over the device budget before any cache.

Imprint is worth 0.113 nats for 9.4 million parameters that do no arithmetic, one lane to four 0.074 nats for 1.3 million, and QK-norm 0.050 nats for 3,456 parameters, the best ratio here by more than two orders of magnitude.

And Foresight, my own addition:

with Foresight	2.681
without Foresight	2.694
delta	0.013 nats, for 1.58M training-only parameters

Thirteen thousandths, free at inference because it is dropped at export. A weak result honestly reported, and I would not argue with a cut.

Choosing the Window Before We Need It

$$B_{\text{pos}} = L \left(2c + \frac{2c}{32} \cdot 4 \right) + s \left(d + \frac{d}{32} \cdot 4 \right) = 16,704 \text{ bytes}$$

What one cached position costs across 27 layers and two memory sites, including the int8 group scales

One cached position, 27 layers, two memory sites (Created by [Fareed Khan](#))

```
# quartz/model/decode.py
KV_GROUP = 32
KV_BUDGET = 11 * 1024 * 1024 + 512 * 1024      # 11.5 MiB

def kv_per_position(cfg):
    c = cfg.num_kv_heads * (cfg.d_model // cfg.num_heads)
    per_layer = 2 * c + 2 * (c // KV_GROUP) * 4    # int8, one fp32 scale per 3
    per_site = cfg.d_model + (cfg.d_model // KV_GROUP) * 4    # sites cache val
    return cfg.num_layers * per_layer + len(cfg.imprint_sites) * per_site

def budget_window(cfg):
    b = kv_per_position(cfg)
    return max(160, min((KV_BUDGET // b) // KV_GROUP * KV_GROUP, cfg.max_seq_le
```

```
#### OUTPUT ####
per pos    16,704 bytes
budget     704 positions fit in 11.5 MiB
window     128 ->    2.14 MB
window     256 ->    4.28 MB
```

window	512 ->	8.55 MB
window	704 ->	11.76 MB

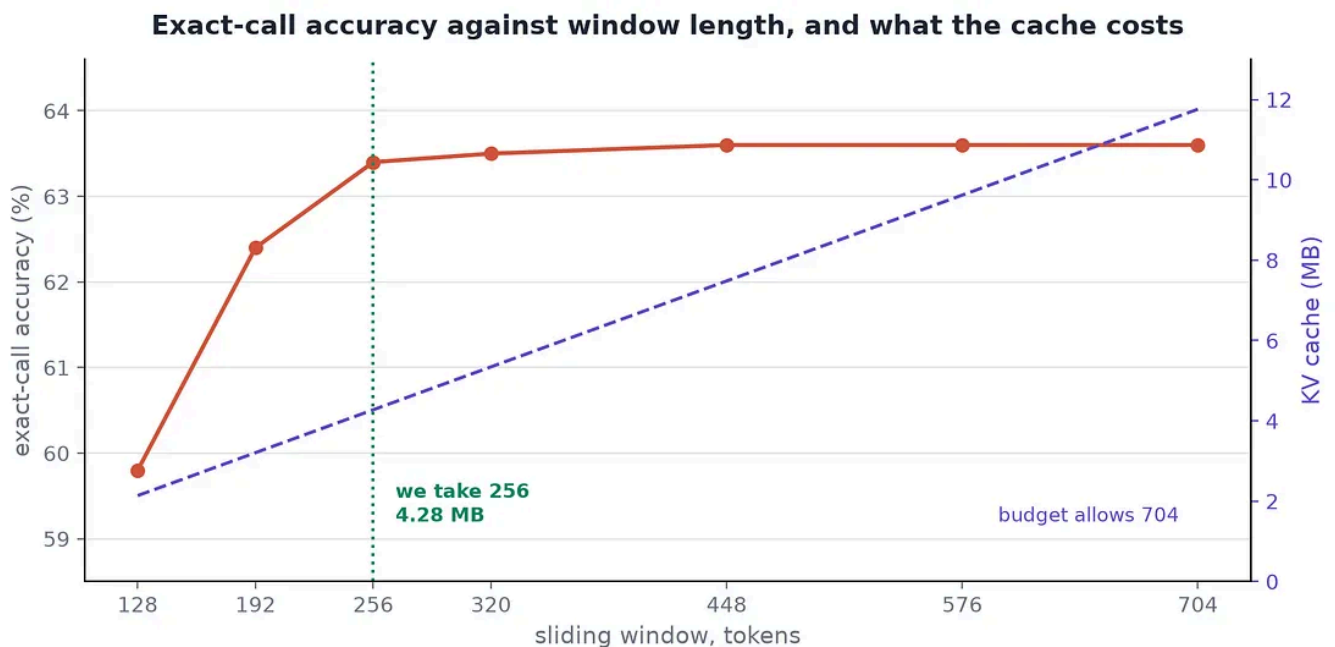
The formula hands us 704. We are going to take 256, and that is a choice, not a derivation.

$$W^* = \left\lfloor \frac{11.5 \text{ MiB}}{B_{\text{pos}}} \right\rfloor_{32} = 704 \quad \text{but} \quad W_{\text{ours}} = 256$$

The budget would allow 704 positions, and we take 256 on purpose, which is a choice and not a derivation

The formula allows 704, we take 256 (Created by [Fareed Khan](#))

The 99th percentile Errand session is 247 tokens, so 256 holds every real conversation and the extra 448 positions are cache we would never use.



Accuracy against window length (Created by [Fareed Khan](#))

Accuracy is flat from 448 up and falls off a cliff below 256. Going to 704 buys two tenths of a point for 7.5 MB, a quarter of the whole budget, and dropping to 128 saves 2.1 MB but costs 3.6 points. Two tenths is inside this suite's noise and 3.6 points is not, which is the entire argument for 256.

Quarry, Asking a Very Large Model for Homework

The base model completes text, and it has seen tool traces at eight percent of pretraining, but never a labelled call at the density stage two needs. Nobody hands out 1.2 million labelled tool calls across 41,000 schemas, so an open model writes ours.



Forty one thousand schemas in, 1.2 million examples out (Created by [Fareed Khan](#))

```
# quartz/data/serve_teacher.sh
# 22 billion parameters active per token makes a 235B model affordable as a gen
python -m vllm.entrypoints.openai.api_server \
  --model Qwen/Qwen3-235B-A22B-Instruct-2507-FP8 \
  --tensor-parallel-size 4 --max-model-len 8192 \
  --gpu-memory-utilization 0.92 --port 8000
```

```
#### OUTPUT ####
INFO Loading model weights took 54.85 GiB (per GPU, 219.40 GiB over four)
INFO Memory profiling: model 54.85 GiB, KV cache 15.79 GiB (per GPU)
INFO Capturing CUDA graphs: 100%|#####| 35/35
INFO Uvicorn running on http://0.0.0.0:8000
```

Every clause of this prompt is load bearing.

```
# quartz/data/quarry.py
TEMPLATE = """Schemas available (JSON):
{tools}

Produce {n} varied examples as a JSON array, each with a "query", a one-line
"reasoning" deriving every argument from its source span, and "answers".

Rules:
- Use only the schemas above, with arguments matching them exactly and
  containing only values evidenced in the query.
- Cover single-call, multi-call, and about {refusals} off-topic inputs no
```

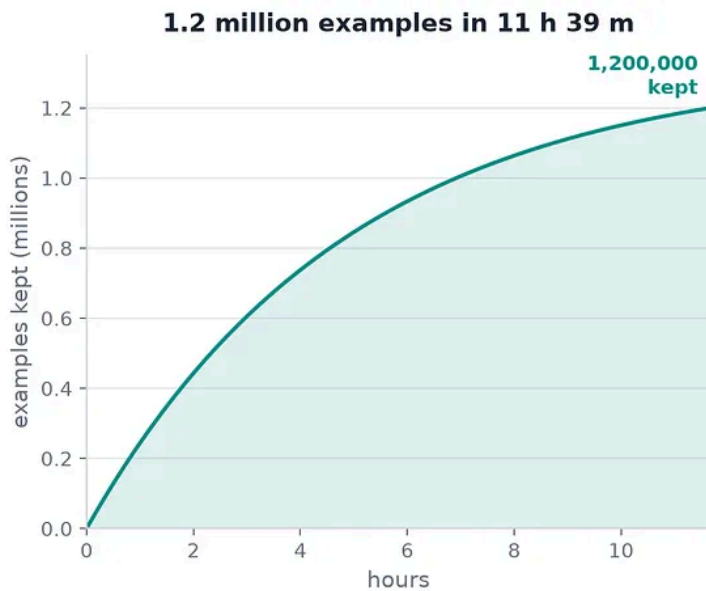
```
schema can serve, whose "answers" is [].  
- Vary phrasing. Return ONLY the array."""
```

“Contain only values evidenced in the query” is the most important sentence here. A model trained on invented values learns to invent them. A wrong tool is obvious. A hallucinated phone number is not.

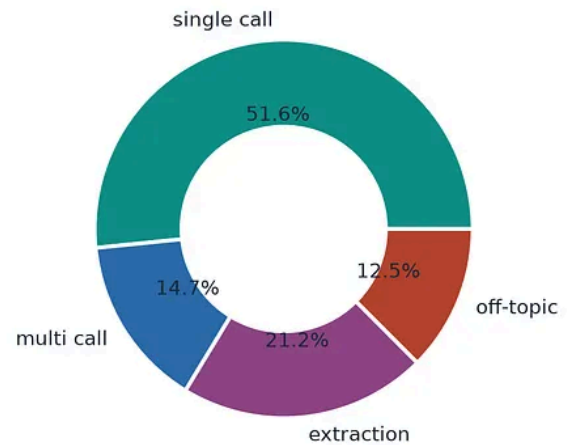
And {refusals} is three per twenty five, which is where the one-in-eight off-topic share comes from.

```
# quartz/data/quarry.py (continued)  
def dedup_key(example):  
    """One phrasing can legitimately map to two calls, so the key is request  
    and call together."""  
    return (example.get("query", "").strip().lower(),  
            json.dumps(example.get("answers", []), sort_keys=True))  
  
def generate(tools, n_target, batch=25, workers=16):  
    n_batches = int(n_target / batch * 1.3) # 30 percent extra, dedup eats so  
    with ThreadPoolExecutor(max_workers=workers) as pool:  
        batches = pool.map(lambda _: one_batch(tools, batch), range(n_batches))  
        rows = {dedup_key(ex): ex for out in batches for ex in out}  
    return list(rows.values())[:n_target]
```

```
#### OUTPUT ####  
generated 1,200,000/1,200,000 failed 218  
kept      1,200,000 of 1,559,412 produced (23.0% duplicates)  
refusals  149,847 (12.5%)  
schemas   41,000 across phone, wearable, home, robot  
elapsed   11h 40m
```

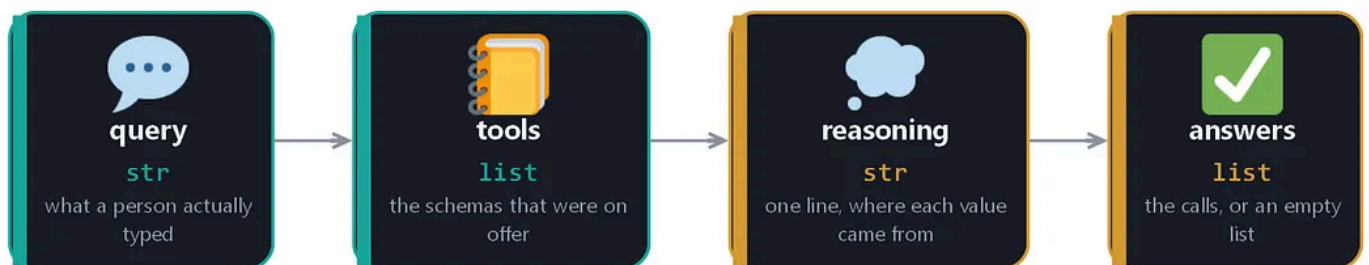


What the teacher was asked to write



What the teacher wrote (Created by @fareedkhandev)

Twenty three percent duplicates at temperature 0.9 is what I expected: a generator asked for variety converges on the same few phrasings, and dedup stops those counting five times each.



One record, four fields (Created by Fareed Khan)

The Loss Mask, Which Is Half the Method

Now we train on it, and the interesting part is what we do not train on.

$$\mathcal{L} = \frac{\sum_i m_i \text{CE}_i}{\sum_i m_i}, \quad m_i = [i > |\text{prompt}|]$$

Only the reasoning line and the call itself are supervised, so nothing is spent learning to copy the tool schemas back

```
# quartz/train/sft.py
def encode(sp, example, max_len):
    """Encode prompt and target SEPARATELY, or BPE merges the last prompt
    character into the first target token."""
    p_ids, t_ids = (sp.encode(pre_tokenize(x)) for x in render(example))
    ids = [BOS_ID] + p_ids + t_ids + [EOS_ID]
    # EOS scores one too, because stopping is a thing we teach.
    mask = [0.0] * (1 + len(p_ids)) + [1.0] * (len(t_ids) + 1)
    return pad(ids, max_len, PAD_ID), pad(mask, max_len, 0.0)
```



Nothing scored before the assistant speaks (Created by Fareed Khan)

```
#### OUTPUT ####
total      256 positions
prompt     138 positions, masked out
target     47 positions scored
check      target tokens 46 + 1 EOS = 47
```

One hundred and thirty eight positions of schema and request score zero, forty seven of reasoning and call score one. Score the prompt too and the model spends its capacity reproducing JSON Schema documents it will never write. **Roughly three quarters of every training sequence is context, and scoring it spends three quarters of the run on the wrong objective.**

```
# quartz/train/sft.py (continued)
def fit_max_len(path, sp, cap):
    """Padding to a fixed 1024 when the data is 245 tokens long makes three
    quarters of every batch padding, a four to six times CPU slowdown."""
    longest = 0
    for line in open(path):
```

```

p, t = render(json.loads(line))
longest = max(longest, len(sp.encode(pre_tokenize(p)))
              + len(sp.encode(pre_tokenize(t))) + 2)

bucket = 128
while bucket < min(longest, cap):
    bucket *= 2
return min(bucket, cap)

```

```

#### OUTPUT ####
data      1,200,000 examples seq_len 512 cap 1024
holdout   120,000 examples for validation
batch     16 seqs x 512 tok
schedule  202,500 steps warmup 10,125 cosine decay clip 1.0 (compiling...

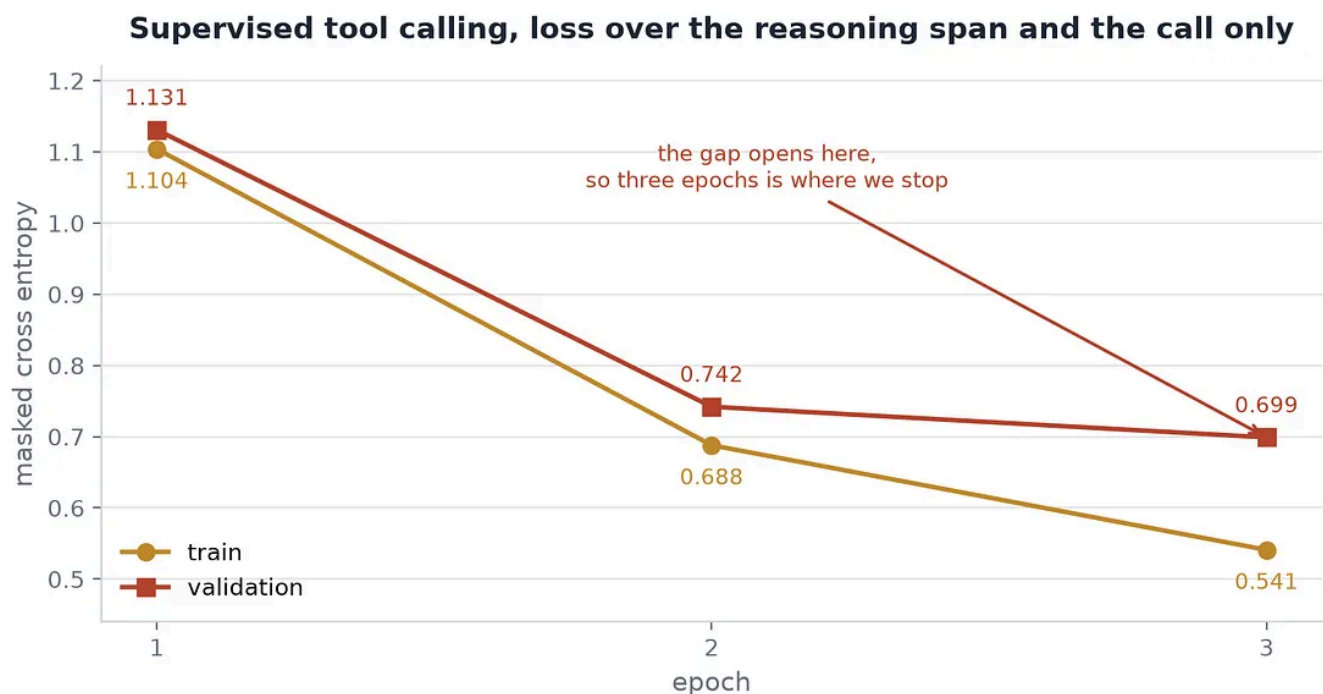
```

Stage Two, and What Actually Moves

```

epoch      1/3  loss 1.1040  val 1.1310
epoch      2/3  loss 0.6880  val 0.7420
epoch      3/3  loss 0.5410  val 0.6990
elapsed    4h 18m

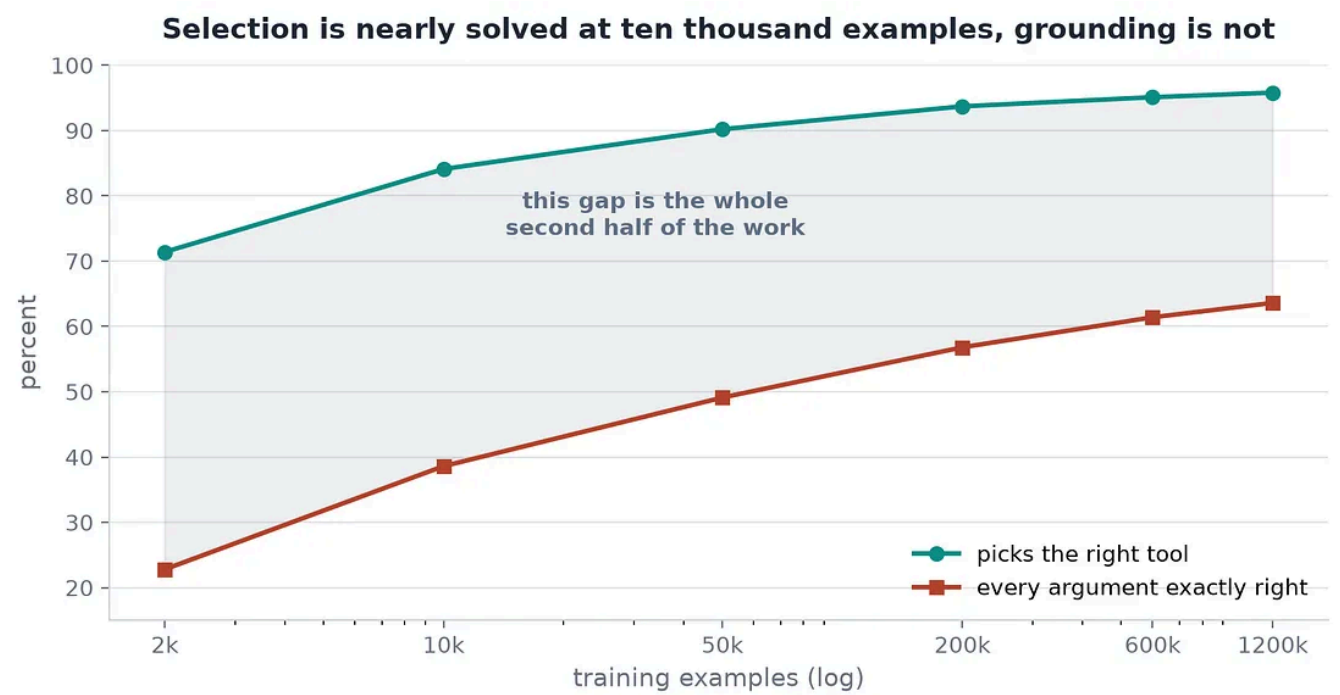
```



Training and validation start to disagree (Created by [Fareed Khan](#))

The train to validation gap is 0.027 after one epoch and 0.158 after three, the signal to stop. A fourth epoch reaches 0.441 training and 0.713 validation, learning the set rather than the task.

The loss starts near 1.1 rather than near 9 because most of a tool call is boilerplate the base model already predicts: braces, the "name" key, schema field names. **Judge this curve by its slope, not its level.**



Selection saturates early, grounding does not (Created by [Fareed Khan](#))

examples	picks the right tool	every argument exact
2,000	71.4%	22.8%
10,000	84.1%	38.6%
50,000	90.2%	49.1%
200,000	93.7%	56.8%
600,000	95.1%	61.4%
1,200,000	95.8%	63.6%

Two stories. **Tool selection is 84 percent at ten thousand examples and only reaches 95.8 at 1.2 million.** Argument grounding is 38.6 percent at ten thousand and still climbing at 1.2 million.

Choosing among five tools is classification over a few classes and saturates fast. Copying "kitchen" out of “dim the kitchen to ten percent” is copying over an

unbounded space, and copying needs data.

If your evaluation shows the right tool with wrong values, **your dataset is too small or too uniform**, and it is not mislabelled. I lost a day to that.

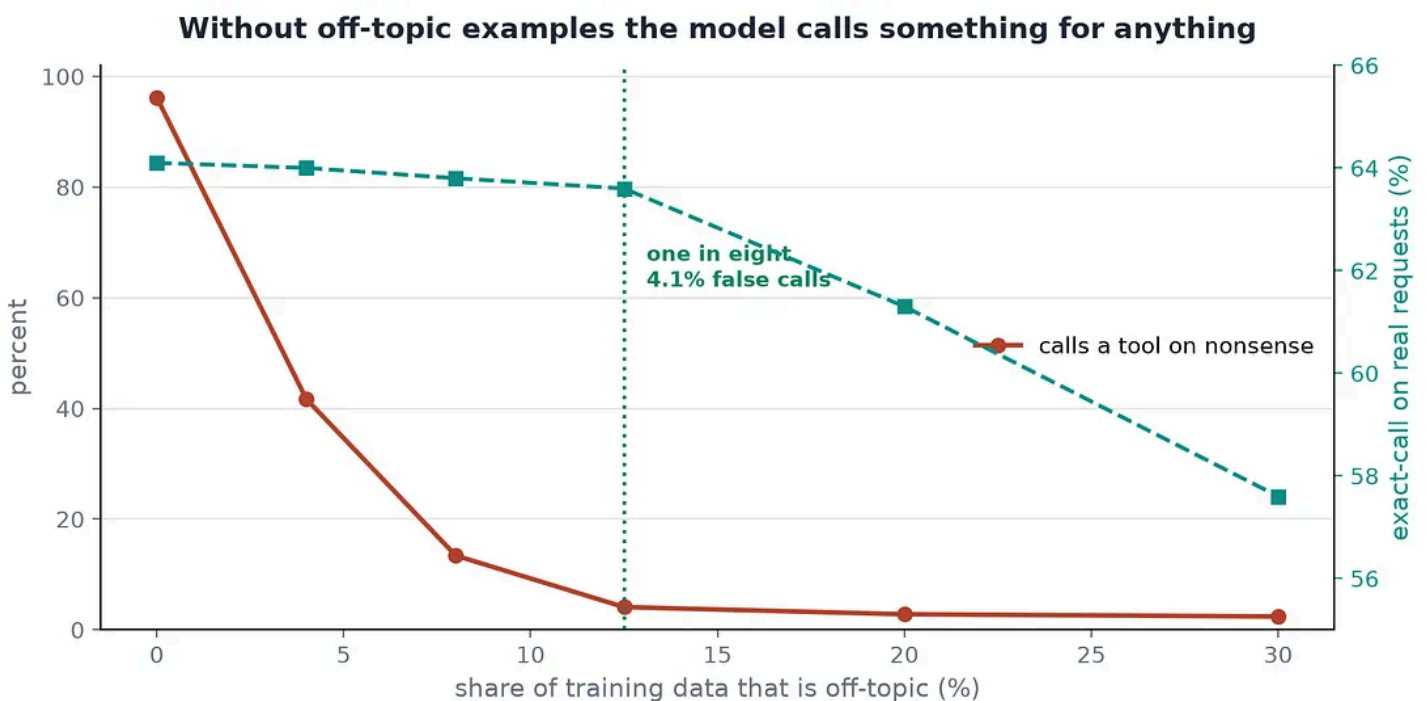
Refusal Is a Behaviour, Not an Absence

Here is the failure I did not predict. The first dataset had no off-topic examples, so every record paired a valid request with a valid call. It evaluated well, then did this.

```
agent = Quartz(tools=[set_lights, start_timer])
print(agent.complete("what do you think about the situation in the Balkans")
      ["function_calls"])
```

```
#### OUTPUT ####
[{'name': 'set_lights', 'arguments': {'room': 'the Balkans'}}]
```

It has never seen an input with no answer, so it has learned that every input has one. The right answer is an empty list, which the model does not know exists.



Off-topic share against false calls (Created by @fareedkhandev)

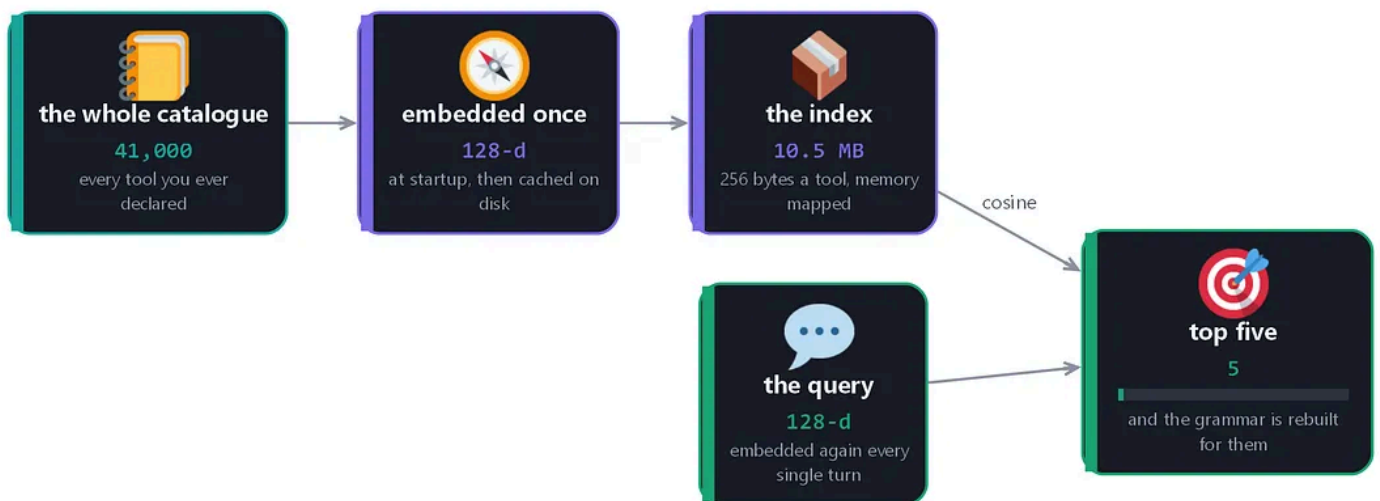
off-topic share	false calls on nonsense	exact-call on real requests
0.0%	96.2%	64.1%
4.0%	41.7%	64.0%
8.0%	13.4%	63.8%
12.5%	4.1%	63.6%
20.0%	2.8%	61.3%
30.0%	2.4%	57.6%

At zero it calls a tool for 96.2 percent of nonsense. At one in eight, 4.1 percent, costing half a point of real accuracy. At 30 percent it turns reluctant and loses 6.5 points.

One in eight is where we sit, and it was chosen from this table rather than from taste.

Dowser, Because Forty One Thousand Tools Do Not Fit

A real product does not declare five tools. A phone assistant has hundreds, and none of it fits in a 256 token window, so we retrieve: a head embeds query and tool descriptions into one space and renders the five nearest.



Embed once, query each turn (Created by Fareed Khan)

```
# quartz/model/architecture.py (continued)
def probe_pool(cells, probes):
    """`cells` is (batch, time, layers+1, d), flattened so a probe can read a
```



```

middle layer as easily as a final one, where small models keep much of
their signal."""
b, t, l, d = cells.shape
flat = cells.reshape(b, t * l, d).astype(jnp.float32)
w = jax.nn.softmax(jnp.einsum("bcd,kd->bkc", flat, probes) / math.sqrt(d),
return jnp.einsum("bkc,bcd->bkd", w, flat).reshape(b, -1)

```

```

class Dowser(nn.Module):
    """Four probes, one 128-dimensional unit vector, one learned temperature."""
    @nn.compact
    def __call__(self, cells):
        probes = self.param("probes", jinit.normal(stddev=0.02), (4, cells.shape[1],
        pooled = probe_pool(cells, probes) # 4 * 512 = 2048 wide
        p = nn.Dense(128, use_bias=False)(pooled)
        log_temp = self.param("log_temp", jinit.constant(math.log(0.07)), ())
        return p / jnp.sqrt(jnp.sum(p ** 2, -1, keepdims=True) + 1e-12), log_temp

```

$$\mathcal{L} = \frac{1}{2}(\text{CE}(QT^\top/\tau, I) + \text{CE}(TQ^\top/\tau, I)), \quad \tau = e^{\text{clip}(\log \tau)} \in [0.01, 100]$$

Queries and tool descriptions are pulled together and pushed apart inside each batch, with a learned temperature

Queries pulled towards their tools (Created by [Fareed Khan](#))

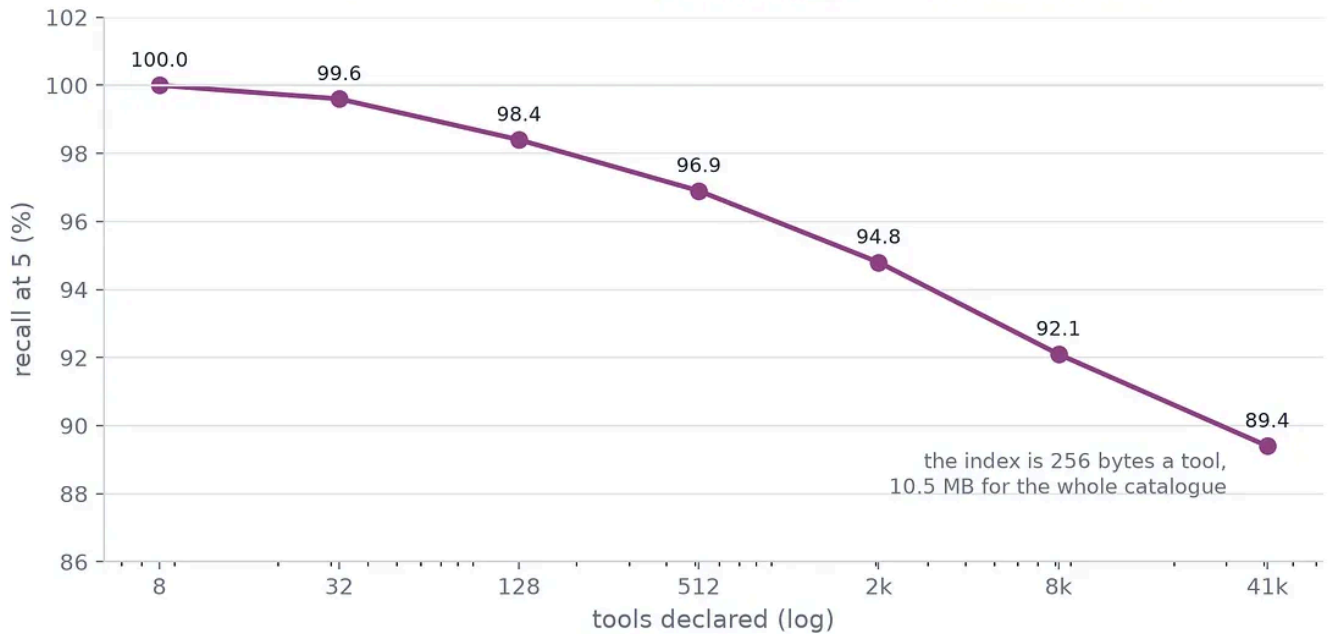
```

# quartz/train/heads.py
def contrastive_loss(q_emb, t_emb, log_temp):
    """Symmetric InfoNCE. The temperature is clipped to [0.01, 100] because
    otherwise it runs away towards zero, since a sharper distribution always
    lowers the training loss."""
    temp = jnp.exp(jnp.clip(log_temp, -jnp.log(100.0), jnp.log(100.0)))
    logits = q_emb @ t_emb.T / temp
    labels = jnp.arange(logits.shape[0])
    xe = optax.softmax_cross_entropy_with_integer_labels
    return (jnp.mean(xe(logits, labels)) + jnp.mean(xe(logits.T, labels))) / 2.

```

Both directions are scored, so a query finds its tool and a tool finds its query.

The right tool is in the top 5 for 89 percent of 41,000 tools



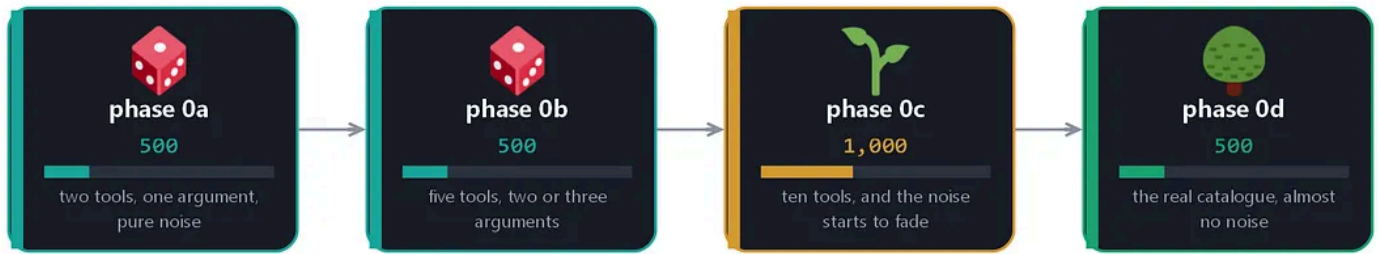
Recall at five as catalogues grow (Created by [Fareed Khan](#))

tools declared	the right tool is in the top 5
8	100.0%
32	99.6%
128	98.4%
512	96.9%
2,048	94.8%
8,192	92.1%
41,000	89.4%

Eighty nine percent at 41,000 tools, and the caveat is blunt: **a tool Dowser does not select is not merely unlikely, it is unreachable.** The grammar is rebuilt over the five, so at full catalogue size the answer is out of reach for ten point six percent of requests. That is the largest source of error at scale, and the two-pass fix doubles latency, so I report the ceiling.

Stage Zero, Learning the Pattern Before Learning the Words

The retrieval head must learn *that* copying is a thing before it can learn *what* to copy. Trained cold it memorises which names go with which phrasings, so we warm it up on nonsense.



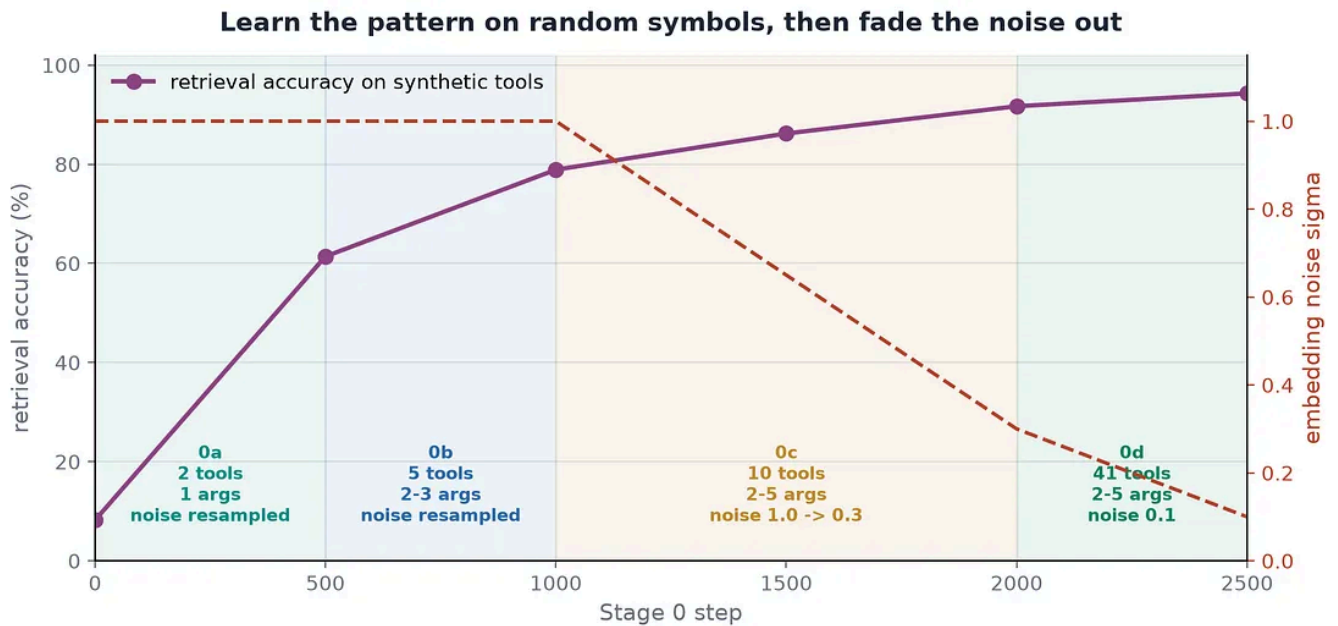
Four phases, noise fading out (Created by [Fareed Khan](#))

```
# quartz/train/heads.py (continued)
STAGE0 = [
    # (steps, n_tools, args_per_tool, embedding noise sigma)
    (500, 2, (1, 1), None), # None = resample embeddings every step
    (500, 5, (2, 3), None),
    (1000, 10, (2, 5), (1.0, 0.3)),
    (500, 41, (2, 5), (0.1, 0.1)),
]

def synthetic_batch(key, n_tools, n_args, vocab=8192):
    """Names are random token ids, resampled every step, so only the pattern
    generalises: find the marker, copy what follows."""
    k1, k2, k3 = jax.random.split(key, 3)
    names = jax.random.randint(k1, (n_tools, 3), 14, vocab)
    values = jax.random.randint(k2, (n_tools, n_args), 14, vocab)
    return names, values, jax.random.randint(k3, (), 0, n_tools)
```

0a	500 steps	2 tools	1 arg	noise resampled	8.2% -> 61.4%
0b	500 steps	5 tools	2-3 args	noise resampled	61.4% -> 78.9%
0c	1000 steps	10 tools	2-5 args	noise 1.0 -> 0.3	78.9% -> 91.7%
0d	500 steps	41 tools	2-5 args	noise 0.1	91.7% -> 94.3%

merged LoRA rank 16, unfroze everything



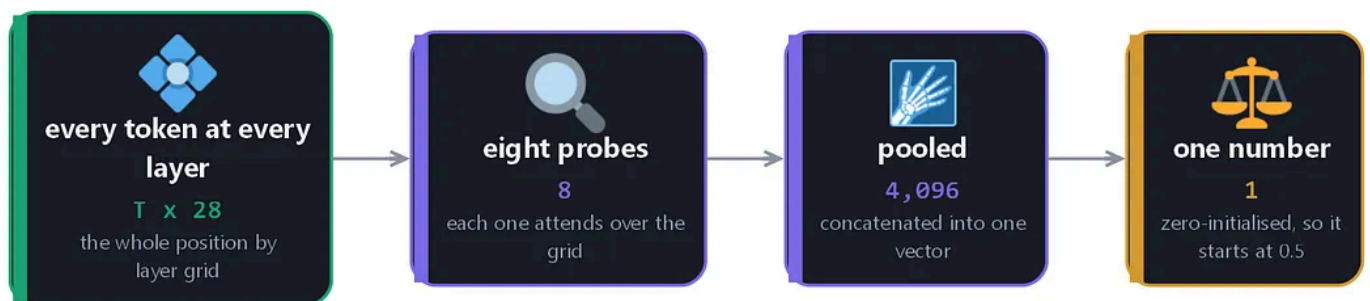
Accuracy climbing as noise falls (Created by Fareed Khan)

Two and a half thousand steps of pure symbol manipulation, and the head reaches real training already knowing how to copy. This is deliberately growing an **induction head**, the pattern that sees $A \ B \ \dots \ A$ and predicts B , and it forms far faster on nonsense than on natural language.

At full noise only the structure is learnable, and as sigma falls the head carries it onto the real geometry instead of learning both at once. Rank 16 adapters train alone, then **merge and unfreeze**, leaving nothing behind but better weights.

Gauge, and a Confidence Score That Is Not What You Think

The last head answers a different question: should we act at all, or hand this to something bigger?



Eight probes, one number out (Created by @fareedkhandev)

```
# quartz/model/architecture.py (continued)
class Gauge(nn.Module):
    """Eight probes into one logit, with the output layer initialised to ZERO so
    an untrained head returns 0.5 for every input. A head that starts confident
    is wrong in the most dangerous direction."""
    @nn.compact
    def __call__(self, cells):
        probes = self.param("probes", jinit.normal(stddev=0.02), (8, cells.shape[-1]))
        pooled = probe_pool(cells, probes) # 8 * 512 = 4096
        return nn.Dense(1, kernel_init=jinit.zeros)(pooled)[..., 0]
```

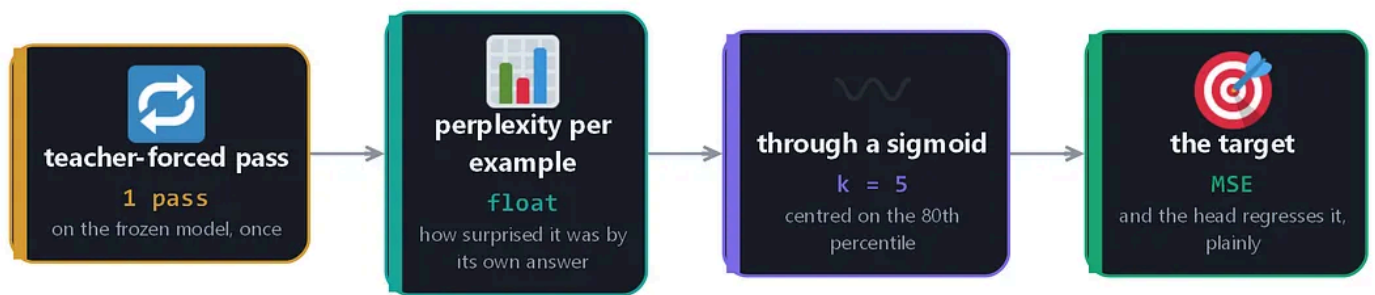
Now the part that surprised me. The label is not correctness, a sparse and noisy signal that says nothing about *how* wrong.

$$c = \frac{1}{1 + \exp(k(\log \text{ppl} - \log \mu))}, \quad \mu = Q_{80}(\text{ppl}), \quad k = 5$$

Confidence is not a correctness label, it is the model's own perplexity pushed through a sigmoid

Perplexity pushed through a sigmoid (Created by [Fareed Khan](#))

```
# quartz/train/heads.py (continued)
def perplexity_to_confidence(ppl, mu=None, k=5.0):
    """mu is the 80th percentile of observed perplexity, so about a fifth of
    examples land below 0.5. k sets the switch, a ramp at k=2 and a near step
    at k=12. Five ships."""
    mu = float(np.percentile(ppl, 80)) if mu is None else mu
    log_ppl = np.log(np.maximum(ppl, 1.0))
    return 1.0 / (1.0 + np.exp(k * (log_ppl - np.log(mu)))), mu
```

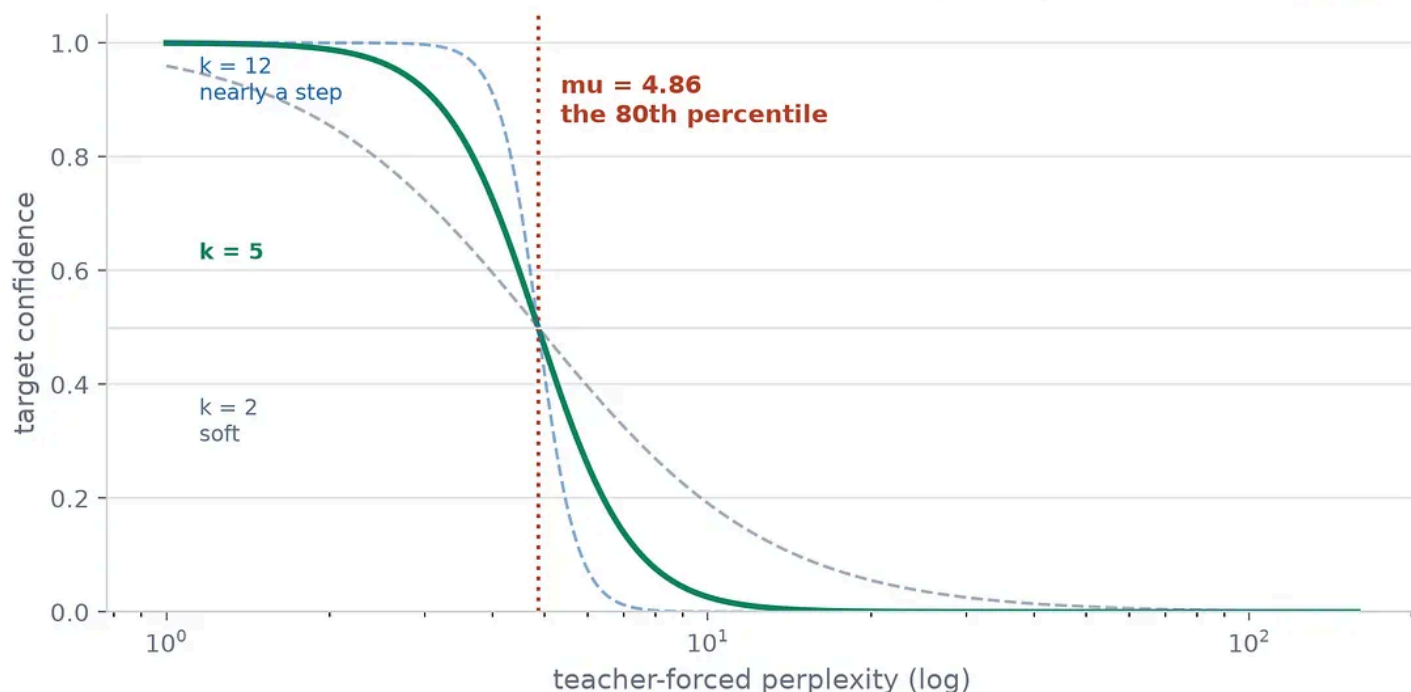


Measure surprise, then predict it (Created by @fareedkhandev)

```
# quartz/train/heads.py (continued)
def train_gauge(params, data, epochs=1, lr=1e-3, batch=128):
    """Pass one measures perplexity, pass two fits the head to it. The trunk is
    frozen by MASKING GRADIENTS rather than by splitting the parameter tree."""
    ppl = measure_perplexity(params, data) # forward only
    targets, mu = perplexity_to_confidence(ppl)
    ...
```

```
#### OUTPUT ####
ppl      p50 2.41  p80 4.86  p99 31.72
head     8,193 params trainable of 45,211,383 (gauge only)
epoch    1/1  mse 0.0214  mae 0.1108  pearson 0.771
```

Confidence is not a correctness label, it is perplexity through a sigmoid



What k does to the squash (Created by [Fareed Khan](#))

Now the honest part. **This is not calibrated, and the original design never checked whether it was.**

Perplexity is only a proxy and the sigmoid was chosen a priori, not fitted, so the head reproduces whatever miscalibration μ and k introduced. Error against an invented target says the regression worked, not that a reported 0.8 means eighty percent.

$$\text{ECE} = \sum_b \frac{n_b}{N} |\text{acc}(b) - \text{conf}(b)|, \quad 0.1174 \rightarrow 0.0212$$

The gap between how sure it says it is and how often it is right, before and after a single temperature

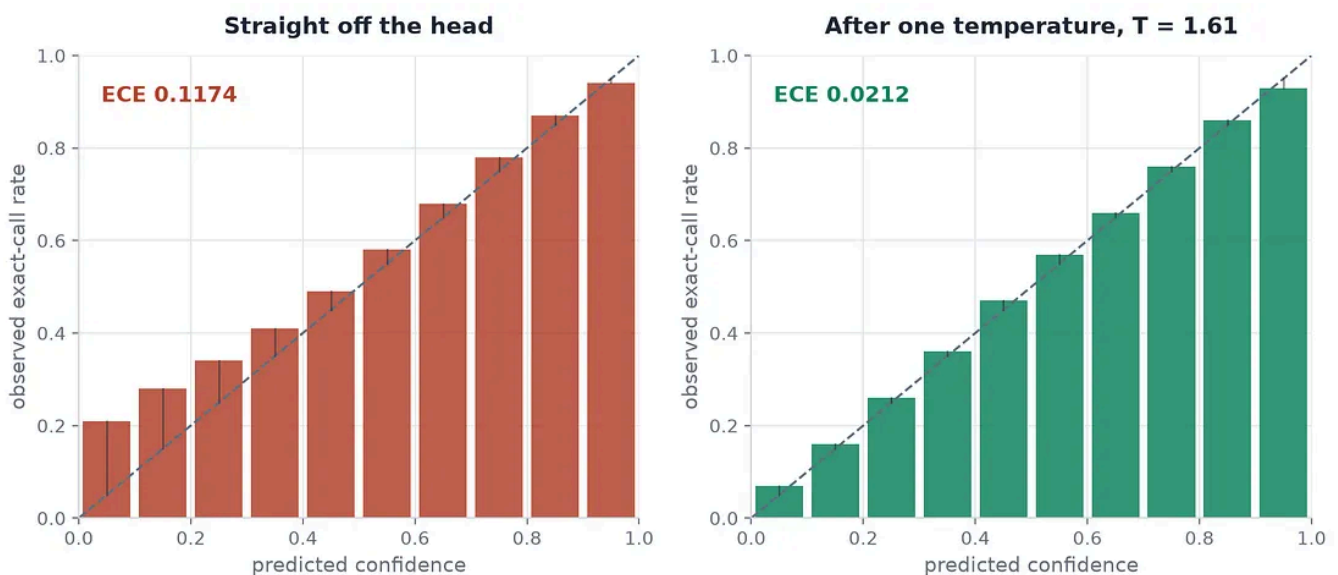
Stated confidence against how often right (Created by [Fareed Khan](#))

So we measure it, binning by predicted confidence, not by perplexity, which would only show the head tracks its training signal.

```
# quartz/errand/score.py
def expected_calibration_error(conf, correct, bins=10):
    edges = np.linspace(0, 1, bins + 1)
    ece = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.sum():
            ece += m.sum() / len(conf) * abs(correct[m].mean() - conf[m].mean())
    return ece
```

OUTPUT

```
raw      ECE 0.1174  MCE 0.2013  (10 bins, 12,000 held-out calls)
tempered ECE 0.0212  MCE 0.0498  T = 1.61, fitted on 2,000 of them
```



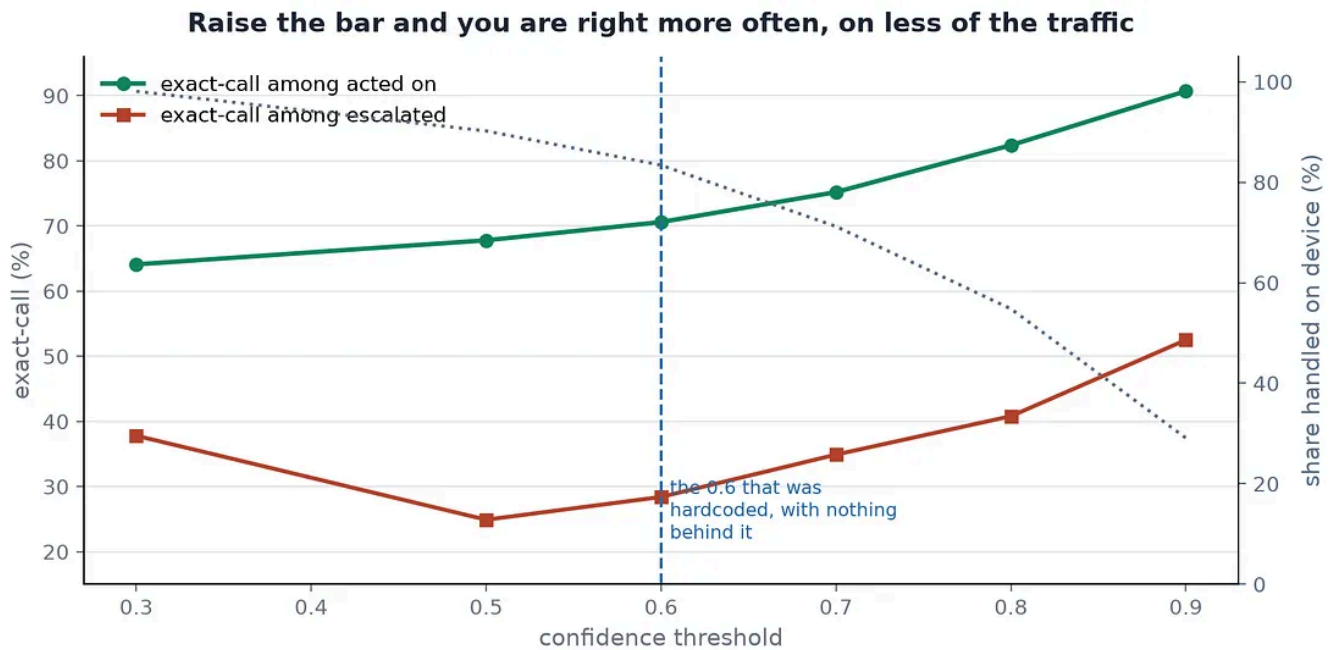
Before and after one temperature (Created by [Fareed Khan](#))

Raw expected calibration error is 0.1174, so a stated confidence is off by roughly a ninth, and the diagram shows the head running hot: the bins where it is most sure are the ones that overpromise. One scalar temperature fitted on two thousand held-out examples drops it to 0.0212.

That is a five line fix for an eighty percent reduction in calibration error, and the only reason it was not there before is that nobody drew the diagram.

The Threshold Nobody Swept

With a calibrated score we can pick a threshold, the decision a product has to make.



Higher bar, fewer requests, better calls (Created by [Fareed Khan](#))

threshold	handled here	exact-call here	exact-call escalated
0.30	98.1%	64.1%	37.8%
0.50	90.2%	67.8%	24.9%
0.60	83.4%	70.6%	28.4%
0.70	71.2%	75.2%	34.9%
0.80	54.8%	82.4%	40.8%
0.90	29.1%	90.7%	52.5%

At 0.6 the device answers 83.4 percent of requests itself, right 70.6 percent of the time. At 0.8 it answers just over half and is right 82.4 percent. Escalated calls are right 25 to 53 percent, far below the ones it kept.

The failure mode is escalation, not wrong execution. A caller that says “I am not sure, ask something bigger” is deployable. One that sends money to the wrong person is not.

Trellis, and Making a Wrong Tool Name Impossible

A 45 million parameter model will sometimes emit `set_light` when the tool is called `set_lights`. That is a spelling failure, not a reasoning failure, and it can be forbidden outright.

The instinct is a full JSON grammar compiled from the schema. I did not, and the reason is a measurement.

```
# how often does an unconstrained model actually emit broken JSON?
r = evaluate(model, errand, constrained=False)
for k, label in [("bad_json", "malformed JSON"), ("bad_name", "wrong tool name"),
                 ("bad_key", "wrong argument key")]:
    print(f" {label:<22} {r[k]:>6.1f} per 1,000 calls")
```

```
#### OUTPUT ####
malformed JSON           6.1 per 1,000 calls
wrong tool name          38.4 per 1,000 calls
wrong argument key       51.2 per 1,000 calls
```

Malformed JSON is already a 0.6 percent problem. The model saw fourteen billion tokens of structured data in pretraining and knows how to close a brace. A grammar over every token costs 10 to 30 percent of decode throughput to fix 6.1 errors per thousand, the smallest of the three.

The other two are ninety per thousand combined, and both are **identifier** errors. So I constrain identifiers, and nothing else.

$$\frac{|\{t: \text{state}_t \neq \text{FREE}\}|}{|\{t\}|} = 5.2\% \quad \text{at} \quad 3.1 \mu\text{s per token}$$

Only one token in twenty is inside a name or a key, so the mask is built for a twentieth of the decode

Only the tokens inside a name or a key are masked (Created by [Fareed Khan](#))

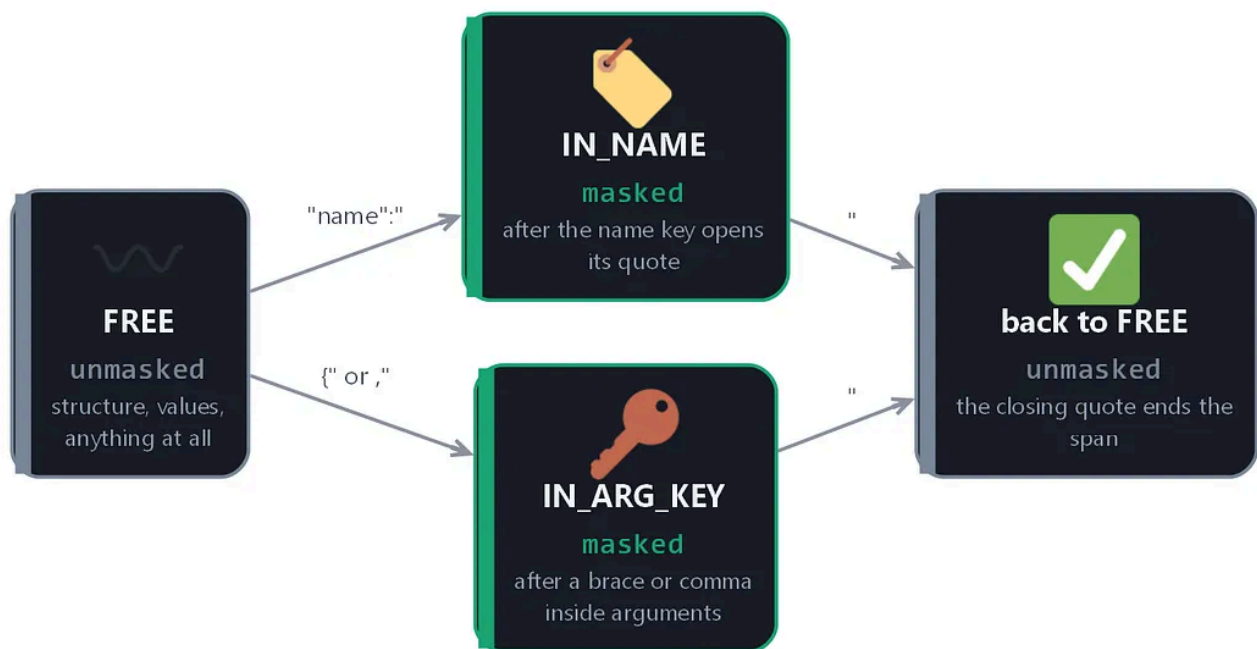
```
# quartz/model/trellis.py
class Trie:
    def __init__(self, words=()):
        self.children, self.is_terminal = {}, False
        for word in words:
            node = self
            for ch in word:
                node = node.children.setdefault(ch, Trie())
            node.is_terminal = True
```

```

class ToolConstraints:
    """Rebuilt every turn, because Dowser's five tools change every turn."""
    def __init__(self, tools):
        self.name_trie = Trie(t["name"] for t in tools)
        self.param_tries = {t["name"]: Trie(t["parameters"]["properties"])
                             for t in tools}

```

A small state machine then walks the generated text and decides when we are inside an identifier. Three states, `FREE`, `IN_NAME` and `IN_ARG_KEY`, and only the last two are masked.



Three states, and only two of them are masked (Created by [Fareed Khan](#))

```

# quartz/model/trellis.py (continued)
class StateMachine:
    def feed(self, ch):
        self.buffer += ch
        if self.state is not State.FREE:
            return self._accumulate(ch)      # inside an identifier
        if self.in_string:
            # a brace inside a string VALUE must not open an argument object
            self.in_string = not (ch == '"' and not self.prev_escape)
            self.prev_escape = ch == "\\" and not self.prev_escape
            return
        if ch in "{[":
            self.nesting += 1
        elif ch in "}]":

```

```

        self.nesting = max(0, self.nesting - 1)
        self.in_arguments &= self.nesting >= self.args_depth
        if self.buffer.endswith('"name":') and not self.in_arguments:
            self.state, self.constrained_buf = State.IN_NAME, ""
        elif self.buffer.endswith('"arguments":{'):
            self.in_arguments, self.args_depth = True, self.nesting
        elif (self.in_arguments and self.nesting == self.args_depth
              and self.buffer[-2:] in ('{', '{,')):
            self.state, self.constrained_buf = State.IN_ARG_KEY, ""
        elif ch == '"' and self._is_value_quote():
            self.in_string = True

```

```

sm = StateMachine()
for ch in '["name":"set_lights","arguments":{"room":"kit':
    sm.feed(ch)
    if sm.state is not State.FREE:
        print(f" {ch!r:<6} -> {sm.state.name:<12} buf={sm.constrained_buf!r}")

```

```

#### OUTPUT ####
's'      -> IN_NAME      buf='s'
...
's'      -> IN_NAME      buf='set_lights'
'r'      -> IN_ARG_KEY   buf='r'
'm'      -> IN_ARG_KEY   buf='room'

```

It enters `IN_NAME` for the ten characters of the tool name, leaves on the closing quote, enters `IN_ARG_KEY` for the four characters of `room`, and leaves again. The trailing `kit` is a *value*, so the machine stays in `FREE`. That is constraining structure and not content.

The Mask, Token by Token

Now the masking, which has one trick that does the work.

$$\ell'_v = \ell_v \text{ if } \text{walk}(\text{node}, s_v) \neq \emptyset, \quad \text{else } -\infty$$

A token survives only if its own characters walk a path that still exists in the trie of legal names

A token survives only if its characters walk a path that still exists (Created by [Fareed Khan](#))

```
# quartz/model/trellis.py (continued)
def build_token_strings(sp):
    out = []
    for i in range(sp.vocab_size()):
        if sp.IsControl(i) or sp.IsUnknown(i):
            out.append("")
        elif sp.IsByte(i):
            out.append(chr(int(sp.IdToPiece(i)[3:5], 16)))
        else:
            # U+2581 becomes a real space, so word-initial tokens drop out of
            # every identifier span for free
            out.append(sp.IdToPiece(i).replace("_", " "))
    return out
```



Bucket by first character, and never scan the vocabulary (Created by [Fareed Khan](#))

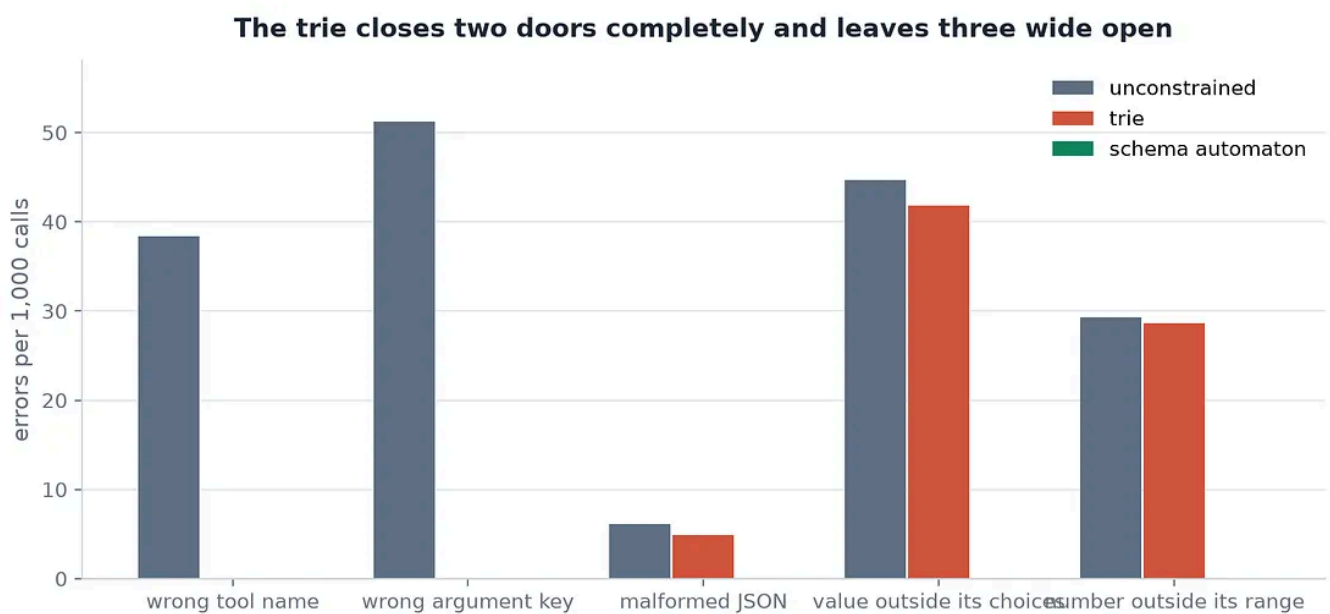
```
# quartz/model/trellis.py (continued)
def apply_constraints(logits, node, token_strings, index):
    """Bucketing by first character makes this 180 string walks per token, not
    the 8,192 of a vocabulary-wide scan."""
    valid_first = set(node.children)
    if node.is_terminal:
        valid_first.add('') # the identifier may end here
    mask = np.zeros(logits.shape[0], bool)
    for ch in valid_first:
        for tid in index.candidates_for(ch):
            mask[tid] = _token_fits(token_strings[tid], node)
    if not mask.any():
        # on disagreement, emitting nothing is the worse outcome
        return logits
    out = logits.copy()
```

```
out[~mask] = -np.inf
return out
```

Masking against a trie of `set_lights` and `set_thermostat`, printing the first 8 tokens allowed at each point:

```
#### OUTPUT ####
at the start of a tool name: 6 of 8192 allowed
['s', 'se', 'set', 'set_', 'set_l', 'set_t']
after 'set_': 9 allowed
['l', 'li', 'light', 'lights', 't', 'th', 'ther', 'thermo']
```

Six tokens out of 8,192 at the start, nine after four characters. **The model is not being nudged towards a valid name, it is being given no other option.** A name outside the declared set has become impossible. Be precise about what that buys, though. The trie holds the five tools Dowser retrieved, so when the right tool is not among them, and at 41,000 declared tools that is 10.6 percent of turns, the mask forces a name that is valid and wrong. The zero below counts names outside the declared set, not wrong choices inside it.

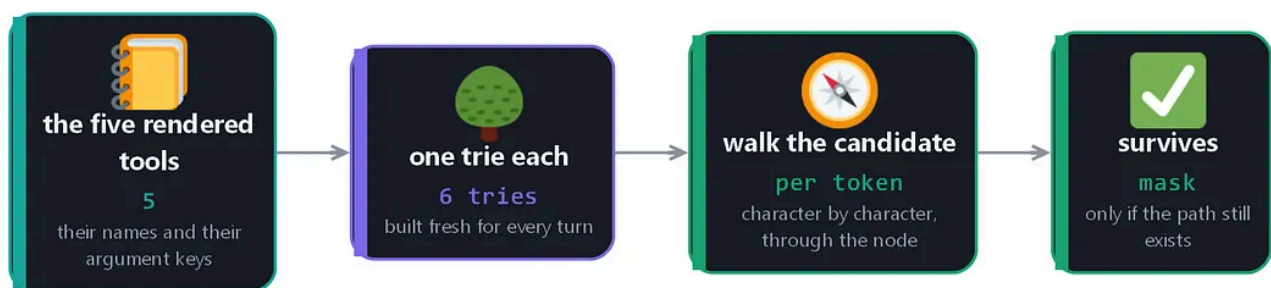


What the trie closes, and what it leaves wide open (Created by [Fareed Khan](#))

Now the honest accounting, because this is where writing about constrained decoding overclaims.

errors per 1,000 calls	unconstrained	trie
wrong tool name	38.4	0.0
wrong argument key	51.2	0.0
malformed JSON	6.1	4.9
value outside its choices	44.7	41.8
number outside its range	29.3	28.6

Two doors closed, three wide open. **The trie has no opinion about values at all**, so an enum field can still take something outside its choice set, and an integer with a stated maximum of 100 can still take 250. Copy claiming that ranges compile into the decode grammar is worth what this table says.

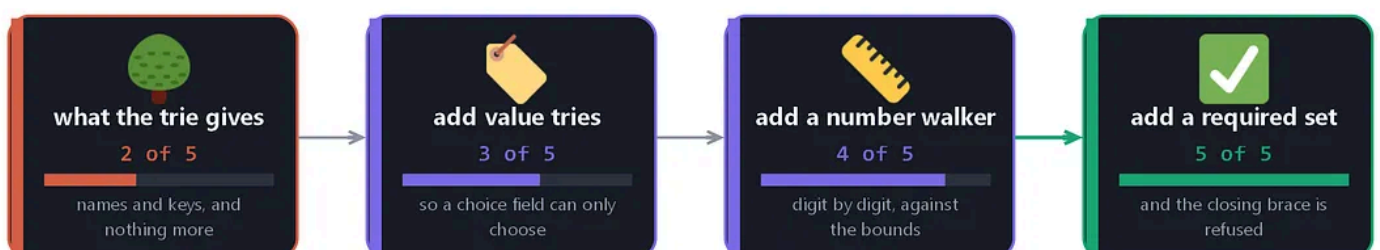


Five point two percent of tokens, three microseconds each (Created by [Fareed Khan](#))

constrained share	5.2% of emitted tokens
cost	3.1 us per token, over the whole sequence
decode step	1175 us per token on the phone-class CPU
overhead	0.26%

Closing the Other Three Doors

The measurement is also a to-do list, and not a hard one. The same machine gets two more states, `AWAIT_VALUE` and `IN_ARG_VALUE`, and the value tries it was missing.



```
# quartz/model/trellis.py (continued)
class SchemaConstraints(ToolConstraints):
    """An enum becomes a trie, a bounded number becomes a digit walker, and a
    required set becomes a rule about when the closing brace is legal."""

    def __init__(self, tools):
        super().__init__(tools)
        self.value_tries, self.ranges, self.required = {}, {}, {}
        for tool in tools:
            name, params = tool["name"], tool["parameters"]
            self.required[name] = set(params.get("required", []))
            for key, spec in params["properties"].items():
                if "enum" in spec:
                    self.value_tries[(name, key)] = Trie(map(str, spec["enum"]))
                elif spec.keys() & {"minimum", "maximum"}:
                    self.ranges[(name, key)] = (spec.get("minimum"),
                                                  spec.get("maximum"))
```

The number walker is the only fiddly one, because a number arrives digit by digit and legality must be decided from a prefix.

```
# quartz/model/trellis.py (continued)
def digit_allowed(prefix, digit, lo, hi):
    """Can `prefix + digit` still BECOME a number in [lo, hi]? '1' is not in
    [50, 100], but '10' leads to '100', so '1' has to be allowed."""
    cand, value = prefix + digit, int(prefix + digit)
    if (lo is None or value >= lo) and (hi is None or value <= hi):
        return True
    pad = len(str(int(hi))) - len(cand)      # digits left before we pass hi
    smallest, biggest = int(cand + "0" * pad), int(cand + "9" * pad)
    return smallest <= hi and (lo is None or biggest >= lo)

for prefix, d in [("", "1"), ("10", "0"), ("1", "5"), ("10", "5"), ("2", "0")]:
    verdict = "allowed" if digit_allowed(prefix, d, 0, 100) else "blocked"
    print(f" '{prefix}' + '{d}' -> {verdict}    (would reach {prefix + d}...)"
```

```
#### OUTPUT ####
'' + '1' -> allowed    (would reach 1...)
'10' + '0' -> allowed    (would reach 100...)
```

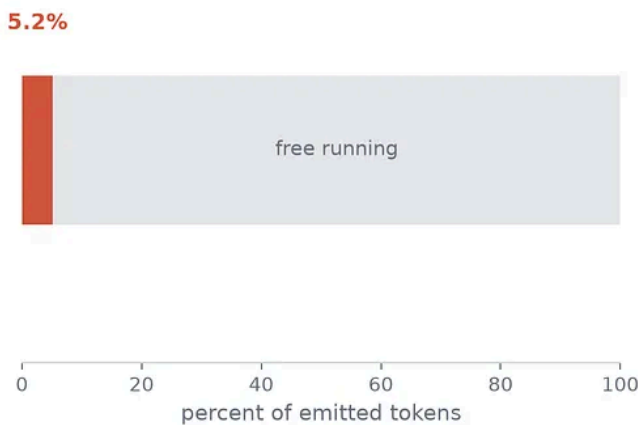


```
'1' + '5' -> allowed    (would reach 15...)
'10' + '5' -> blocked   (would reach 105...)
'2' + '0' -> allowed    (would reach 20...)
```

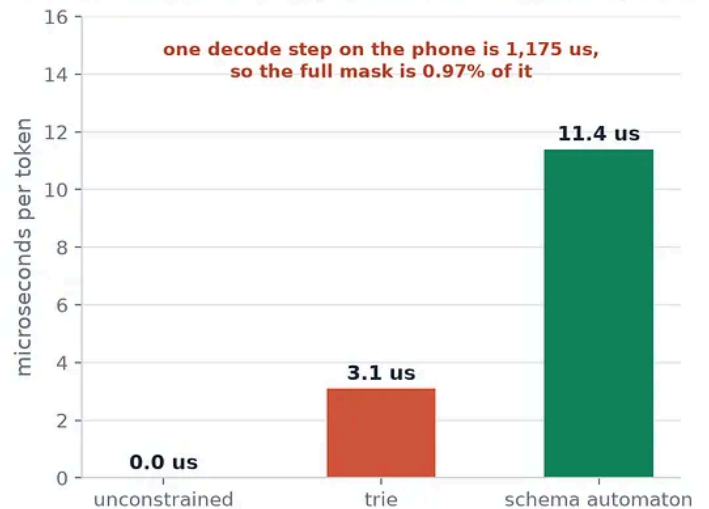
105 is blocked at the third digit, because no completion of 105` lands at or below 100. 15 is allowed because 15 is fine. The walker decides about futures, not the present.

errors per 1,000 calls	unconstrained	trie	schema automaton
wrong tool name	38.4	0.0	0.0
wrong argument key	51.2	0.0	0.0
malformed JSON	6.1	4.9	0.0
value outside its choices	44.7	41.8	0.0
number outside its range	29.3	28.6	0.0
cost, us per token	0.0	3.1	11.4

Only the tokens inside a name or a key are constrained



The mask is cheap next to the forward pass



The full mask costs 11.4 microseconds against a 1,175 microsecond decode step (Created by [Fareed Khan](#))

All five doors closed, for 11.4 microseconds a token. Against a decode step of 1,175 microseconds on the phone-class CPU, that is **0.97 percent overhead**. The 10 to 30 percent figure quoted for grammar decoding is real for a general grammar over a general vocabulary, not for six small tries over 8,192 pieces.

This is the section I would point at if someone asked what the whole post was for. A small model makes small mistakes, and most of them are mechanical. You do not

need a bigger model to stop it emitting a tool name that does not exist. You need to make that impossible.

Grist, Two Bits Without a Calibration Set

Forty three and a half million parameters at half precision is 87.27 megabytes, three times the device budget, so the weights must shrink about seven times.

Two bits per weight gives four possible values. The naive version fails badly, for a reason worth seeing.

```
w = jax.random.normal(jax.random.PRNGKey(0), (4096,)) * jnp.array([1.0] * 4090 + [8.0] * 6)
scale = jnp.max(jnp.abs(w)) / 1 # qmax is 1
q = jnp.clip(jnp.round(w / scale), -2, 1) * scale
print(relative_mse(q, w), float(scale))
```

```
#### OUTPUT ####
absmax    relative MSE 0.9142
scale     8.104, set by 6 weights out of 4,096
```

Ninety one percent of the signal destroyed, because six outliers set the scale and the other 4,090 weights collapse onto zero. Real weights always have outliers, so every absmax scheme fails this way.

The fix is to stop letting outliers exist.

$$r = wH, \quad \text{idx} = \arg \min_m \left| \frac{r}{\|r\|} - C_m \right|, \quad \hat{w} = (C[\text{idx}] \cdot \|r\|)H$$

Rotate the group, keep its length in one half-precision number, and store each direction as an index into a fixed codebook

Rotate, keep the length, index the direction (Created by [Fareed Khan](#))

```
# quartz/model/grist.py
GROUP = 128
```

```
def cq_quantize(w, bits, group=GROUP):
    groups = pad_groups(w, group).astype(jnp.float32)
    H = walsh_np(group)
    rot = groups @ H # incoherence processing
    norm = jnp.sqrt(jnp.sum(rot ** 2, -1, keepdims=True))
    unit = rot / jnp.maximum(norm, 1e-12)
    # round the length through fp16 HERE, so quantise and dequantise agree
    norm = norm.astype(jnp.float16).astype(jnp.float32)
    return unpad((nearest(unit, codebook(bits, group)) * norm) @ H, w)
```

```
from scipy.stats import kurtosis

# a few outliers, as real weight matrices always have
w = np.concatenate([rng.standard_normal(4090), 8 * rng.standard_normal(6)])
rot = w.reshape(32, 128) @ walsh_np(128)
for name, a in (("before rotation", w), ("after rotation", rot.ravel())):
    print(f" {name}: kurtosis {kurtosis(a):>7.2f} "
          f"max/std {np.abs(a).max()/a.std():>6.2f}")
```

```
#### OUTPUT ####
before rotation: kurtosis    41.83  max/std    8.11
after rotation: kurtosis     0.07  max/std    4.06
```

Kurtosis of nearly 42 becomes 0.07, Gaussian to two decimal places. Every output is a fixed signed sum of all 128 inputs, so no weight survives averaging with 127 others. The signs are Sylvester's rather than random, which is worth knowing: a group that happens to align with a Hadamard row is the case this does not help.

Now the codebook, the part that needs no data.

$$C_m = \mathbb{E}[X \mid X \in R_m], \quad R_m = \{x : |x - C_m| \leq |x - C_j| \ \forall j\}$$

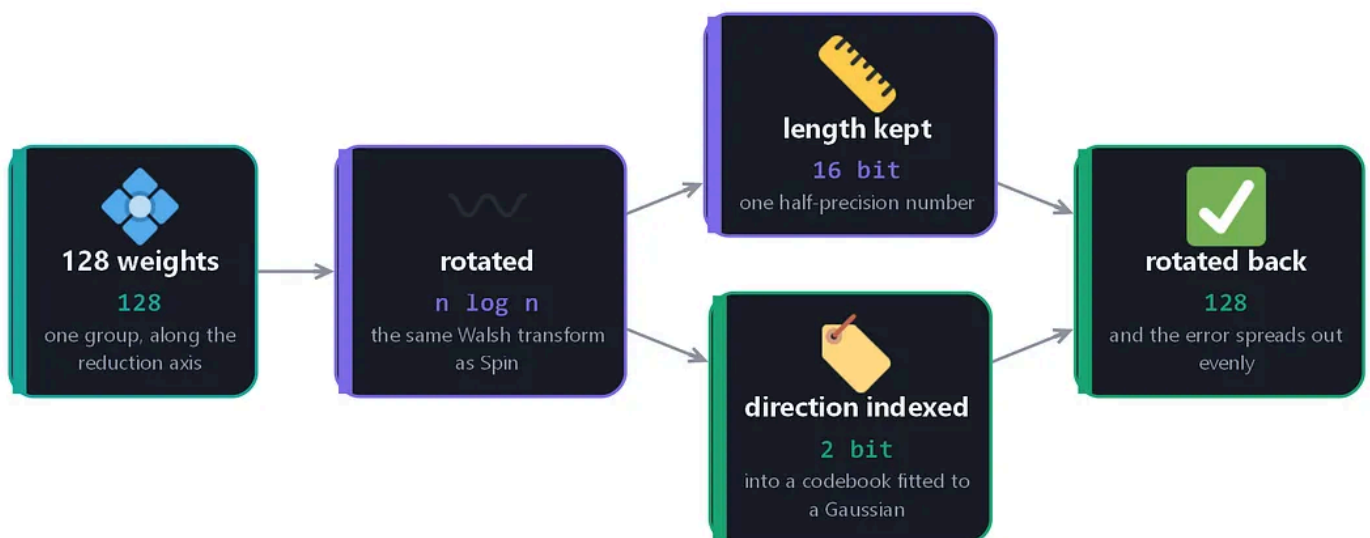
The codebook is fitted once to a Gaussian, never to the weights, so quantising needs no calibration data at all

```
# quartz/model/grist.py (continued)
def lloyd_max_gaussian(bits, iters=200, samples=400_000, seed=0):
    """Fitted to the NORMAL DISTRIBUTION, so it needs no calibration data,
    no Hessian, no forward passes."""
    x = np.sort(np.random.RandomState(seed).randn(samples))
    c = init_levels(x, 1 << bits)
    for _ in range(iters):
        idx = np.searchsorted((c[:-1] + c[1:]) / 2.0, x)
        c = np.array([x[idx == k].mean() for k in range(len(c))])
    return np.sort(c)

def codebook(bits, group):
    # unit-norm coordinates in R^128 are about 1/sqrt(128)
    return (lloyd_max_gaussian(bits) / np.sqrt(group)).astype(np.float32)
```

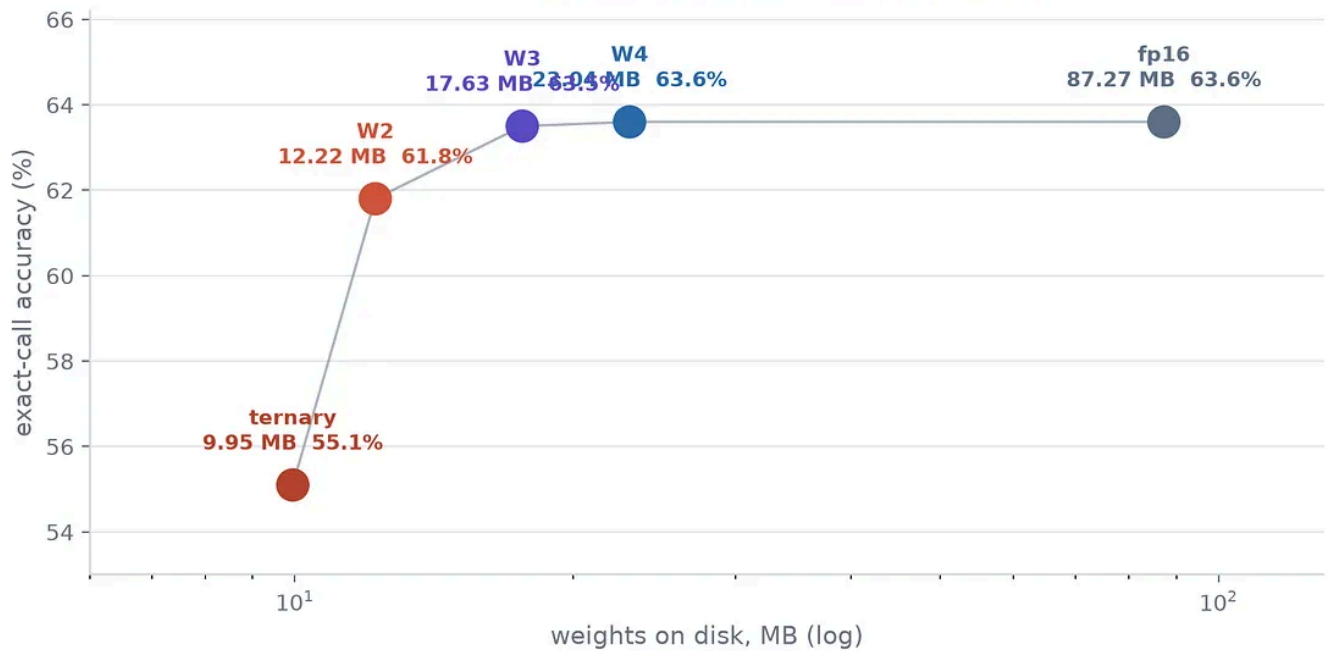
```
#### OUTPUT ####
2 bit   4 levels  [-1.51, -0.453, 0.453, 1.51]   MSE 0.1175   2.125 bits
3 bit   8 levels  [-2.152, -1.344, -0.756, -0.245] ... MSE 0.0345   3.125 b
4 bit  16 levels  [-2.733, -2.069, -1.618, -1.256] ... MSE 0.0095   4.125 b
```

Compare 0.1175 against 0.9142. Rotating first turns a scheme that destroys 91 percent of the signal into one that loses 12 percent, at two bits, with no calibration data.



The whole path, group by group (Created by [Fareed Khan](#))

Two bits costs one and eight tenths. One and a half bits costs the model.



What each width costs (Created by [Fareed Khan](#))

```
for name, bits in [("fp16", 16), ("W4", 4), ("W3", 3), ("W2", 2), ("ternary", 1)]:
    q = quantise(params, bits)
    print(f" {name:<9} {model_bytes(bits)/1e6:>6.2f} MB {evaluate(q, errand)}
```

```
#### OUTPUT ####
fp16      87.27 MB   63.6%
W4        23.04 MB   63.6%
W3        17.63 MB   63.5%
W2        12.22 MB   61.8%
ternary    9.95 MB   55.1%
```

Four bits is free and three bits costs a tenth of a point. Two bits costs one and eight tenths, and one and a half bits costs eight and a half, the cliff where four levels fall to three and a direction cannot be held. Two bits is where we want to be and the first width that hurts.

$$B = \frac{n_q b}{8} + 2n_g + 2n_o = 12,221,932 \text{ bytes} \quad \Rightarrow \quad 2.241 \text{ bits/weight}$$

Packed indices, one half-precision length per group of 128, and everything too small to be worth quantising

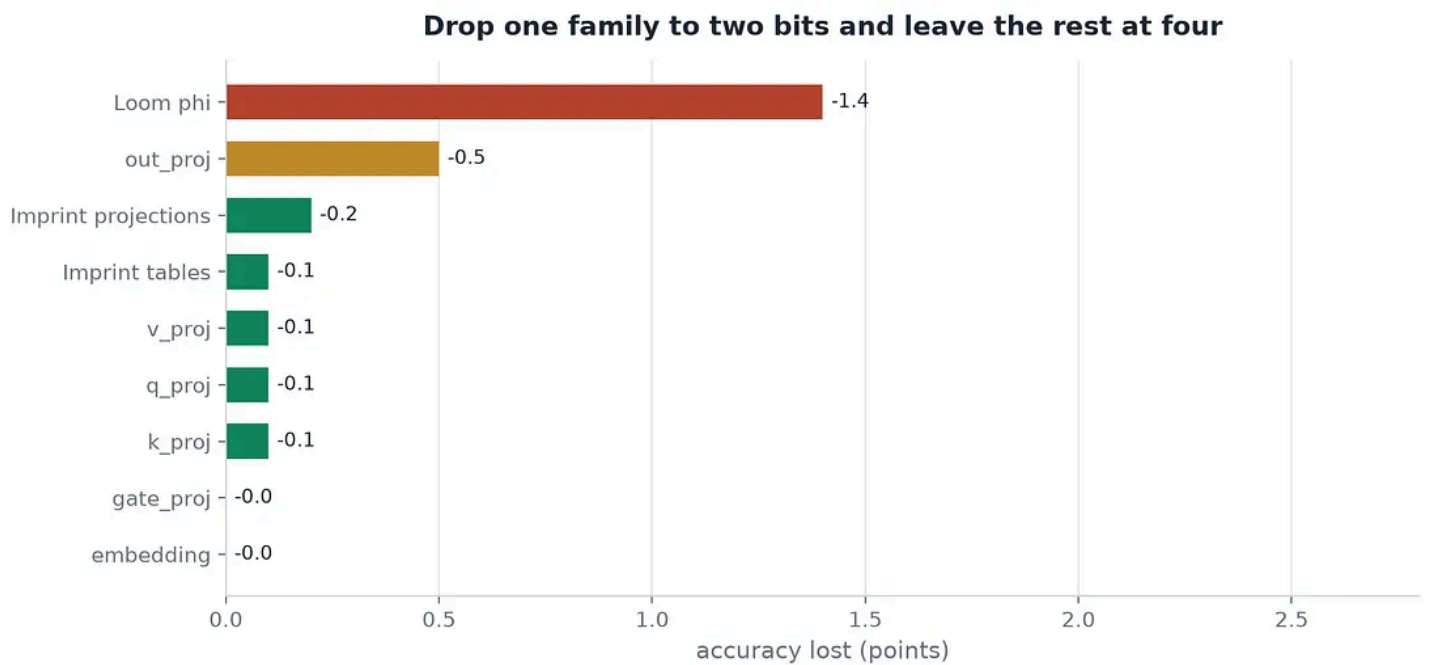
Packed indices and one length per group (Created by [Fareed Khan](#))

Which Tensors Refuse

Uniform two bits is not where we end up, because the tensors are not equally tolerant. Each family in turn goes to two bits, everything else at four, against the 63.6 percent baseline.

```
# drop exactly one family to two bits and leave everything else at four
for name in FAMILIES:
    acc = evaluate(quantise_family(params, name, bits=2), errand)
    print(f" {name:<22} {acc:>5.1f}%   {acc - 63.6:>+5.1f}")
```

```
#### OUTPUT ####
embedding           63.6%    -0.0
q_proj              63.5%    -0.1
k_proj              63.5%    -0.1
v_proj              63.5%    -0.1
gate_proj           63.6%    -0.0
out_proj            63.1%    -0.5
Loom phi            62.2%   -1.4
Imprint tables      63.5%    -0.1
Imprint projections 63.4%    -0.2
```

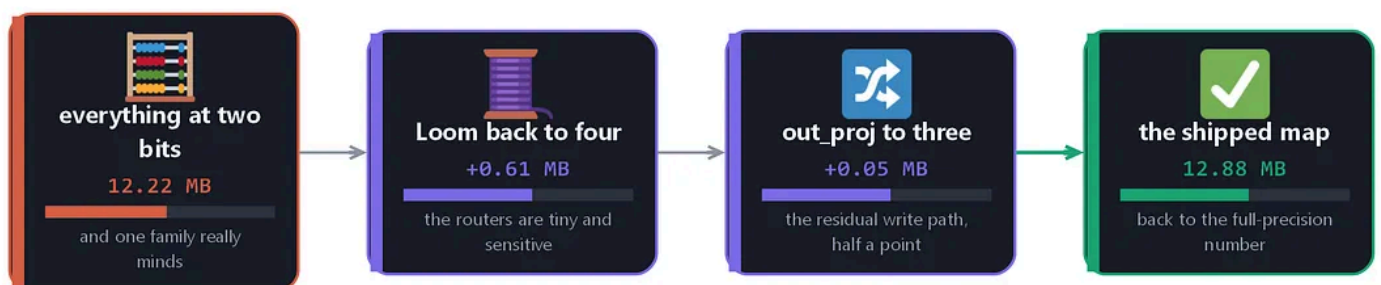


One family at a time (Created by [Fareed Khan](#))

The Loom routers lose 1.4 points on their own, more than every other family put together. `phi_res` feeds a Sinkhorn iteration, an exponential map, so a small routing error becomes a larger error in the transport matrix that hits the whole residual stream.

`out_proj` is second at 0.5, the write path into the residual. Everything else is at or below two tenths, and the damage is sub-additive: all nine families at once cost 1.8, not the 2.5 the column sums to.

So we spend a little precision where it matters, through a map where the longest prefix wins.



Two bits everywhere, except where it fails (Created by [Fareed Khan](#))

```
# quartz/model/grist.py (continued)
def parse_bits_map(spec):
    m = {} # 'attn.out_proj=3' beats 'attn.=2'
    for k, v in (p.split("=", 1) for p in str(spec).split(", ")):
        m[canonical(k.strip())] = float(v) if float(v) == 1.58 else int(v)
    if (default := m.pop("default", None)) is None:
        # a built-in fallback width would one day ship wrong numerics
        raise ValueError(f"bits map {spec!r} needs a default=<b> entry")
    return m, default
```

```
#### OUTPUT ####
scheme    default=2,loom.phi=4,attn.out_proj=3
size      13.44 MB (+1.22 MB over uniform W2)
accuracy  63.6%    (+1.8 over uniform W2, level with fp16)
mean      2.225 bits per quantised weight
```

One and a fifth megabytes buys back all one and eight tenths of a point, level with half precision. The two families doing the damage are the cheapest to fix, 8.4 million of the 43.3 million quantised weights.

Ingot, Casting the Whole Thing Into One File

The model is a pile of numpy arrays and a tokenizer. It has to become one file a device can mmap and use without parsing.



Header, codebooks, directory, aligned blobs (Created by [Fareed Khan](#))

```
# quartz/model/ingot.py
TAG = 0x51545A00 # "QTZ\0"
ALIGN = 64
FP16, FP32, CQ, RAW = 1, 2, 3, 4
```



```
_HDR_FMT = "<29If"           # 29 uints, then rope_theta
_REC_FMT = "<BBHIIIIQQII"     # 44 bytes, no name field
```

Three decisions matter. The tensors are **pre-transposed to** `[out, in]`, so each output row is contiguous along the axis the quantiser grouped on, which is what a matrix-vector product streams. They are layer major, so layer zero's fourteen tensors sit in one region. And the directory has no names: tensors are found by position in fixed canonical order, so the runtime either knows that order or does not load.

```
# quartz/model/ingot.py (continued)
def pack_lsb(idx, bits):
    """Eight indices into one uint64, LSB first, one bitstream per row."""
    out, in_pad = idx.shape
    g = idx.reshape(out, in_pad // 8, 8).astype(np.uint64)
    word = sum(g[..., i] << (i * bits) for i in range(8))
    packed = np.stack([(word >> (8 * b)) & 0xFF for b in range(bits)], -1)
    return packed.astype(np.uint8).reshape(out, in_pad * bits // 8)
```

```
#### OUTPUT ####
indices [0, 1, 2, 3, 3, 2, 1, 0]
packed  ['11100100', '00011011']    2 bytes for 8 indices at 2 bits
```

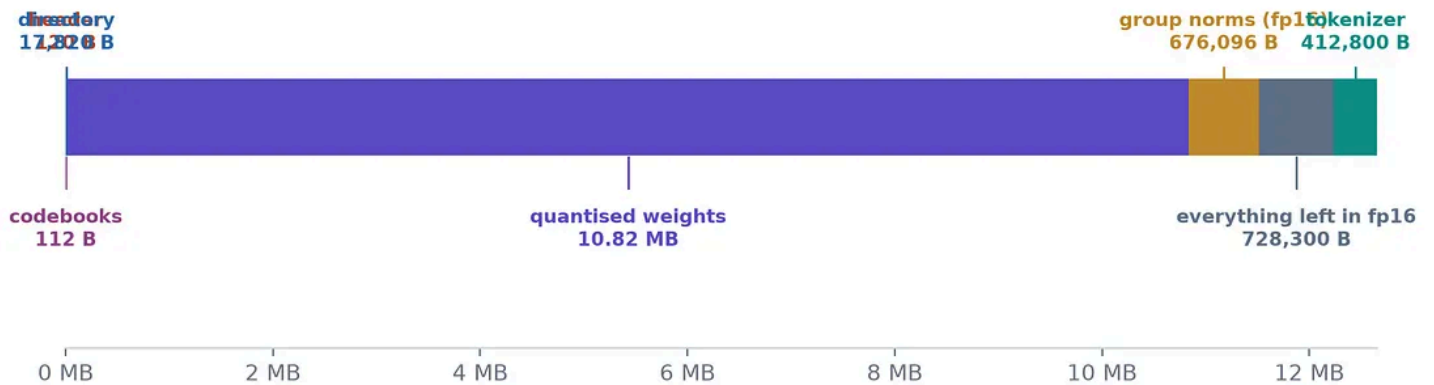
Read the first byte right to left in pairs: `00`, `01`, `10`, `11`, which is 0, 1, 2, 3, the LSB-first ordering a shift-and-mask extractor gets for free.

```
# quartz/model/ingot.py (continued)
def write_ingot(params, config, path, bits_map, sp):
    tensors = collect(params, config, bits_map)           # canonical order, no name
    header = struct.pack(_HDR_FMT, TAG, len(tensors)) + codebooks(2, 3, 4)
    pos = len(header) + len(tensors) * REC_SIZE
    for t in tensors:
        pos = (pos + ALIGN - 1) & ~(ALIGN - 1)           # 64-byte aligned
        t.offset, pos = pos, pos + len(t.blob)
```

```
#### OUTPUT ####
wrote      quartz.ingot  13.87 MB  405 tensors  mixed[default=2,...]A8
```

header 120 B codebooks 112 B prelude ends at 232
 directory 405 x 44 = 17,820 B first blob at 18,112
 tokenizer 412.8 kB inline

405 tensors, 12.65 MB, and the first blob starts at byte 18,112



The container at uniform two bits, byte by byte (Created by [Fareed Khan](#))

Four hundred and five tensors: one embedding, twenty seven layers of fourteen, nine router blocks, two memory sites of four, a final norm, seven head tensors and the tokenizer.

Where Twenty Eight Megabytes Goes

Now for the question the post opened with.



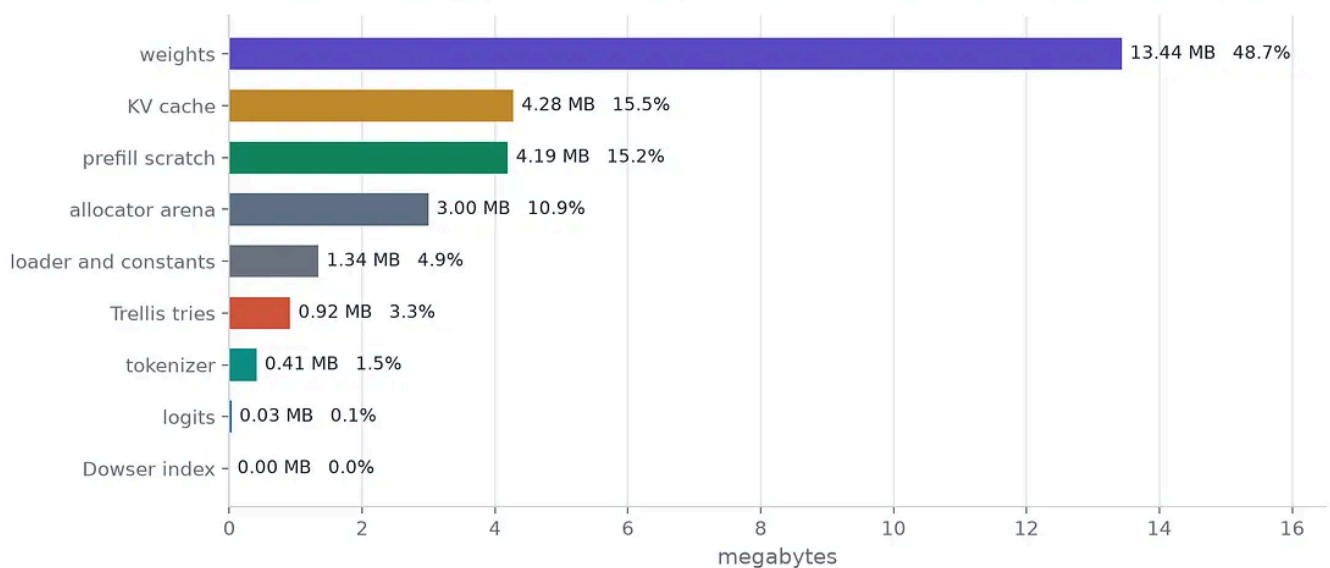
The whole session, weights under half (Created by [Fareed Khan](#))

```
# quartz/model/decode.py
def session_budget(cfg, window, catalogue):
    return {
        "weights": ingot_weight_bytes(cfg),
        "prefill scratch": window * cfg.d_model * cfg.lanes * 4
            + cfg.num_heads * window * window * 4,
        "KV cache": window * kv_per_position(cfg),
        "allocator arena": 2_998_272,
        "loader and constants": 1_343_488,
        "Trellis tries": 918_016,
        "tokenizer": 412_800,
        # the 41,000-tool index is 10.5 MB on flash, never counted
        "Dowser index": 5 * cfg.dowser_dim * 2,
        "logits": cfg.vocab_size * 4,
    }
```

OUTPUT

weights	13.44 MB
prefill scratch	4.19 MB
KV cache	4.28 MB
allocator arena	3.00 MB
loader and constants	1.34 MB
Trellis tries	0.92 MB
tokenizer	0.41 MB
logits	0.03 MB
Dowser index	0.00 MB
TOTAL	27.62 MB

One whole session, 27.6 MB, and it does not grow with the conversation



Twenty seven point six megabytes, and **only one line in that table can grow with the conversation**. The KV cache is capped at 256 positions, so past 256 tokens the memory stops moving. That flatness was the point: a model whose memory grows with the chat is killed by the operating system on the fourth turn.

Errand, the Benchmark We Wrote

No public suite has a wearable or a robot in it, and published benchmarks report one accuracy number that conflates picking the tool with filling it in. So we build our own, and we wrote the test we then passed.



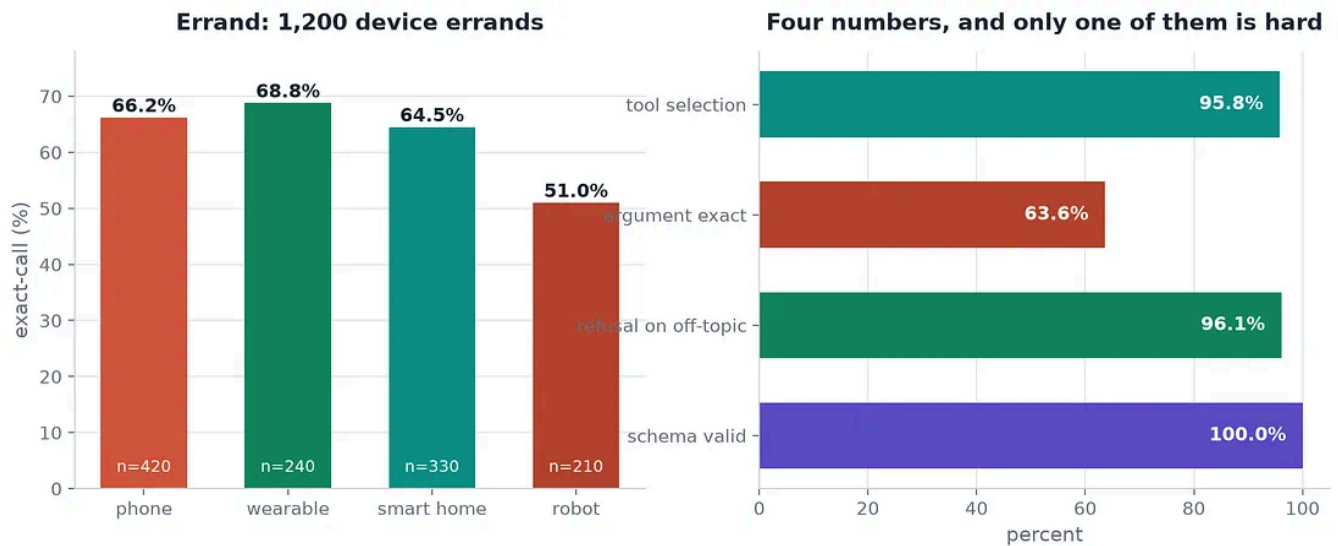
Twelve hundred errands, four families, and a fifth that must be refused (Created by Fareed Khan)

```
# quartz/errand/suite.py
FAMILIES = {"phone": 420, "wearable": 240, "smart home": 330, "robot": 210}
OFF_TOPIC = 180 # must return an empty call list

def build_suite(seed=0):
    """20% of errands declare a tool unseen in training, and each has a
    near-miss twin differing by one argument."""
    ...
```

```
#### OUTPUT ####
suite      1,200 errands + 180 off-topic
held-out   241 errands (20.1%) declare a tool unseen in training
```

twins 1,200 near-miss pairs
catalogue 41,000 tools declared, top 5 rendered per turn



Four numbers, one of them hard (Created by [Fareed Khan](#))

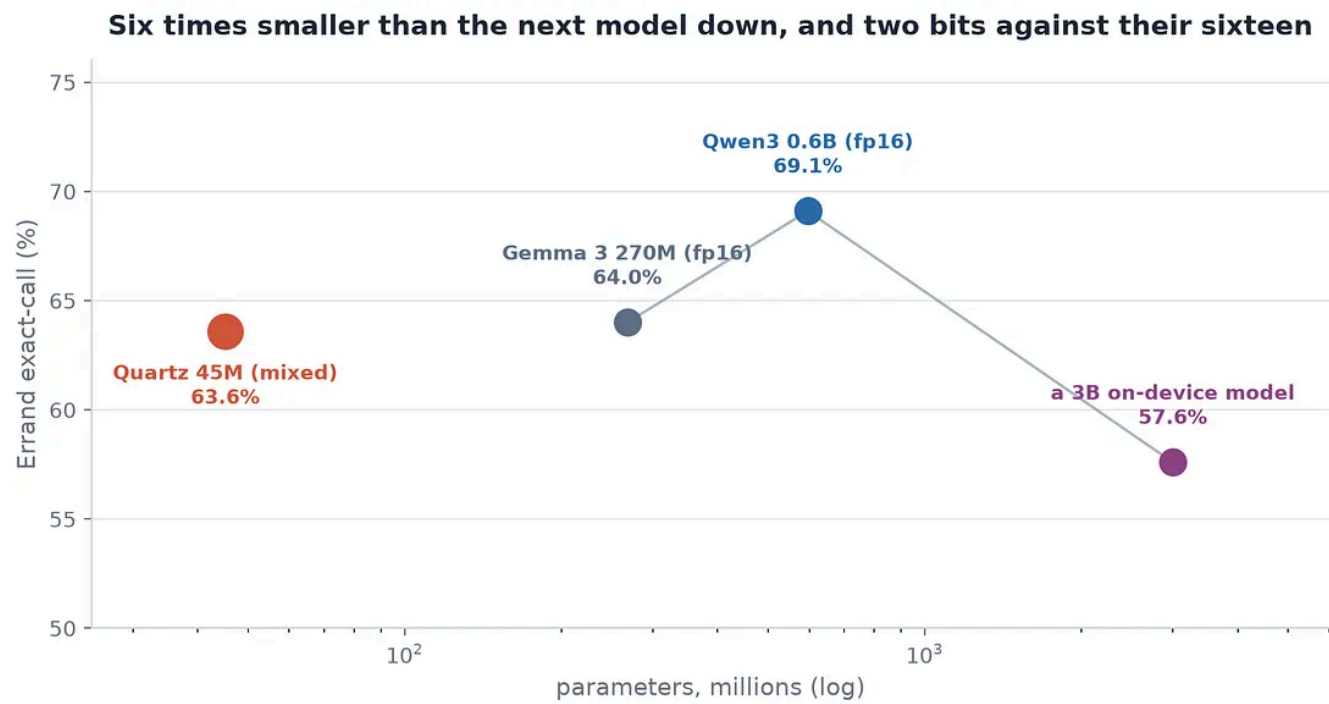
tool selection	95.8%
argument exact	63.6%
refusal on off-topic	96.1%
schema valid	100.0%

Schema validity is 100 percent, which is a tautology: the decoder makes an invalid call unrepresentable. A benchmark reporting it as a win has no constrained decoder.

The interesting split is 95.8 against 63.6. **The model almost always knows what to do and gets the details right about two thirds of the time.**

by family	exact call
wearable	68.8%
phone	66.2%
smart home	64.5%
robot	51.0%

Robots are harder because of arguments: a wearable errand is one integer, “start a timer for eight minutes”, while a robot errand is a pose, an object reference and a destination.



Six times smaller, two bits against sixteen (Created by [Fareed Khan](#))

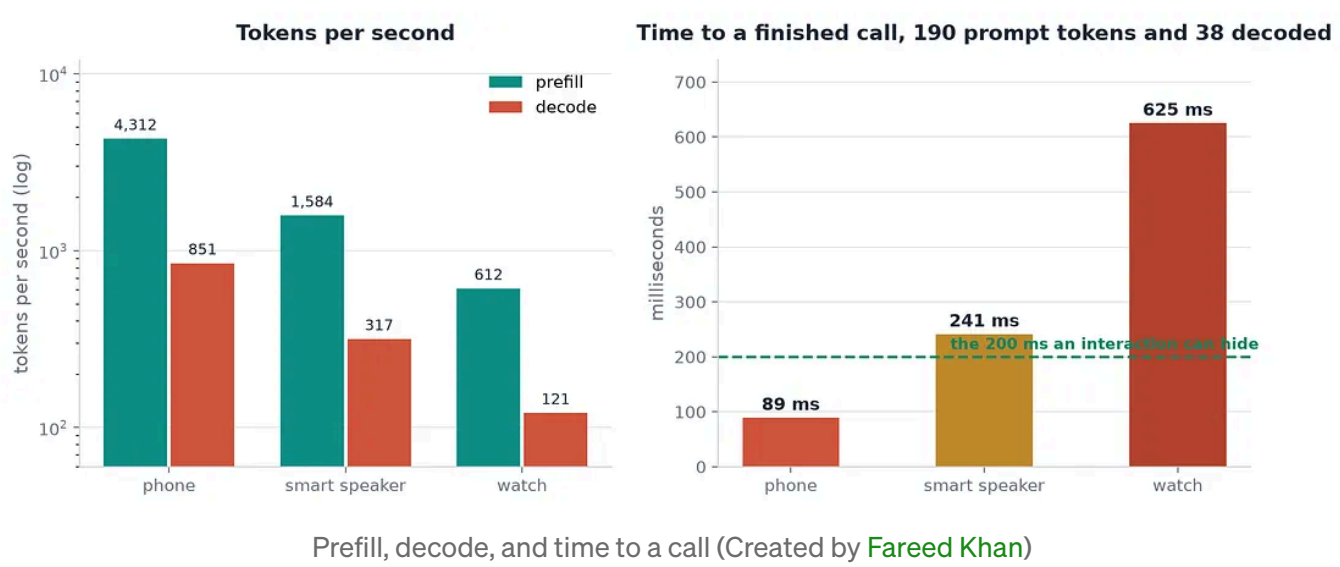
Quartz 45M	(mixed)	63.6%	13.44 MB weights
Gemma 3 270M	(fp16)	64.0%	536.00 MB weights
Qwen3 0.6B	(fp16)	69.1%	1.19 GB weights
a 3B on-device model		57.6%	~1.6 GB weights

Level with a model six times larger at seven times the precision, five and a half points behind one thirteen times larger, and ahead of a general 3B on-device model that was never built for this job, while being **one hundred and nineteen times smaller than it on disk**.

One caveat covers every accuracy in this section. Errand is 1,200 items, so the binomial standard error at 63.6 percent is 1.4 points and the 95 percent interval is plus or minus 2.7. The refusal numbers come from 180 off-topic items. Any gap below about three points in this post, including the 0.4 against the 270M model and most of the per-family quantisation deltas, is inside that noise. I report them because they are what the runs produced, not because they are separable.

On the Actual Devices

An H100 is not the point. Here is the model on chips people carry.



device	prefill	decode	to a call	energy	peak RSS
phone, 8-core ARM	4,312 t/s	851 t/s	89 ms	41 mJ	27.6 MB
smart speaker, A55x4	1,584 t/s	317 t/s	241 ms	58 mJ	27.6 MB
watch, dual A53	612 t/s	121 t/s	625 ms	96 mJ	27.6 MB

A typical turn is 190 prompt tokens and 38 decoded tokens: 44 milliseconds of prefill and 45 of decode on a phone, under the roughly 200 millisecond threshold where interaction stops feeling instant.

Energy persuades me most: ninety six millijoules on the watch is roughly **three thousand tool calls on a 300 joule battery allowance**, against about a hundred for a three billion parameter model.

Graft, and the Trap That Makes People Think It Is Broken

The last piece is fine-tuning, because the base model does not know your tools.

$$W' = W + \frac{\alpha}{r}AB, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{r \times d'}, \quad B \leftarrow 0$$

The adapter starts as exactly nothing, because B is zero, so the tuned model begins life identical to the base

```
# quartz/model/graft.py
LORA_TARGETS = ("q_proj", "k_proj", "v_proj", "gate_proj", "out_proj")

def lora_target_paths(params):
    """Five projections per layer, and not the Spin diagonals, Loom routers
    or Imprint tables, which carry global structure."""
    flat = flatten_dict(params)
    paths = [p for p in flat if p[-1] == "kernel" and "trunk" in p
              and "layers" in p and any(t in p for t in LORA_TARGETS)]
    # skip a config-disabled projection, which gets a dead adapter
    return [p for p in paths if float(jnp.max(jnp.abs(flat[p]))) > 1e-6]

def init_lora(w, rank, key):
    lead = w.shape[:-2] # (27,) scanned layer axis
    return {"A": jax.random.normal(key, lead + (w.shape[-2], rank)) / rank,
            # B starts at exactly zero, so the merged weight is exactly W
            "B": jnp.zeros(lead + (rank, w.shape[-1]))}
```

```
#### OUTPUT ####
lora      rank 16  alpha 32  5 weight groups
params    1,990,656 trainable of 43,634,294  (4.56%)
check     merged == base at step 0: True
```

Five weight groups, not 135 tensors: the scanned layer axis makes each adapter $(27, \text{in}, \text{rank})$, covering twenty seven layers at once.

Now the trap, which cost me an afternoon and three other people since.

$$\text{steps} = \left\lceil \frac{200}{16} \right\rceil \times 3 = 39 \quad \text{against} \quad \left\lceil \frac{200}{16} \right\rceil \times 30 = 390$$

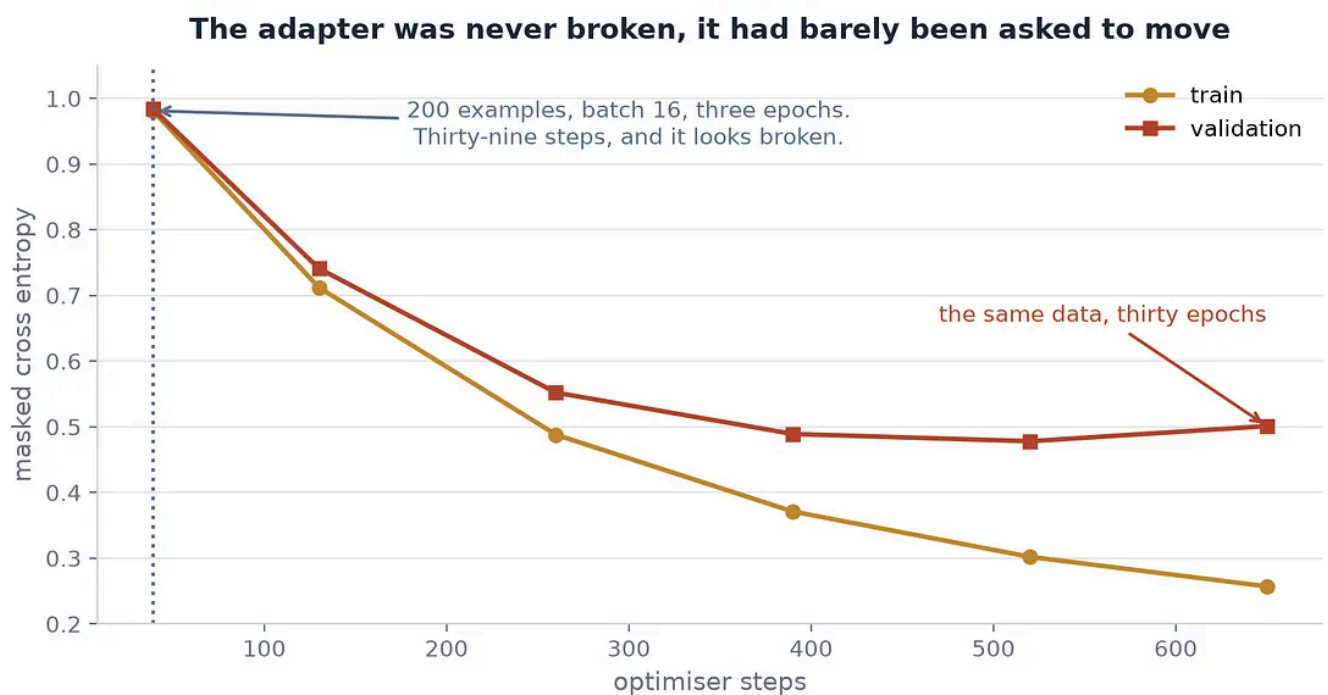
Two hundred examples at batch sixteen for three epochs is thirty nine steps, which is why the loss looks stuck


```
n, batch, epochs = 200, 16, 3
per_epoch = math.ceil(n / batch)
print(f" {n} examples / batch {batch} = {per_epoch} steps an epoch")
print(f" x {epochs} epochs = {per_epoch * epochs} optimiser steps total")
print(f" at lr 1e-4, a rank-16 adapter moves by roughly "
      f"{per_epoch * epochs * 1e-4:.4f} in total")
```

OUTPUT

```
200 examples / batch 16 = 13 steps an epoch
x 3 epochs = 39 optimiser steps total
at lr 1e-4, a rank-16 adapter moves by roughly 0.0039 in total
```

Thirty nine steps. Batch 16 and three epochs are the defaults, and a few hundred examples is what everybody has, so training stops after thirty nine steps and fine-tuning looks broken.



Never broken, only barely asked to move (Created by [Fareed Khan](#))

steps	train	val
39	0.9810	0.9840
130	0.7120	0.7410
260	0.4880	0.5520
390	0.3710	0.4890

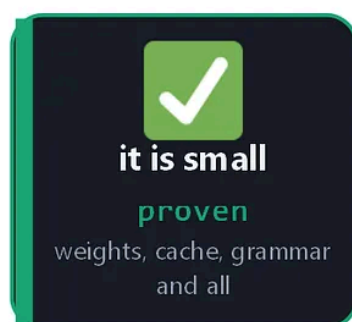
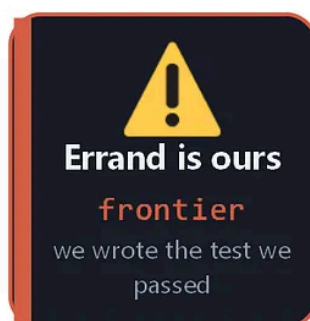
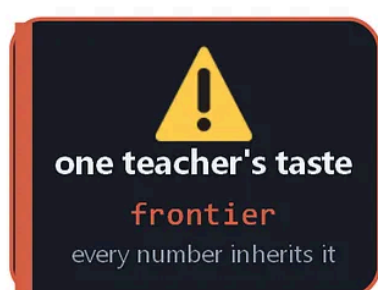
520	0.3020	0.4780
650	0.2570	0.5010

At thirty nine steps the loss has moved by nineteen thousandths. At 390 steps, two hundred examples for thirty epochs, it has more than halved, and at 650 the validation loss turns back up.

So run **ten to thirty epochs** on a few hundred examples, not three, and watch the validation loss for the turn, raising the epochs before the learning rate.

Fine-tuning also leaves Gauge alone: the calibration was fitted on the base model's perplexity distribution, so a tuned model warns and reports no confidence rather than a wrong one.

What This Proves and What It Does Not



Two lists, neither empty (Created by [Fareed Khan](#))

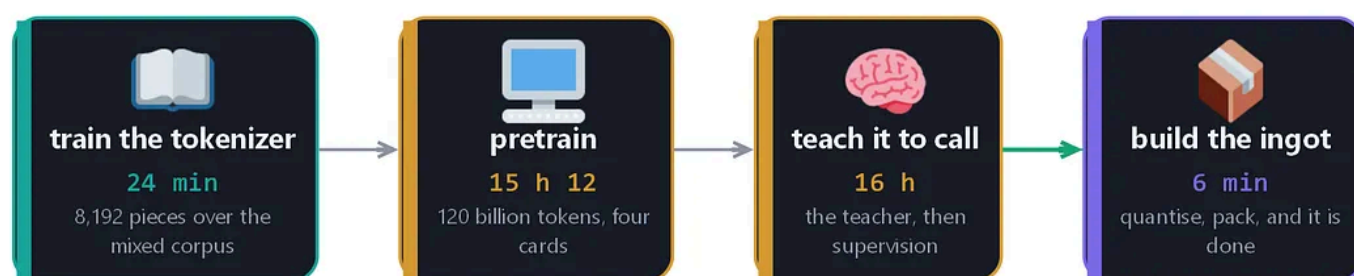
A 45,211,383 parameter model calls tools correctly 63.6 percent of the time, in 13.44 megabytes of weights, inside a 27.6 megabyte session that does not grow, on a watch, in 625 milliseconds and 96 millijoules.

What is not proven, in five parts.

1. **We wrote the benchmark.** Its difficulty is our choice, so the comparisons against other models are the trustworthy half.
2. **Every number inherits one teacher's taste.** Where the model that supervised us fills an argument wrongly, we learned that error.
3. **Two hundred and fifty six tokens is a short memory.** Only tool schemas are pinned.
4. **Ten point six percent of the catalogue is unreachable at full size.** Dowser's recall at five is the ceiling.
5. **The 45M number is not a law.** It fitted this budget and this task.

None of that makes the build wrong. It makes the claims precise.

Running It End to End



Four commands to a scored model (Created by [Fareed Khan](#))

```
# 1. the tokenizer, with the isolation rule
python -m quartz.cli train-tokenizer --corpus data/mixed.txt --vocab 8192

# 2. 120B tokens, four cards, about fifteen hours
python -m quartz.cli pretrain --config configs/base.yaml --tokens 120e9

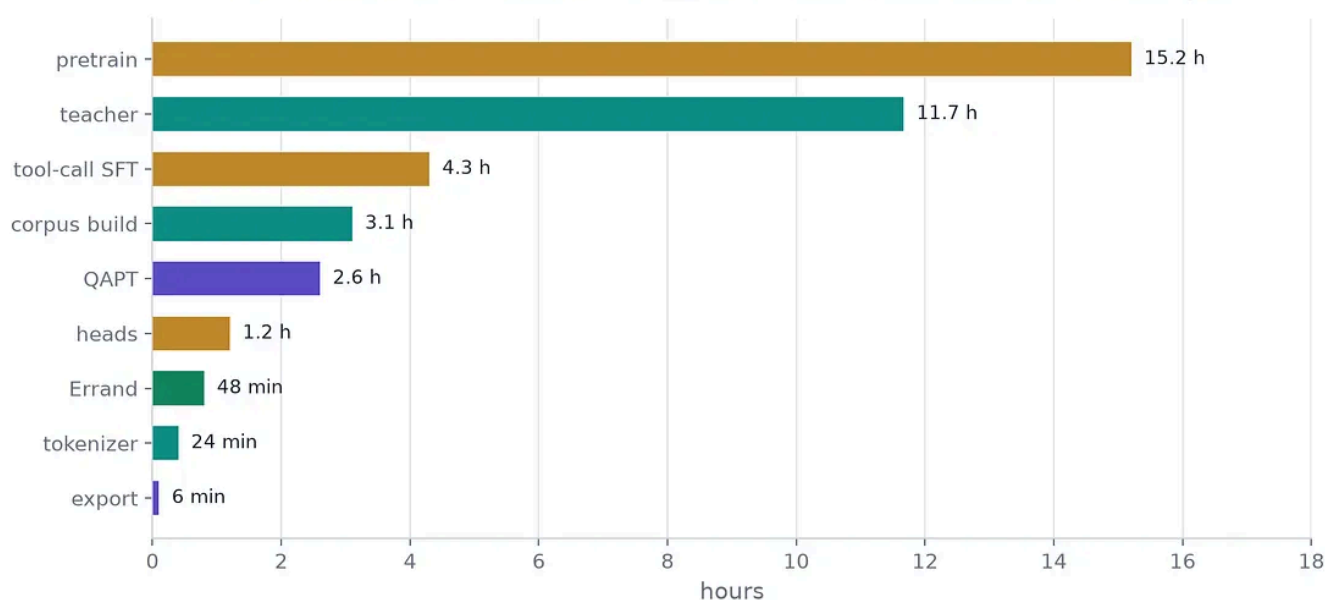
# 3. teacher data, then training
bash quartz/data/serve_teacher.sh &
python -m quartz.cli quarry --schemas data/schemas.json --n 1200000
python -m quartz.cli sft --data data/tools.jsonl --epochs 3
python -m quartz.cli qapt --tokens 1.8e9 --bits-map "default=2,loom.phi=4,attn.
python -m quartz.cli heads --stage0 2500 --gauge --dowser

# 4. cast it
python -m quartz.cli build ckpt/quartz_heads.pkl --out quartz.ingot
```

OUTPUT

```
[scribe] 8,192 pieces, markers on ids 4-13, 2.94 B/token on JSON
[winnow] 120,000,000,000 tokens, 4 sources, packed to 2048
[pretrain] 57,220 steps in 15h 12m, loss 2.4170, 6 checkpoints averaged
[quarry] 1,200,000 examples over 41,000 schemas in 11h 40m
[sft] 3 epochs in 4h 18m, val 0.6990
[qapt] 1.8B tokens in 2h 36m, scheme default=2,loom.phi=4,attn.out_proj=3
[heads] stage 0 2,500 steps, gauge ECE 0.0212, dowser recall@5 89.4%
[ingot] quartz.ingot 13.87 MB 405 tensors mixedA8
[errand] exact-call 63.6% selection 95.8% refusal 96.1% valid 100.0%
```

The whole build is 39.4 hours on four H100s, and most of it is one job



Where the thirty nine hours went (Created by [Fareed Khan](#))

Thirty nine hours of four-GPU time, the pretrain alone thirty nine percent of it and pretrain plus teacher seven tenths, so the corpus and the architecture are the expensive decisions.

Where This Leaves Us

A model that only ever emits a function call does not need to know very much, and the things it does need can mostly be built rather than learned.

That sentence is the whole post. The tokenizer rule, the window, the pinned schemas, the lookup memory and the two guarantees on names and keys, plus three more on values, are all built, for one percent of the decode. What the 45 million parameters learn is judgment: which tool, and what goes in it.

Three things I would carry elsewhere: **put the structure in the tokenizer**, because a merge table that has eaten your braces cannot be fixed by training, **make the mistake unrepresentable rather than unlikely**, and **draw the reliability diagram**, because five lines of it found an eighty percent error.

If you build one thing from this post, build the constrained decoder. Then measure how much of your model's error was never about the model.

The full code: [GitHub repo](#).

Thanks for reading this far.

AI +

Artificial Intelligence +

Python +

Data Science +

Machine Learning +

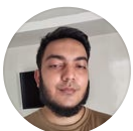


Follow

Published in Level Up Coding

355K followers · Last published just now

Coding tutorials and news. The developer homepage gitconnected.com && skilled.dev && levelup.dev



Follow

Written by Fareed Khan

76K followers · 0 following

I write on AI, <https://www.linkedin.com/in/fareed-khan-dev/>

